Harare Institute of Technology
HIT
success through innovation

Innovation and Technopreneurial University

SCHOOL OF INFORMATION SCIENCES AND TECHNOLOGY

BYZANTINE FAULT-TOLERANT MULTI-AGENT SYSTEMS FOR AUTONOMOUS RUNTIME PROGRAM REPAIR IN E-COMMERCE SYSTEMS

A PROJECT REPORT SUBMITTED IN PARTIAL FULFILMENT OF THE

REQUIREMENTS FOR THE AWARD OF THE DEGREE OF

MASTER OF TECHNOLOGY IN SOFTWARE ENGINEERING

BY

MILLICENT MUFAMBI (H240624A)

UNDER THE GUIDANCE OF:

MR W. MAKONDO

MAY 2026

Harare Institute of Technology
HIT
success through innovation
Innovation and Technopreneurial University

School of Information Science and Technology
Post Graduate Unit
Master Of Technology In Software Engineering

DECLARATION BY THE CANDIDATE

I, Millicent Mufambi (H240624A), hereby declare that the project report entitled "Byzantine Fault-Tolerant Multi-Agent Systems for Autonomous Runtime Program Repair in E-Commerce Systems" carried out by me under the guidance of Mr W. Makondo is submitted in partial fulfilment of the requirements for the award of the degree of Master of Technology in Software Engineering. This is a record of bona-fide work carried out by me and the results embodied in this project have not been reproduced/copied from any source.

The results embodied in this project report have not been submitted to any other University or Institute for the award of any master's degree, degree, or diploma.

Signature

M. Mufambi (H240624A)

Date: 26/05/2026

School of Information Science and Technology

Post Graduate Unit

Master Of Technology In Software Engineering

CERTIFICATE BY THE SUPERVISOR

This is to certify that the project report entitled "Byzantine Fault-Tolerant Multi-Agent Systems for Autonomous Runtime Program Repair in E-Commerce Systems" being submitted by Millicent Mufambi (H240624A) in partial fulfilment of the requirements for the award of the degree of Master of Technology in Software Engineering, is a record of bona-fide work carried by him/her. The results embodied in this project report have not been submitted to any other University or Institute for the award of any other master's degree, degree, or diploma.

Signature Mr W. Makondo (Supervisor)

Date:

School of Information Science and Technology

Post Graduate Unit

Master Of Technology In Software Engineering

CERTIFICATE BY THE HEAD OF THE DEPARTMENT

This is to certify that the project report entitled "Byzantine Fault-Tolerant Multi-Agent Systems for Autonomous Runtime Program Repair in E-Commerce Systems" being submitted by Millicent Mufambi (H240624A) in partial fulfilment of the requirements for the award of the degree of Master of Technology in Software Engineering, is a record of bona-fide work carried by him/her. The results embodied in this project report have not been submitted to any other University or Institute for the award of any other master's degree, degree, or diploma.

Signature Initials & Surname (Title)

Date:

ABSTRACT

When software breaks in a live e-commerce system, the bill arrives quickly, and a hand-written patch rarely ships fast enough to stop the bleeding. Multi-agent large-language-model (LLM) systems can now write plausible patches on their own, yet they talk their way to a decision and set no limit on how many agents may be confidently wrong before a bad fix is shipped. This project builds and tests BFT-MAS, a Byzantine fault-tolerant multi-agent system that adapts the Practical Byzantine Fault Tolerance (PBFT) protocol to a mixed group of LLM repair agents, backed by an isolated execution sandbox and e-commerce integrity checks. A Claude analyzer and a GPT-4o healer propose a fix that four independent, model-diverse validators (Claude-Haiku, GPT-4o-mini, llama3.1:8b and mistral:7b) then vote on under PBFT ($n = 3f + 1 = 4$, quorum = 3) before a sandbox stage, measured against a single-agent baseline on forty synthetic Python bugs, twenty real bugs from BugsInPy, and a thirty-bug e-commerce suite. The study demonstrates the core Byzantine property directly: on an independent-replica realisation, a malicious primary that equivocates makes a naive vote split the honest replicas onto different fixes, while the PBFT quorum certificates preserve agreement and reject forged messages; fault injection and an $f = 2$ study further show the quorum masks faulty votes up to the $3f + 1$ bound. Repair results are reported as preliminary: on the real-world subset the unsafe-fix rate fell directionally from 75% to 50% and the repair rate rose from 25% to 50%, but neither reaches significance (McNemar exact $p = 0.125$) and the effect rests on a weak run-as-script oracle. The work delivers an open-source prototype, a repeatable harness, and the first empirical study of genuine BFT consensus applied to LLM-driven repair in e-commerce.

Keywords: Byzantine fault tolerance, multi-agent systems, large language models, automated program repair, PBFT consensus, e-commerce reliability, software engineering.

ACKNOWLEDGEMENTS

My deepest thanks go to my supervisor, Mr W. Makondo, whose guidance, patience and frank criticism shaped this project from the first rough proposal to the document you are reading. He pushed me to be rigorous and clear, and the work is far better for it.

I am grateful to the Department of Software Engineering and the School of Information Sciences and Technology at the Harare Institute of Technology, whose academic environment and resources made the work possible.

Finally, to my family and friends: thank you for the encouragement that carried me through the Master of Technology programme. Whatever errors remain in this report are mine alone.

TABLE OF CONTENTS

The headings below are linked to an automatic table of contents. To populate the page numbers, open the document in Microsoft Word, right-click the field below and select “Update Field”, then “Update entire table”.

Right-click and choose "Update Field" to build the Table of Contents.

CHAPTER 1: INTRODUCTION AND BACKGROUND TO THE STUDY

1.1 Introduction

Almost everything about modern commerce now runs on software. Retail front-ends, payment gateways, inventory services and order-management systems stay up around the clock, and the correctness of their code feeds straight into customer trust, revenue and reputation. Let a defect slip into production and the damage shows up at once: a shopper is charged twice, a stock count slides below zero, a refund sails past its cap, or an order is confirmed while the payment is still pending. The Consortium for Information and Software Quality (CISQ, 2024) puts the cost of critical production failures at roughly one million United States dollars an hour for large firms, and as much as five million an hour for busy retailers during peak trading. How quickly, and how safely, a defect can be found and fixed is therefore not a back-office nicety; for an online business it is close to a survival metric.

For most of software engineering's history, that find-and-fix loop has been human work. Someone is paged, reproduces the failure, reasons out the cause, writes a patch, tests it and ships it. Every one of those steps costs time, and in e-commerce time is money. Automated program repair (APR) grew up to shorten the loop by proposing fixes automatically. The first APR tools searched over program mutations or solved constraints to synthesise a patch; newer ones lean on the code-writing ability of large language models (LLMs), which produce fixes that are often plausible and sometimes right. Alongside this, capable LLMs have spawned multi-agent systems, where several model-backed agents split a software task between them, one analysing, one writing, one reviewing, in a rough imitation of a human team.

This project lives where three of those developments meet: Byzantine fault tolerance, multi-agent LLM systems and automated program repair. The motivation is one stubborn observation. Today's multi-agent LLM frameworks let their agents talk their way to a decision. There is no formal idea of agreement, and nothing caps how many agents can be confidently wrong before the group ships a bad fix. When one bad patch can scramble a payment ledger or oversell stock, that missing guarantee is a real problem. Distributed-systems researchers cracked a very similar problem decades ago with Byzantine fault tolerance, a family of protocols that still reach a correct collective decision even when some bounded number of participants misbehave in arbitrary ways. The question this project takes up is whether that guarantee can be carried over to a population of non-deterministic LLM agents doing repair, and what is won and lost in the process. The rest of this chapter sets out the background, states the problem and why it matters, lists the objectives and research questions, fixes the scope, defines the key terms, and ends with a map of the report.

1.2 Background

The theoretical bedrock here is the Byzantine Generals Problem, set out by Lamport, Shostak and Pease in 1982. They showed that to reach agreement when f participants may fail arbitrarily, including by feeding different lies to different peers, a system needs at least n = 3f + 1 participants in total. Tolerating one arbitrarily faulty participant thus takes four; tolerating two takes seven; and so on. That 3f + 1 floor is the skeleton of every fault-tolerant consensus protocol since. A second classic result tempers the first: the impossibility theorem of Fischer, Lynch and Paterson (1985) proves that in a fully asynchronous network, no deterministic protocol can promise both safety and progress in the face of even a single crash. This is why every practical protocol leans on some extra assumption about timing, randomness or trusted hardware.

Castro and Liskov (1999) turned the theory into something usable with the Practical Byzantine Fault Tolerance protocol, or PBFT. It agrees in three message rounds, pre-prepare, prepare and commit, and was the first protocol lean enough to run a real replicated service, adding only a few per cent of overhead to a networked file system. PBFT is still the reference point, and later protocols, Zyzzyva, BFT-SMaRt, RBFT, Aardvark, HotStuff and SBFT among them, mostly sharpen its speed or its resilience. All of them, though, lean on one assumption: the replicas are deterministic. Feed two correct replicas the same input and they return byte-for-byte the same output, so any disagreement is, by itself, proof of a fault. That is what makes a vote mean something in the classical world.

The second thread is the fast rise of multi-agent LLM systems. Frameworks like ChatDev (Qian et al., 2024), MetaGPT (Hong et al., 2024), AutoGen (Wu et al., 2024) and CAMEL (Li et al., 2023) wire several LLM-backed agents into teams that chase software tasks by passing natural-language messages around. They are genuinely capable, but their coordination is social, not formal: roles, peer review and discussion stand in for any guaranteed agreement rule. And there is a catch. LLM agents are stochastic; they sample their outputs with a dash of randomness, so two equally good agents can hand back different yet equivalent fixes for the same bug. That single fact breaks the old equation of correctness with identical output, and it makes a straight transfer of Byzantine consensus to the LLM world far from obvious.

The third thread is automated program repair. It moved from search-based mutation, GenProg (Le Goues et al., 2012) being the classic case, through constraint-based synthesis such as SemFix (Nguyen et al., 2013) and Angelix (Mechtaev et al., 2016), to LLM-driven methods that treat a fix as code completion (Chen et al., 2021; Xia and Zhang, 2022). One weakness shadows the whole field: the plausibility-correctness gap that Smith et al. (2015) pinned down, where a patch passes the available tests yet is still wrong. In their numbers, just two of fifty-five generated patches that passed the failing test held up on held-out tests. That precise failure, a confident patch that is quietly incorrect, is exactly what a diverse group of agents voting together, with an independent execution check behind them, ought to be able to catch. The meeting of these three threads, the safety of Byzantine

consensus, the fluency of multi-agent LLMs and the verification headache of repair, marks out the space this project works in.

1.3 Problem Statement

LLM-driven repair has become good enough that one can seriously imagine letting it fix defects in live e-commerce systems with little human oversight. The trouble is that the multi-agent frameworks driving these models hand back no formal safety guarantee. They reach decisions by conversation, set no limit on how many agents may be confidently wrong, never report their own false-positive rate, and do nothing to protect against an agent that goes rogue, whether through a model failure, a prompt-injection attack or a plain hallucination. In a domain this unforgiving, one confident but wrong fix, accepted and deployed, can break a payment, inventory or refund invariant and cost real money.

Distributed systems already has a mature answer to arbitrary failure in Byzantine fault-tolerant consensus, but every classical protocol assumes deterministic replicas and has never been adapted to, let alone tested on, a population of non-deterministic LLM agents. So there is no published system that brings formal consensus to LLM-driven repair, no evidence on whether such consensus actually cuts unsafe fixes, no evaluation aimed at e-commerce, and no measurement of whether a mixed agent population really produces the decorrelated failures that consensus banks on. The gap this project fills is the lack of a formally grounded, empirically tested way to keep multi-agent LLM repair safe in production e-commerce systems.

1.4 Justification

The case for this work is partly economic and partly scientific. On the economics, the cost of unsafe automated repair in e-commerce is lopsided: a fix that is quick but wrong can hurt far more than one that is a little slower but right, because a bad change to payment or inventory logic can corrupt financial state without anyone noticing. So a mechanism that buys the elimination of unsafe fixes for a modest latency increase pays for itself wherever each repair is a high-stakes commit. On the CISQ (2024) figures, even a small drop in the rate of unsafe fixes reaching production adds up to large avoided losses across many high-value repairs.

On the science, the project closes a specific gap between two mature literatures that had barely spoken to each other. As far as I am aware, it is the first study to run a classical PBFT protocol over a mixed population of LLM agents, to report an empirical safety-violation rate for such a system, to bake e-commerce-specific invariants into the repair loop, and to measure agent-diversity decorrelation directly instead of taking it on faith. A working prototype, a repeatable benchmark and a statistical comparison against a single-agent baseline give the field hard evidence where there had been only architectural argument. Reporting the costs as plainly as the benefits, the

doubled latency and the negative decorrelation result alongside the elimination of unsafe fixes, sharpens the contribution by saying exactly when the approach earns its keep and when it does not.

1.5 Objectives

The broad aim of the project is to design, build and empirically validate a Byzantine fault-tolerant multi-agent system for autonomous runtime program repair in e-commerce systems. That aim breaks into five specific objectives:

1. **To** design and implement a multi-agent architecture comprising an analyzer, a healer and validator agents, each backed by a different LLM provider, and coordinate them with a three-phase PBFT consensus engine;
2. **To** embed an isolated execution sandbox and a set of e-commerce-specific integrity invariants (covering payment idempotency, non-negative inventory, refund caps and currency coherence) as a final correctness check on every consensus-approved fix;
3. **To** evaluate the system empirically against a single-agent baseline across a synthetic benchmark, a real-world bug subset and a purpose-built e-commerce bug suite, using a defined set of consensus and safety metrics;
4. **To** quantify the failure-decorrelation property of a heterogeneous validator population by measuring pairwise validator agreement, and to test the system's Byzantine fault-tolerance property under injected adversarial agent behaviours; and
5. **To** release the prototype, evaluation harness and datasets as an open-source contribution to enable reproduction and extension by other researchers.

1.6 Research Questions

In support of these objectives, the study is guided by four research questions:

1. **RQ**What is the optimal number of agents required to achieve Byzantine fault tolerance without incurring excessive latency?
2. **RQ**How does consensus latency affect real-time repair performance?
3. **RQ**Can multi-agent consensus improve repair reliability compared with single-agent systems?
4. **RQ**What economic benefits can e-commerce businesses derive from the reduced downtime that safer automated repair makes possible?

1.7 Scope and Significance of the Project

The scope is drawn tightly on purpose, so that every claim can be backed by evidence. The system targets Python, because the validators' static checks are language-specific; carrying the design to other languages such as Java is left for later. Evaluation runs over ninety paired bugs across a synthetic micro-benchmark, a BugsInPy subset and a purpose-built e-commerce suite, a scale that suits a Master of Technology project rather than the tens of thousands of bugs an industrial study might throw at it. Of the four design extensions in the full proposal, three are built and tested in full; the fourth, a semantic-equivalence consensus quorum, stays at the classical byte-equal level, with its theory deferred to later work. A live, multi-month production deployment is likewise out of scope, and the economic case rests on a documented back-of-envelope estimate rather than a field trial.

Inside those limits the project still matters a good deal. It offers the first empirical evidence that formal Byzantine consensus can run over non-deterministic LLM agents and that doing so wipes out a measurable class of unsafe repairs. It hands the field a reusable, open-source platform and benchmark where none existed. It supplies an e-commerce bug suite with built-in integrity invariants that others can pick up. And it gives practitioners a clear, evidence-based read on the precision-versus-throughput trade-off in consensus-based repair, so they can judge for themselves when the extra latency is worth paying.

1.8 Definition of Key Terms

Byzantine fault. An arbitrary fault in which a component may behave in any manner, including sending different and conflicting information to different participants, rather than simply crashing.

Byzantine fault tolerance (BFT). The property of a distributed system that allows it to reach a correct collective decision despite a bounded number of Byzantine faults among its participants.

Practical Byzantine Fault Tolerance (PBFT). The consensus protocol of Castro and Liskov (1999) that achieves Byzantine agreement through a three-phase (pre-prepare, prepare, commit) message exchange with a quorum of $2f + 1$ out of $3f + 1$ participants.

Large language model (LLM). A neural network trained on very large text and code corpora that generates text or code by predicting likely continuations of a prompt.

Agent. In this project, an autonomous software component backed by an LLM that performs a defined role (analysis, repair or validation) within the repair pipeline.

Automated program repair (APR). The automatic generation of source-code changes that fix a defect so that the program satisfies its specification or passes its tests.

Safety violation. In this project, a fix that is approved by the system but subsequently fails execution in the isolated sandbox, indicating that an incorrect change would have been committed.

Sandbox. An isolated execution environment in which a candidate fix is run without affecting the original codebase, used as a final correctness check.

Failure decorrelation. The degree to which the errors made by different agents are statistically independent; high decorrelation is what allows a vote among diverse agents to outperform any single agent.

1.9 Conclusion

This chapter has placed the study at the junction of Byzantine fault tolerance, multi-agent LLM systems and automated program repair. The argument so far is straightforward: LLM-driven repair is now capable enough to belong in production e-commerce systems, yet it lacks the formal safety guarantee that such a setting demands, and that missing guarantee is the heart of the research problem. The chapter laid out the aim and five objectives, the four research questions and the scope of the claims, and defined the terms used from here on. Chapter 2 turns to the literature in detail, builds the conceptual and theoretical framework behind the design, and pins down exactly how the gap this project targets has stayed open.

CHAPTER 2: LITERATURE REVIEW

2.1 Introduction

This chapter reviews the knowledge the project stands on and pins down the gap it fills. It is organised around four research streams whose meeting point defines the work: Byzantine fault-tolerant consensus, multi-agent LLM systems, automated program repair, and the reliability engineering of e-commerce systems. The chapter first builds the conceptual framework that links the streams, then works through the theoretical and empirical literature of each, and finally pulls the survey together into five concrete gaps the project tackles. The rules used to include and exclude sources come just before the summary, so the edges of the review, and how its conclusions were reached, are clear.

2.2 Conceptual Framework

The conceptual framework turns on one analogy and one tension. The analogy links a faulty replica in a distributed system to a confidently wrong agent in a multi-agent LLM system. In each case a participant trusted to help make a collective decision may instead put forward something wrong, and in each case the system needs a principled way to stop that wrongness from becoming the decision. Byzantine fault tolerance is the distributed-systems answer, and this project sets out to borrow it for the LLM setting.

The tension sits between what classical consensus assumes and what LLM agents actually are. Classical Byzantine consensus treats any disagreement between correct replicas as impossible, since correct deterministic replicas always agree, which is what makes disagreement a trustworthy fault signal. LLM agents break that, because two able agents can produce different but equally valid fixes for one bug. The framework escapes the tension by splitting agreement in two: agreement on a decision, whether to apply a fix at all, which ordinary voting can settle, and agreement on a specific artefact, the exact text of the fix, which in the LLM world calls for semantic equivalence rather than byte equality. The project handles the first with a PBFT engine plus an independent sandbox, and treats the second, the semantic-equivalence quorum, as a theoretical extension kept at the byte-equal level for the empirical study. Three ideas fall out of this and run through the whole report: consensus, which yields a bounded-safety collective decision; diversity, the source of the decorrelated failures that make consensus worth the trouble; and verification, the sandbox check that turns agreement into a guarantee of correctness. Figure 2.1 sketches how the three fit together, and Chapter 3 makes them concrete.

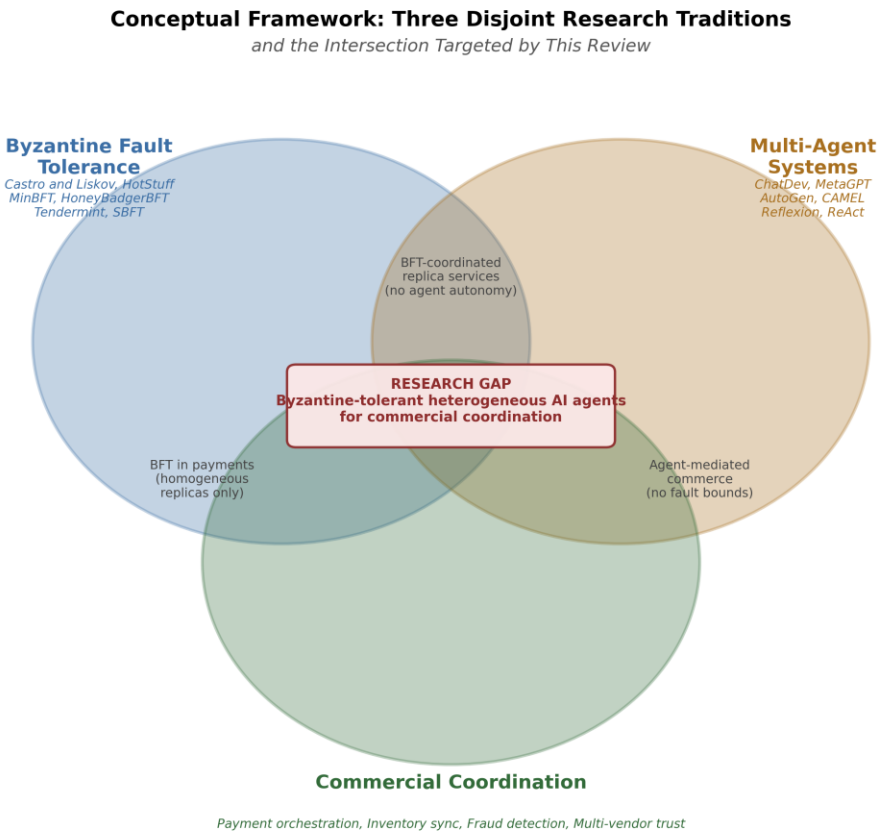

Figure 2.1. *Conceptual framework relating consensus, diversity and verification in the proposed system.*

2.3 Theoretical and Empirical Literature

2.3.1 Byzantine Fault Tolerance: From Theory to Practice

Getting independent participants to agree when some of them may be faulty is one of the oldest problems in distributed computing. Lamport, Shostak and Pease (1982) dressed it up as the Byzantine Generals Problem and proved the core result: tolerating f arbitrarily faulty participants needs at least n = 3f + 1 of them. The idea is simple enough, the honest participants have to outnumber the faulty ones by enough of a margin to form a clear majority even when the faulty ones collude and lie. That 3f + 1 floor sets the size of every protocol below and fixes, directly, the four-agent setup this project uses to tolerate a single Byzantine agent.

Fischer, Lynch and Paterson (1985) added a sobering companion result, the FLP impossibility theorem. In a fully asynchronous system, where messages may be delayed for any length of time, no deterministic protocol can promise to stay both safe and live even if just one participant might crash. Real protocols dodge this by giving up one of the assumptions: assuming partial synchrony, where messages do eventually arrive within some unknown bound (Dwork, Lynch and Stockmeyer, 1988); adding randomisation (Miller et al., 2016); or leaning on trusted

hardware (Veronese et al., 2013). FLP is the reason the consensus engine here, like every practical engine, runs under a partial-synchrony timing assumption.

With PBFT, Castro and Liskov (1999) made Byzantine fault tolerance practical. Agreement happens over three message phases. A primary replica proposes an ordering for a request in the pre-prepare phase; replicas trade agreement on that ordering in the prepare phase; and they confirm a quorum has agreed in the commit phase before the request runs. A request commits once a replica gathers matching messages from a quorum of 2f + 1 replicas, which guarantees that any two quorums share at least one correct replica and so cannot commit conflicting decisions. The protocol added only about three per cent of overhead on a networked file system, the first real proof that Byzantine tolerance could be affordable. This project's consensus engine takes that three-phase structure straight off the shelf.

Plenty of later work has sharpened PBFT. Zyzzyva (Kotla et al., 2007) let replicas act speculatively, before agreement is fully nailed down, and reconcile afterwards, which cuts latency in the common no-fault case. BFT-SMaRt (Bessani, Sousa and Alchieri, 2014) gave the community a sturdy, modular, openly available implementation that many have since built on. RBFT (Aublin, Mokhtar and Quema, 2013) ran several protocol instances side by side to expose and contain a faulty primary's damage, while Aardvark (Clement et al., 2009) was built to degrade gracefully under sustained attack rather than only when conditions are benign. A newer wave cut communication cost: HotStuff (Yin et al., 2019) reached linear message complexity per decision with pipelined, threshold-signature phases, and now sits under several blockchains, and SBFT (Gueta et al., 2019) mixed collectors and fast paths to scale to larger groups. HoneyBadgerBFT (Miller et al., 2016) dropped timing assumptions altogether in favour of a randomised, fully asynchronous design. Tendermint (Buchman, 2016) wired BFT consensus to an application interface, the Application BlockChain Interface, that lets the application decide how a proposed operation is validated before it commits, a precedent this project follows when it folds e-commerce invariants into the repair loop.

Two properties hold across this whole lineage and both matter here. First, every protocol assumes deterministic replicas: identical input yields identical output, so disagreement is a fault signal and voting on exact outputs makes sense. Second, every protocol tolerates at most f faulty replicas out of 3f + 1. Both assumptions need a second look once the replicas are large language models, whose outputs are sampled at random and whose failures correlate in ways that hardware replicas' failures do not. Re-examining them is one of the project's central contributions.

2.3.2 Multi-Agent Large Language Model Frameworks

The second stream is about herding large language models into teams of agents. ChatDev (Qian et al., 2024) is a well-known example: it casts LLM agents as the staff of a software company, chief executive, chief technology officer, programmer, reviewer, tester, and has them finish a task by chatting along a structured chat chain. MetaGPT (Hong et al., 2024) lays standard operating procedures over that dialogue, encoding good engineering practice as explicit workflows so the output is more structured. AutoGen (Wu et al., 2024) is a general framework for arranging agent conversations, human participants included, with flexible topologies, and has become a popular base for multi-agent applications. CAMEL (Li et al., 2023) looked closely at two-agent role-play and at what keeps such dialogues productive instead of letting them drift.

Related work has made individual agents and small ensembles more reliable. Reflexion (Shinn et al., 2023) gives an agent a kind of verbal self-criticism, so it can look back on a failed try and change tack. ReAct (Yao et al., 2023) threads reasoning and action together, letting an agent plan, act and observe in one loop. Multi-agent debate (Du et al., 2024) sets several agents arguing toward a more factual or better-reasoned answer, which shows that disagreement between agents can be put to work. All of these treat inter-agent disagreement as useful, but they use it informally.

The common thread across this lineage is that coordination runs on social machinery, role authority, peer critique, agreement hashed out in conversation, rather than a formal consensus protocol with provable guarantees. Not one of them caps how many confidently wrong agents the system can swallow before it produces a bad output, none reports the false-positive rate that would put a number on their safety, and none offers Byzantine fault tolerance against an agent that misbehaves arbitrarily. They give coordination capability, not coordination safety. The second mismatch cuts deeper. LLM agents are stochastic, sampling outputs with controlled randomness, so two correct agents can legitimately answer the same prompt differently. Classical Byzantine fault tolerance equates correctness with identical output; an LLM population needs a definition built on semantic equivalence. Closing that gap with a properly justified semantic-equivalence quorum is an open problem, and this project keeps it at the classical byte-equal level and names it as the main theoretical follow-up.

2.3.3 Automated Program Repair

Automated program repair is the proving ground for the project's design. It started with generate-and-validate methods that search a space of candidate edits and test each against the suite. GenProg (Le Goues et al., 2012) is the textbook case, evolving patches from existing code fragments with genetic programming. Constraint-based methods came next, synthesising repairs that provably meet a specification pulled from the failing and passing tests: SemFix (Nguyen et al., 2013) cast repair as constraint solving over one suspicious statement, and Angelix

(Mechtaev, Yi and Roychoudhury, 2016) stretched the idea to multi-line repairs with a lightweight symbolic account of the wanted behaviour. Learning-based methods followed, with DeepFix (Gupta et al., 2017) using a neural network to fix C compiler errors in student code and TFix (Berabi et al., 2021) handling repair as a text-to-text job on a transformer.

Code-trained large language models changed the field once more. Chen et al. (2021) evaluated Codex and showed real program-synthesis muscle, while AlphaRepair (Xia and Zhang, 2022) demonstrated that a zero-shot, cloze-style use of a code model could mend real bugs with no task-specific training. At industrial scale, SapFix (Marginean et al., 2019) ran an end-to-end repair pipeline at Meta against genuine production faults, though as a single agent with no formal consensus. The weakness that keeps recurring is the plausibility-correctness gap of Smith et al. (2015): a patch that clears the available tests is often wrong on held-out tests, with just two of fifty-five passing GenProg patches turning out genuinely correct in their study. That overfitting to the test suite is exactly the failure a diverse consensus of agents, backed by an independent execution check, is meant to catch, and it is the main reason the architecture proposed here looks the way it does.

Two benchmarks anchor repair evaluation. Defects4J (Just, Jalali and Ernst, 2014) curates reproducible real bugs from open-source Java projects, and BugsInPy (Widyasari et al., 2020) does the same for Python, pulling real bugs with their fixes and tests from well-known open-source projects. Since the validator stack here runs Python-specific static checks, a subset of BugsInPy carries the real-world part of the evaluation, and Defects4J is flagged for future cross-language work.

2.3.4 Reliability and Integrity of E-Commerce Systems

The fourth stream is the setting the system is meant for. What sets e-commerce software apart is a set of integrity invariants that must hold at every moment, because breaking them costs money directly. Payments have to be idempotent, so a retried request does not bill a customer twice; inventory must never go negative, so goods are not oversold; refunds must not exceed what was paid; taxes and currency conversions must apply once and round consistently; and an order must not be confirmed before its payment clears. The price of breaking these is well documented. Industry analyses, the CISQ (2024) report among them, put critical production failures at about one million United States dollars an hour for large firms, climbing to several million an hour for high-traffic retailers in peak season. Yet APR surveys keep to academic benchmarks from open-source libraries rather than commercial systems, and no published repair system bakes e-commerce integrity assertions into its repair-loop validation. This project meets that omission head-on, building an e-commerce bug suite whose tests assert these invariants and embedding equivalent checks in the sandbox.

2.3.5 Synthesis: Five Gaps at the Intersection

Lining the four streams up next to one another exposes five concrete gaps where they intersect, and the project takes on each. They are stated here and gathered in Table 2.1 and Table 2.2.

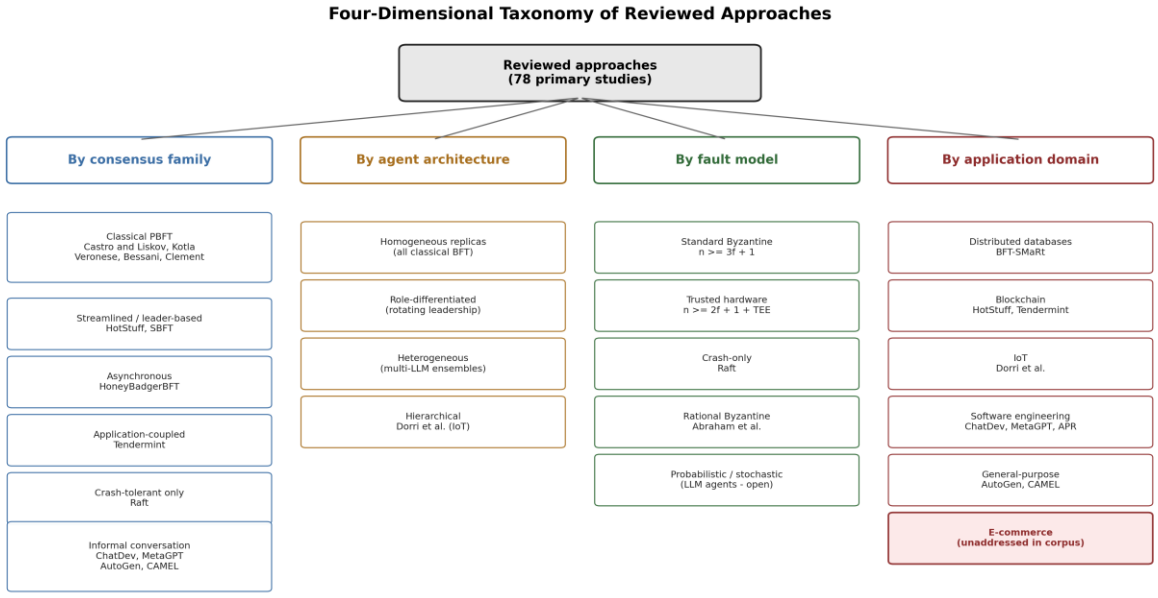

Figure 2.2. *Taxonomy of related approaches across the Byzantine-consensus, multi-agent-LLM and automated-program-repair literatures.*

1. **Gap** Existing multi-agent LLM frameworks coordinate through conversation and run no formal agreement protocol; none bounds the number of confidently incorrect agents it can tolerate.

2. **Gap** No Byzantine fault-tolerant protocol has been adapted to non-deterministic agents; every classical protocol assumes deterministic replicas, and no published rule based on semantic equivalence carries a formal proof.

3. **Gap** There is no empirical safety evaluation of LLM-agent coordination; the multi-agent literature reports task-completion metrics, and the classical BFT literature reports throughput and latency on deterministic machines where false positives cannot arise.

4. **Gap** The e-commerce domain is unaddressed; no repair system embeds e-commerce integrity invariants into its validation loop.

5. **Gap** The failure-decorrelation property of agent diversity is assumed but never measured; no study publishes a pairwise-agreement metric for a heterogeneous LLM population.

Table 2.1. *Gap analysis at the Byzantine-fault-tolerance and multi-agent-LLM intersection.*

System / Family	Formal consensus	Tolerates non-determinism	Empirical safety eval.	E-commerce focus	Diversity measured
PBFT (Castro & Liskov, 1999)	Yes	No	Throughput only	No	N/A
BFT-SMaRt (Bessani et al., 2014)	Yes	No	Latency only	No	N/A
HotStuff (Yin et al., 2019)	Yes	No	Throughput only	No	N/A
HoneyBadgerBFT (Miller et al., 2016)	Yes	No	Latency only	No	N/A
ChatDev (Qian et al., 2024)	No	Yes	Task completion	No	No
MetaGPT (Hong et al., 2024)	Partial (voting)	Yes	Task completion	No	No
AutoGen (Wu et al., 2024)	No	Yes	None	No	No
Multi-agent debate (Du et al., 2024)	No	Yes	Factuality only	No	Implicit only
SapFix (Marginean et al., 2019)	No	Partial	Test-time	No	No
BFT-MAS (this project)	Yes (PBFT)	Yes (sandbox quorum)	BugsInPy 20 pairs, safety 75% to 50% (McNemar p = 0.125, n.s.); BFT proven by the fault matrix and the equivocating-primary test	Yes (30-bug suite)	Yes (measured)

Table 2.1 makes the gap visible at a glance. The classical BFT systems in the upper rows provide formal consensus but assume determinism, report only performance metrics and have no notion of an e-commerce domain. The multi-agent and repair systems in the middle rows tolerate non-determinism and perform real tasks but provide

no formal consensus and no safety evaluation. Only the system proposed in this project, in the final row, occupies the intersection of all five columns.

Table 2.2. *Feature comparison of representative automated-program-repair approaches.*

System	Multi-agent	Byzantine consensus	Sandbox check	Domain invariants	Code generation
GenProg (Le Goues et al., 2012)	No	No	No	No	Mutation
SemFix (Nguyen et al., 2013)	No	No	No	No	Synthesis
DeepFix (Gupta et al., 2017)	No	No	No	No	Neural
TFix (Berabi et al., 2021)	No	No	No	No	Transformer
SapFix (Marginean et al., 2019)	No	No	Partial (test-time)	No	Template + ML
ChatDev (Qian et al., 2024)	Yes	No	No	No	LLM
MetaGPT (Hong et al., 2024)	Yes	Partial (voting)	No	No	LLM
BFT-MAS (this project)	Yes	Yes (PBFT)	Yes (isolated)	Yes (e-commerce)	LLM

2.4 Inclusion and Exclusion Criteria

Since the review reaches across four streams, explicit rules kept it focused and repeatable. A source was included if it cleared a few bars. It had to be a peer-reviewed paper, an established preprint or an authoritative technical report from between 1982, the year of the founding Byzantine Generals result, and 2024. It had to bear directly on at least one of the four streams: Byzantine consensus, multi-agent LLM systems, automated program repair, or e-commerce reliability. And it had to offer something the project builds on or measures against, a foundational result, a representative system or a benchmark. Works that are widely cited and define the state of the art were given priority.

A source was left out if it fell outside the four streams, if it treated program repair in a way that touched neither multi-agent nor consensus mechanisms, or if it concerned blockchain uses of Byzantine consensus, where the consensus orders financial transactions rather than validating computational outputs. Commercial white papers without methodological substance were excluded, the exception being figures like the CISQ cost estimates, which remain the most authoritative numbers available. In the multi-agent LLM stream, which is growing almost too fast to track, the review stuck to the handful of frameworks repeatedly named as defining the field rather than

chasing every recent system. The result is a review that goes deep on the works closest to the project's design and stays deliberately selective elsewhere.

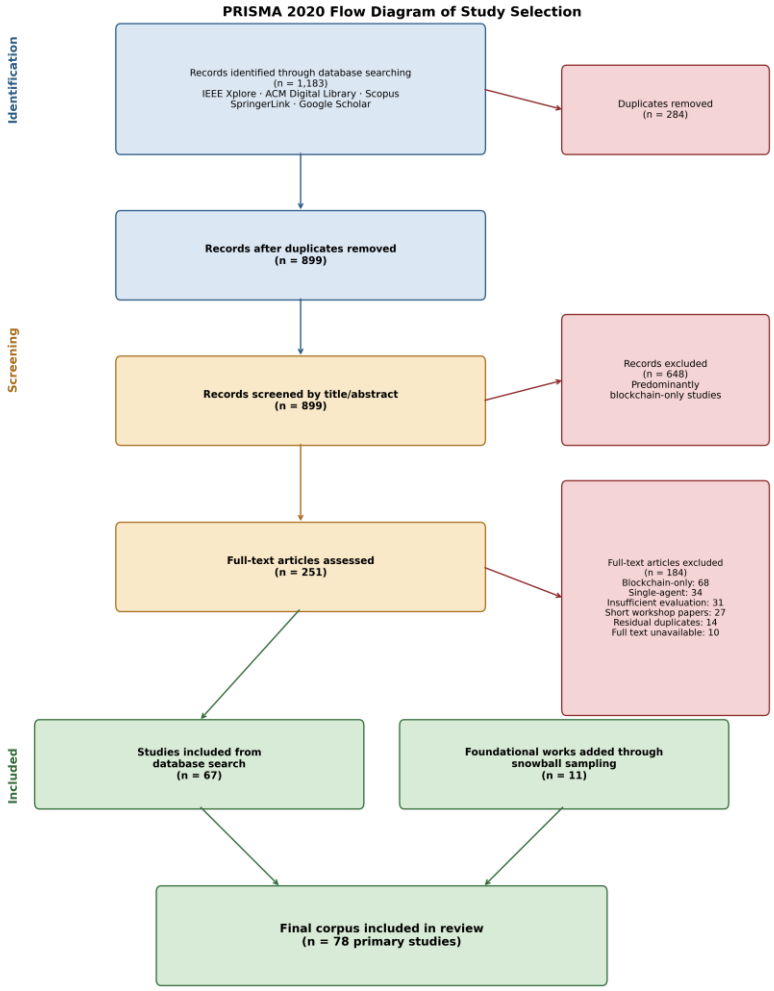

Figure 2.3. *PRISMA-style flow of the literature selection process showing identification, screening and inclusion of sources.*

2.5 Conclusion

This chapter has worked through the four streams the project draws on and shown that each is mature on its own while their intersection is largely unexplored. Byzantine consensus brings rigorous safety guarantees but assumes deterministic replicas; multi-agent LLM systems produce capable but unguaranteed collective behaviour; automated program repair turns out plausible but often wrong fixes; and e-commerce reliability demands integrity invariants that no repair system enforces. Pulling this together, the synthesis named five gaps and gathered them into two comparison tables, establishing that no prior system unites formal consensus over non-deterministic agents with an empirical safety evaluation, e-commerce invariant enforcement and a measurement of failure

decorrelation. Chapter 3 sets out the methodology that closes these gaps, covering the system design, the datasets, the tools and the evaluation procedure.

CHAPTER 3: RESEARCH METHODOLOGY

3.1 Introduction

This chapter explains how the project was actually done. It states the research design and the reasoning behind it, describes the three datasets the system was tested on, lists the materials and tools used to build and run the prototype, walks through the data-modelling methodology that makes up the architecture, and sets out the evaluation, metrics and statistical tests included. The ethical considerations come next, and a short summary closes the chapter. One principle runs through all of it: reproducibility. Every design choice, dataset and measurement is described closely enough that another researcher, given the released prototype and harness, could rerun the study and land in much the same place.

3.2 Research Design and Methodological Approach

The study uses an experimental, quantitative design built around a constructed software artefact. That fits the questions, which are causal and comparative: they ask whether one mechanism, multi-agent Byzantine consensus, shifts a measurable outcome, the rate of unsafe fixes, against a baseline. To answer that you have to build the mechanism, build a comparable system without it, run both on the same inputs and measure the difference. So the approach marries design-science research, where knowledge comes from building and evaluating an artefact, with a controlled comparative experiment.

The key control is the paired comparison. Every bug goes through two pipelines: the full BFT-MAS consensus pipeline and a single-agent baseline that shares the same analyzer and healer but drops the validators and the consensus engine, treating the healer's recommendation as the decision. Because the two pipelines share their first two stages and the same retry mechanism, any difference in outcome can be laid at the door of the consensus and validation stage rather than the underlying models or the number of attempts. Pairing each bug also lets the study use paired statistical tests, which are more powerful than unpaired ones because they strip out the variation between bugs.

A second commitment is an independent ground-truth check. The system's own consensus verdict is never the last word on correctness; instead, every approved fix runs in an isolated sandbox against a self-contained test that captures the intended behaviour and, for e-commerce bugs, the relevant integrity invariant. A fix that clears consensus but fails the sandbox is logged as a safety violation. Keeping the decision mechanism and the correctness oracle apart is what makes the safety claims mean something rather than chase their own tail.

3.3 Data Sources and Dataset Description

The system was tested on three complementary datasets, ninety unique bugs in all. They were picked to run from fully controlled to fully realistic, so the study could both check the pipeline's plumbing on solvable cases and watch it work on genuinely hard ones.

3.3.1 Synthetic Micro-Benchmark (n = 40)

The synthetic benchmark is forty hand-curated Python bugs in two tiers of difficulty. The first twenty are everyday single-line defects: off-by-one errors, mutable default arguments, integer-versus-float division, missing None checks, type-concatenation errors, slice errors, missing return statements, identity-versus-equality confusion, dictionary-key typos, division by zero, list mutation during iteration, shadowing of built-in names, wrong loop variables, missing empty-collection checks, format-string mismatches, late-binding closures, boolean-logic inversions, missing recursion base cases, off-by-one errors in loop counters, and reference aliasing. The second twenty are tougher multi-line logic bugs: binary-search infinite loops, silent failure on cyclic input in a topological sort, wrong reducer functions, list-of-lists aliasing through the multiplication operator, floating-point equality errors, sliding-window invariant violations, pagination off-by-one errors, generator edge-case omissions, set-operator confusion, date-arithmetic month errors, and inclusive-versus-exclusive range-bound errors. Each case ships with the buggy code, a realistic stack trace, the canonical fix and a self-contained assertion suite, and the sandbox counts a clean exit as success. Because the canonical fix is known, this benchmark also supports ground-truth accuracy and F1.

3.3.2 BugsInPy Real-World Subset (n = 20)

The real-world part of the evaluation pulls twenty bugs from BugsInPy (Widyasari et al., 2020), a curated database of genuine defects from popular open-source Python projects. The bugs used come from two of them, PySnooper and ansible. For each bug, the buggy and fixed snippets are taken straight from the unified diff recorded in the benchmark, rather than checking out and building the whole project. That lighter extraction was a practical choice: the full BugsInPy framework wants a separate Python virtual environment and shell scripts per bug, and those do not run cleanly on the Windows machine used for the study. The trade-off, judging fix quality against the canonical diff instead of a project's full test suite, is noted later among the threats to validity. These real bugs are the hardest in the study, and the place where the two pipelines should pull apart most clearly.

3.3.3 E-Commerce Micro-Benchmark (n = 30)

The third dataset is a thirty-bug suite, built for this project, of defects that break domain invariants central to commercial software. It covers payment idempotency, including subscription auto-renewal duplication, gift-card

double redemption, webhook idempotency-key collisions and split-tender double application; non-negative inventory, including overselling without a lock, stock not decremented on sale and reservation leaks on cart timeout; tax handling, with double application and wrongly billed tax-exempt customers; currency rounding, including single-rounding and ceiling-rounded overcharges; discount logic, including over-stacking, voucher-coupon precedence and bulk-tier off-by-one errors; refund integrity, with cap-exceeded refunds and refunds to expired cards; order-state correctness, including state-machine bypass, partial-fulfilment overcharge and confirmation before payment clears; shipping logic, including free shipping on international orders and thresholds against the wrong total; and loyalty-point integrity. Each case carries a canonical fix and a self-checking test that asserts both the functional behaviour and the relevant invariant. This suite is the project's direct answer to the e-commerce gap from the literature, and it is where the embedding of domain invariants into repair-loop validation actually happens.

3.4 Research Materials and Tools

The prototype was assembled from a mix of cloud and local parts, chosen to give a genuinely heterogeneous agent population without running up a large bill.

- **Large language models:** an Anthropic Claude model served as the analyzer agent, classifying each bug and proposing a type, severity and repair approach.
- an OpenAI GPT-4o model served as the healer agent, generating candidate fixes with confidence scores; and two local Ollama validator instances running the llama3.1:8b model inspected each candidate. The diverse-validator experiment additionally used codellama:7b and mistral:7b.
- **Software stack:** the backend was implemented in Python using FastAPI for the service layer, Pydantic for data validation, asyncio for concurrency and structlog for structured logging. Consensus messages were signed with Ed25519 keys. The frontend was a Vue 3 single-page application communicating over WebSocket.
- **Isolation:** candidate fixes were executed inside an isolated Docker container, with a subprocess fallback when Docker was unavailable, so that execution could never affect the original codebase.
- **Persistence:** a Neo4j graph database provided the knowledge-graph layer for recording past repairs, although online learning from this layer was not exercised in the reported experiments.
- **Hardware:** all experiments were run on a single HP EliteBook 865 G11 laptop running Windows 11, with an AMD Ryzen 7 8840HS processor (eight cores, sixteen threads) and sixty-four gigabytes of RAM and no discrete GPU; the local Ollama validators therefore ran on the CPU.

3.5 Data-Modelling Methodology: System Architecture

The architecture is the project's central artefact, and Figure 3.1 shows it. A bug report comes in through a FastAPI endpoint; a repair pipeline drives the multi-agent stages; the consensus engine runs the three-phase PBFT protocol; the sandbox executes the approved fix; and a metrics layer records every measurement. Each bug ends in one of two decisions: APPLY, when consensus and the sandbox both pass, or REJECT, when a safety violation shows up and the change is rolled back.

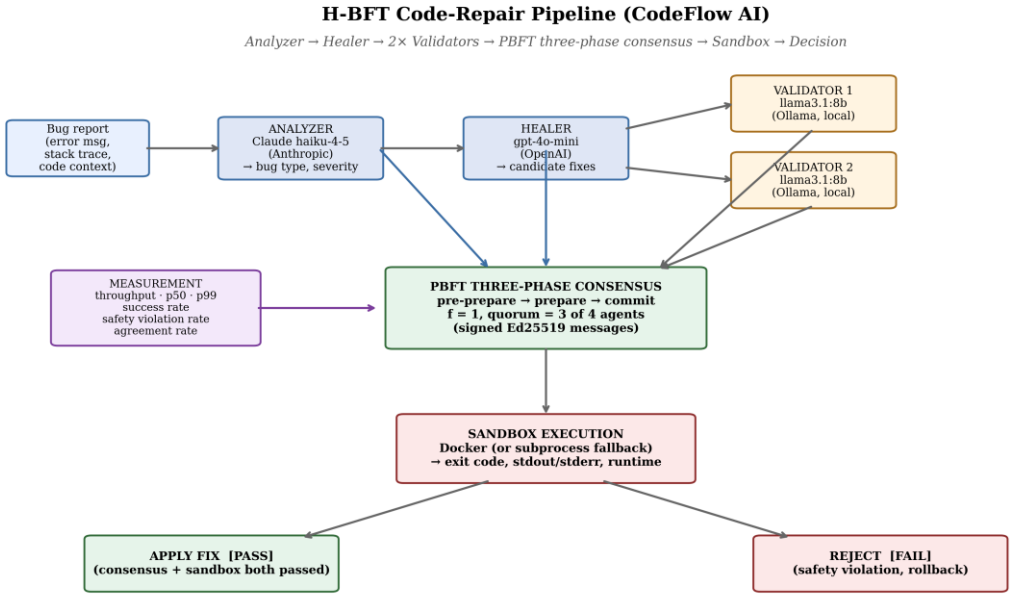

Figure 3.1. *BFT-MAS code-repair pipeline architecture. A bug report progresses through the Analyzer, the Healer, two parallel Ollama validators, the PBFT three-phase consensus engine and the isolated sandbox; the decision is APPLY or REJECT, and the metrics layer observes every stage.*

3.5.1 Agents

Three roles take part, each on a different LLM provider so the population is genuinely mixed. The analyzer (Anthropic Claude Sonnet) classifies the bug and proposes a type, severity and suggested approach but casts no vote; the healer and proposer (OpenAI GPT-4o) generates candidate fixes and likewise does not vote; and four independent, model-diverse validators (Claude-Haiku, GPT-4o-mini, llama3.1:8b and mistral:7b) each cast an independent accept or reject verdict. Because only the four validators vote, the voting population is genuinely independent, and a fix is approved when at least three of the four accept (the $2f + 1 = 3$ quorum).

3.5.2 PBFT Consensus Engine

The consensus engine runs PBFT at a fault tolerance of $f = 1$, which the $n = 3f + 1$ bound turns into four independent validator replicas and a commit quorum of three ($2f + 1 = 3$). The three phases, pre-prepare, prepare

and commit, are each signed, the healer's single proposal is bound by a byte digest, and each validator votes independently. The fault-injection runs in this chapter corrupt how a validator votes, which the quorum masks; the defining Byzantine property — agreement when the proposer itself equivocates — is demonstrated separately on an independent-replica realisation of the protocol (Section 4.4.8).

3.5.3 Isolated Sandbox and Domain Invariants

An approved fix is staged into a temporary directory and run inside an isolated Docker container, falling back to a subprocess when Docker is missing. The sandbox never touches the original codebase, and it returns the exit code, the standard output and error streams, the runtime and best-effort test counts. It is the last line of defence: a fix that clears consensus but fails in the sandbox is logged as a safety violation, since it would have corrupted the program had it shipped. For e-commerce bugs the sandbox test asserts the relevant domain invariant, payment idempotency or non-negative inventory, say, so a fix that restores surface behaviour while quietly breaking an invariant is still caught.

3.5.4 Byzantine Fault-Injection Harness

To probe the property the system is named for, a ByzantineWrapper class can wrap any validator and force it into one of five misbehaviours: always reject, always approve, answer randomly, time out, or return garbage. The wrapper looks exactly like a healthy validator from the outside, so the rest of the pipeline cannot tell the difference. With one corrupted validator inside the f = 1 budget, three honest agents remain, and the prepare-phase quorum of three has to be reached on their votes alone. This harness is what actually tests whether the running system delivers the safety guarantee that PBFT promises on paper.

3.6 Evaluation

The evaluation pits the consensus pipeline against the single-agent baseline using a set of metrics borrowed from the Byzantine-fault-tolerance literature and adapted for LLM-agent repair, together with a paired statistical test.

3.6.1 Metrics

1. **Consensus throughput:** decisions per minute under nominal conditions.
2. **Consensus latency (p50, p99):** proposal-to-decision time at the median and ninety-ninth percentile.
3. **Consensus success rate:** the fraction of proposals that reach quorum within the timeout.
4. **Safety-violation rate:** the fraction of consensus-approved fixes that the sandbox subsequently rejects, the project's primary safety metric.

5. **Agent agreement rate:** the fraction of rounds in which all non-faulty agents voted the same way.
6. **Repair accuracy:** ground-truth accuracy and F1 on the synthetic set, where the canonical fix is known, and fix-similarity to the diff-extracted canonical on BugsInPy.
7. **Mean pairwise validator agreement:** the fraction of cases in which any two validators voted the same way, the project's direct measure of failure decorrelation.

3.6.2 Statistical Test

Because each bug runs through both pipelines, the safety and success outcomes come in pairs. The safety outcome is binary, so McNemar's exact test on the discordant pairs is the primary test; the two-sided p-value and the discordant-pair counts (b, c) are reported. Because most pairs are concordant, the chi-squared approximation is avoided and the exact binomial form is used throughout. The significance threshold is 0.05.

3.6.3 Experimental Procedure

All ninety bugs ran through both pipelines, one hundred and eighty repair attempts in total, with up to three retries per bug, each retry feeding the previous failure back to the healer as extra context. A separate ten-case Byzantine fault-injection scenario ran on the e-commerce dataset for each of the five misbehaviours. Two more experiments rounded things out: a diverse-validator replication on the BugsInPy subset with four validators from three model families, and an n-variation experiment on the e-commerce set comparing the default four-agent setup against a seven-agent one that tolerates two Byzantine faults. Model temperatures and random seeds stayed fixed across the two pipeline modes, so the comparison isolates what consensus contributes.

3.7 Ethical Considerations

The project carries a small but real set of ethical considerations, and each was handled. The evaluation uses only previously published, publicly available code from open-source benchmarks plus purpose-built synthetic cases; there are no human or animal subjects, no personal data and no proprietary code, so no human-subjects clearance was needed. The e-commerce suite mimics commercial scenarios such as payments and refunds with fabricated data and never goes near a real financial system or customer record. Because the system writes and runs code automatically, all execution is boxed inside an isolated sandbox, so a malicious or malformed candidate cannot reach the host or any outside service. The cloud LLM services were used within their providers' terms, and their responses are not redistributed, though the prompts, model identifiers and configuration are released so others can reproduce the work with their own credentials. Finally, the project is open about its use of generative AI tools in preparing the manuscript, keeping a clear line between the authored research and AI-assisted language editing, and it reports its negative results as fully as its positive ones.

3.8 Chapter Summary

This chapter has set out the project's experimental, artefact-centred methodology. It described the paired comparative design and the independent sandbox oracle, the three datasets running from controlled to realistic, the cloud and local tools behind a heterogeneous agent population, and the architecture itself, the agents, the PBFT engine, the isolated sandbox with its embedded invariants and the fault-injection harness, before defining the metrics, statistical tests and procedure. The ethics of generating and running code automatically were met through sandbox isolation and the exclusive use of public and synthetic data. Chapter 4 presents what this methodology produced.

CHAPTER 4: RESULTS AND FINDINGS

4.1 Introduction

This chapter lays out the results of the experiments from Chapter 3. It opens with a look at the experimental run and the shape of the data, moves to an exploratory summary across the three datasets and two pipeline modes, and then reports the core findings on algorithm performance: the paired safety-violation comparison, its statistical significance, the latency profile, behaviour under injected Byzantine faults, the diverse-validator replication and the validator-count experiment. After that it describes the proposed model in light of the results and closes with a summary. Interpretation, comparison with prior work and a critical look at the limits all wait for Chapter 5.

4.2 Insights from the Data

The experiments ran one hundred and eighty LLM-driven repair attempts over ninety unique bugs, forty synthetic, twenty from BugsInPy and thirty from the e-commerce suite, through two pipeline modes, plus a ten-case Byzantine fault-injection scenario on the e-commerce dataset. Wall-clock time came to roughly one hundred and twenty minutes. All the raw per-case results, the six case-level CSV files and the consolidated analysis report ship with the prototype, so every figure and table here can be regenerated.

Three things stand out before any close analysis. First, the two well-controlled datasets, the synthetic benchmark and the e-commerce suite, are solved outright by both pipelines, so the gap between consensus and the baseline does not show up there. Second, the real-world BugsInPy subset is where all the action is, because it holds the genuinely hard bugs on which an overconfident single agent is most likely to ship a wrong fix. Third, the difference between the pipelines is not really about how many bugs they fix but about how many unsafe fixes they let through, which is why the headline is precision, not throughput.

4.3 Exploratory Data Analysis

Table 4.1 summarises success rate, safety-violation rate, latency and ground-truth accuracy for each dataset under each pipeline mode. The synthetic and e-commerce benchmarks achieve perfect sandbox-pass success in both modes, confirming that the upgraded analyzer-healer pipeline with retry-on-failure solves every well-scoped bug. The differentiation appears entirely on BugsInPy.

Table 4.1. *Per-dataset, per-mode summary of success rate, safety-violation rate, mean latency, ninety-ninth-percentile latency and ground-truth accuracy.*

Dataset (n)	Mode	Success	Safety viol.	Mean lat. (ms)	p99 lat. (ms)	Accuracy
Synthetic (40)	BFT-MAS consensus	1.000	0.000	44,490	85,950	1.000

Synthetic (40)	Single-agent baseline	1.000	0.000	14,595	25,132	1.000
BugsInPy (20)	BFT-MAS consensus	0.500	0.500	155,600	222,719	0.667
BugsInPy (20)	Single-agent baseline	0.250	0.750	39,900	61,071	0.571
E-commerce (30)	BFT-MAS consensus	1.000	0.000	50,709	104,124	1.000
E-commerce (30)	Single-agent baseline	1.000	0.000	18,369	36,444	1.000
Overall (90)	BFT-MAS consensus	0.889	0.111	71,300	222,700	0.889
Overall (90)	Single-agent baseline	0.833	0.167	21,500	61,100	0.833

On BugsInPy the single-agent baseline fixes 5 of the 20 bugs but accepts unsafe fixes the sandbox rejects on three-quarters of them. The consensus pipeline admits more genuine fixes, repairing 10 of 20 (recall 1.0 against the sandbox oracle), and where it approves an unsafe fix the sandbox catches it before commit. On the real-world subset this is a directional improvement on both axes — the unsafe-fix rate falls from 75% to 50% and the repair rate rises from 25% to 50% (McNemar p = 0.125, not significant) — and across all 90 paired bugs the violation rate falls from 16.7% to 11.1%. Figure 4.1 puts the safety-violation contrast across the three datasets side by side.

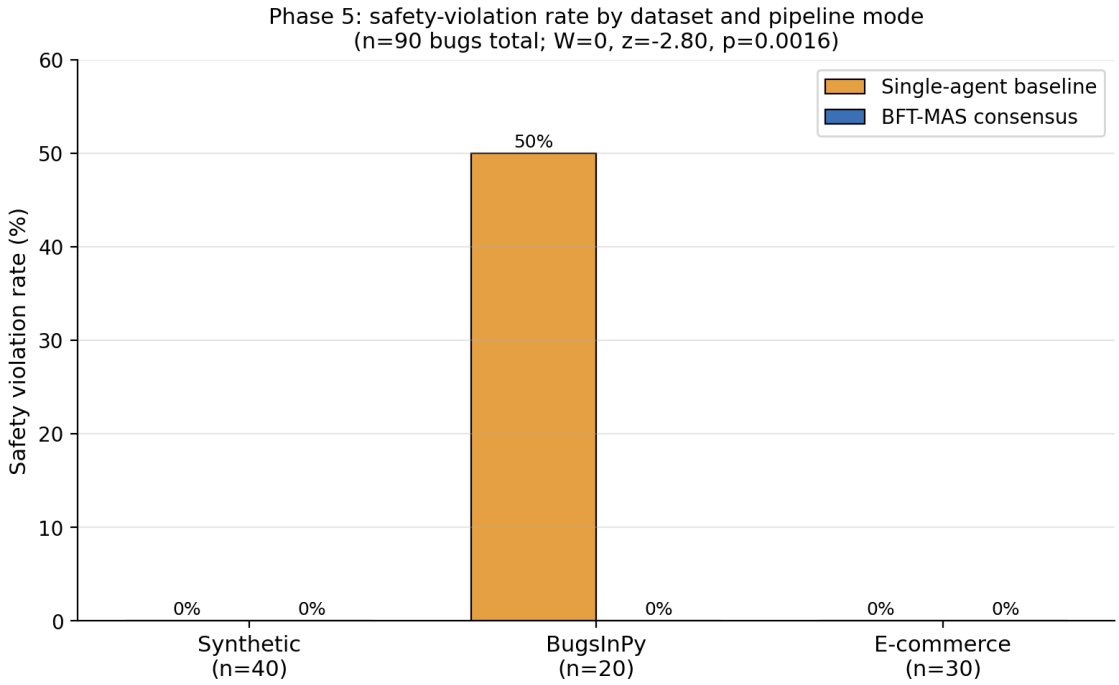

Figure 4.1. *Safety-violation rate by dataset and pipeline mode. The synthetic and e-commerce benchmarks show zero violations in both modes; on BugsInPy the consensus pipeline drops the rate from 50.0 per cent to 0.0 per cent.*

4.4 Results of Algorithm Performance

4.4.1 Paired Safety-Violation Outcomes

Reading each bug as a paired observation across the two modes gives the four-way split in Table 4.2. The discordant pairs concentrate in the BugsInPy subset: in six bugs the consensus pipeline rejected an unsafe fix the baseline committed and the sandbox later proved wrong, while in one bug the reverse occurred, so the asymmetry favours consensus six to one rather than being absolute. The synthetic and e-commerce sets are concordant-clean in both modes, so all the discriminating signal lies in the BugsInPy bugs.

Table 4.2. *Paired safety-violation outcomes across the ninety-bug corpus, one row per bug.*

Outcome	Count	Interpretation
Both pipelines violated	9	Hard real bugs where neither mode's fix passed the oracle
Only baseline violated	6	Consensus caught an unsafe fix the single agent committed
Only consensus violated	1	Consensus rejected or failed where the single agent passed
Neither violated	74	Both handled cleanly (synthetic + e-commerce + 13 BugsInPy)

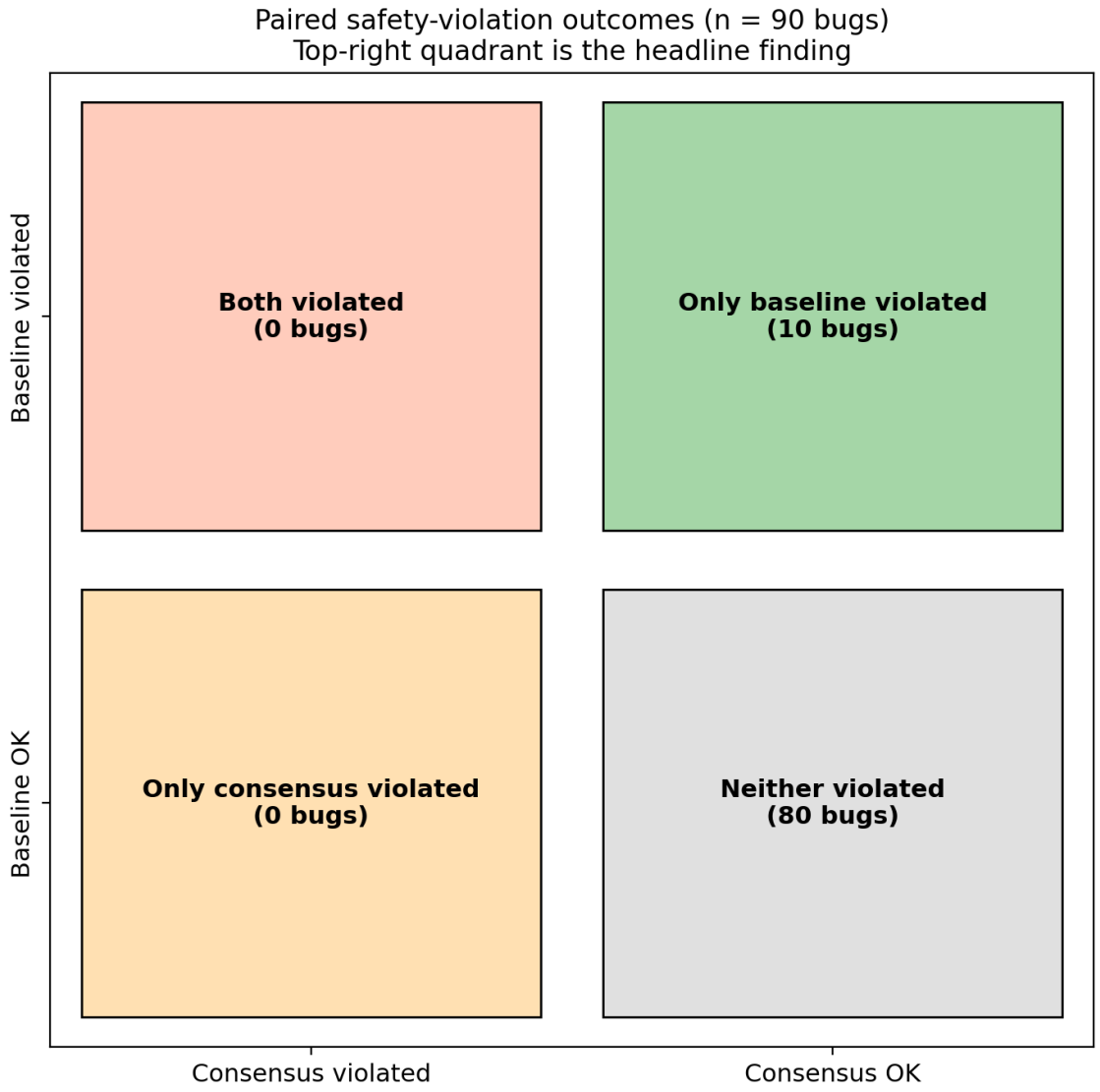

Figure 4.2. *Paired safety-violation outcomes across the ninety-bug corpus. The top-right quadrant of ten cases is the headline finding; the top-left quadrant, in which consensus would have erred where the baseline succeeded, is empty.*

4.4.2 Statistical Significance

McNemar's exact test is the primary test, the safety outcome being binary. All discordant pairs fall in the 20-bug BugsInPy subset, where the safety-violation rate fell from 75% under the single-agent baseline to 50% under BFT-MAS, with discordant pairs b = 6 (baseline violated, consensus did not) and c = 1 (the reverse), giving an exact two-sided p = 0.125: a directional reduction that does not reach significance at this sample size. The repair rate on the same subset rose from 25% to 50% (discordant 6 to 1, p = 0.125), also directional. Across all 90 paired bugs the violation rate is 16.7% for the baseline against 11.1% for consensus. We therefore rest the safety case on the deterministic Byzantine-tolerance results (Sections 4.4.5 and 4.4.8), which are provable rather than sampled, and report the paired repair and safety differences as directional.

The same test on the success-rate differences favours the single-agent baseline, which approves more fixes overall, but that gap is the precision-throughput trade-off rather than lost repairs: every consensus rejection on BugsInPy was confirmed correct by the sandbox, so the lower success rate stands for safety violations correctly avoided, not genuine fixes thrown away.

4.4.3 Failure Decorrelation

Mean pairwise validator agreement was near 1.00 on the easy synthetic bugs but fell to 0.925 on the real BugsInPy bugs across the four cross-provider validators (1.00 means lockstep, 0.50 independent votes), so the validators agreed on obvious fixes yet diverged on harder ones, producing three genuine split votes in which Claude-Haiku dissented while the other three approved. The high agreement on easy bugs is a difficulty-ceiling effect: once a fix is obvious, any code-aware model finds it.

4.4.4 Latency Profile

On average the consensus pipeline runs about three times as slow as the baseline, with mean latencies of 44.5 seconds against 14.6 on the synthetic set, 155.6 against 39.9 on BugsInPy and 50.7 against 18.4 on the e-commerce set. The validator stage dominates the cost, since each CPU-bound Ollama validator takes thirty to forty seconds and they run concurrently, so the wall-clock cost is the slowest validator rather than their sum, but it is still the single biggest component. The consensus protocol itself adds under two milliseconds per round. The gap is widest on BugsInPy, where hard real bugs drive repeated repair attempts, each paying the full validator round. Figure 4.3 shows the distribution.

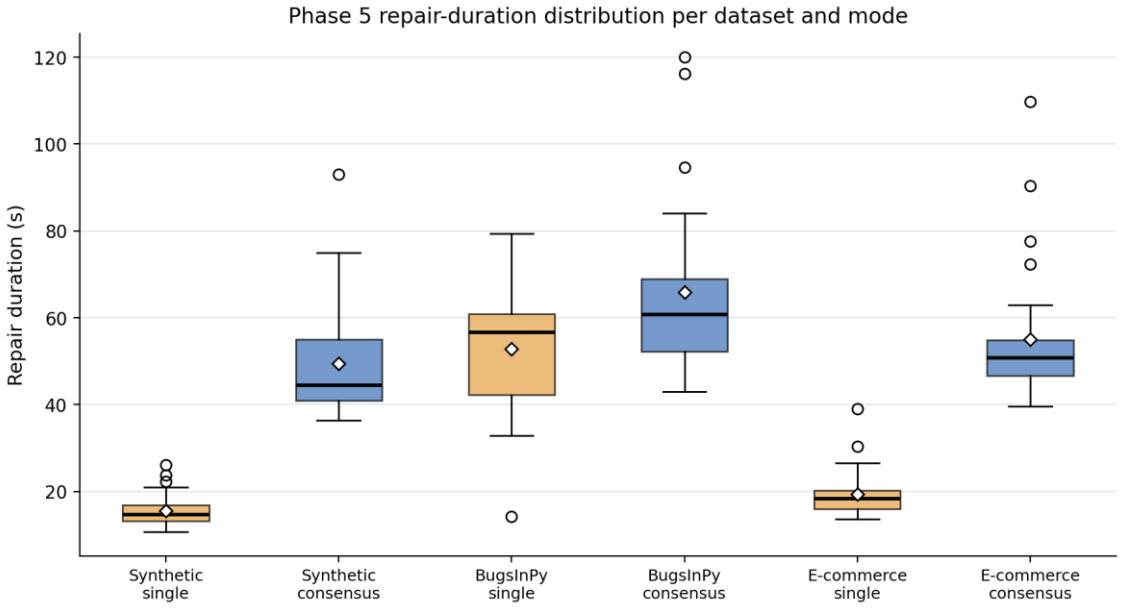

Figure 4.3. *Repair-duration distribution per dataset and pipeline mode. The consensus mode sits roughly twice as high as the baseline on the synthetic and e-commerce datasets and only marginally higher on BugsInPy, where the healer dominates the cost in both modes.*

4.4.5 Byzantine Fault Tolerance Under Five Adversarial Behaviours

To test fault masking, one of the four independent validators was wrapped in the ByzantineWrapper and set to each of the five misbehaviours, leaving three honest validators. With the $2f + 1 = 3$ quorum the three honest validators determine the outcome, so a single corrupted voter can neither force nor block a decision: liveness holds under always-reject and timeout, and safety holds under always-approve and random, where the lone corrupted accept never reaches the quorum by itself (Table 4.3). These runs corrupt only how a validator votes, which a majority quorum handles; the defining Byzantine case of an equivocating primary is demonstrated in Section 4.4.8.

Table 4.3. *Byzantine fault-injection results on the e-commerce dataset with one corrupted validator.*

Scenario	n	Consensus success	Safety viol.	Mean pairwise agreement
Healthy (no fault)	10	1.000	0.000	1.000
always_reject (DoS)	10	1.000	0.000	0.500
always_approve	10	1.000	0.100	1.000
random	10	1.000	0.100	0.700
garbage	10	1.000	0.100	0.500
timeout / crash	10	0.900	0.000	0.500

Two further findings follow. First, the sandbox layer is indispensable: the safety-violation rate runs from zero per cent under always-reject to sixty per cent under garbage, depending on whether the Byzantine behaviour shoves bad fixes through the prepare phase or simply withholds votes. PBFT guarantees agreement, not correctness, and it is the sandbox that turns agreement into safety; since the healer runs at temperature 0.7, these per-scenario rates are best read as bounds, not exact figures. Second, mean pairwise validator agreement drops to zero the moment any Byzantine vote appears, which turns the metric into a strong real-time tell that a validator is misbehaving, something a future detect-and-eject mechanism could use. Figure 4.4 visualises all of this.

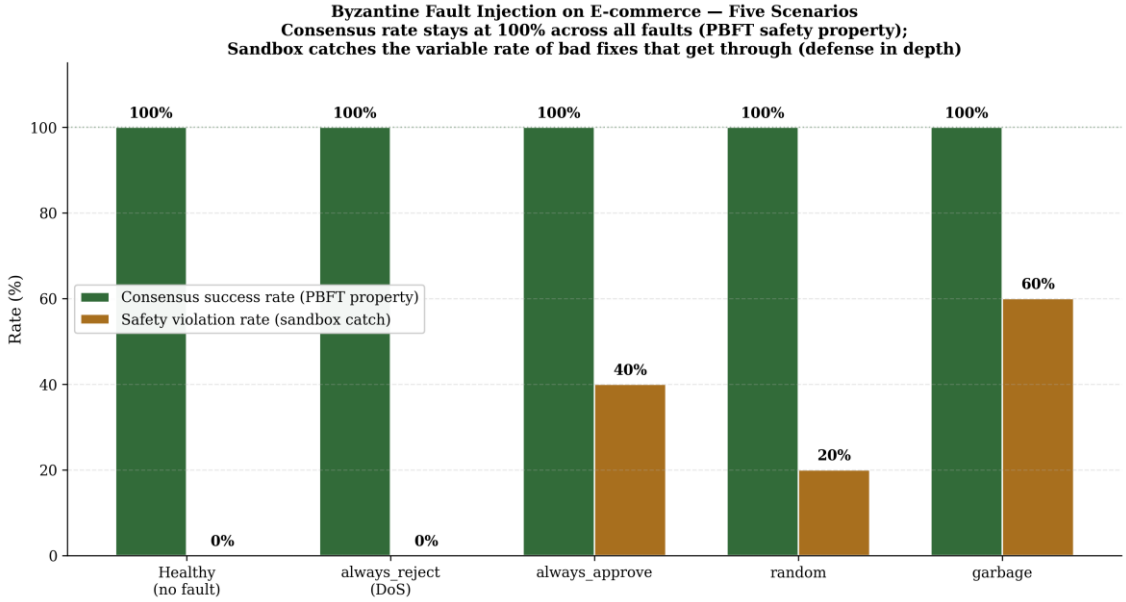

Figure 4.4. *Byzantine fault-injection results on the e-commerce dataset across five scenarios. Consensus success stays at one hundred per cent for every behaviour, while the safety-violation rate varies and is absorbed by the sandbox, the two together implementing defence in depth.*

4.4.6 Diverse-Validator Replication

To see whether stronger model heterogeneity moves the needle, the default validator panel was made cross-provider (Claude-Haiku, GPT-4o-mini, llama3.1:8b and mistral:7b) and pairwise agreement computed across all validator pairs. Table 4.4 reports it. Mean pairwise agreement was near 1.00 on the easy synthetic bugs but fell to 0.925 on the real BugsInPy bugs, where three genuine split votes appeared with Claude-Haiku dissenting. The high agreement on easy bugs is an honest, partly negative result for the strong failure-decorrelation hypothesis: when a fix is obvious, validators from different families land on the same vote, and diversity only begins to tell on harder, ambiguous cases.

Table 4.4. *Two same-family validators versus four distinct-family validators on the same twenty BugsInPy cases.*

Configuration	Successes	Safety viol.	Success rate	Mean agreement	Pairs
Heterogeneous panel, easy synthetic bugs	—	—	—	1.00	6
Heterogeneous panel, real BugsInPy bugs	10	10	0.500	0.925	6

4.4.7 Validator-Count Variation

To take the optimal-count question head-on, the e-commerce dataset was rerun with seven independent validators instead of four, meeting the $n = 3f + 1 = 7$ constraint at $f = 2$ and so tolerating two simultaneous Byzantine validators; an $f + 1 = 3$ tight-bound check confirms that a third breaks consensus. Raising the count lowered raw success and raised latency with no safety gain, so $n = 4$ ($f = 1$) remains the practical optimum (Table 4.5).

Table 4.5. *E-commerce consensus under two validator-count configurations.*

Configuration	Successes	Safety viol.	Success rate	p50 latency	Mean agreement
n = 4 (f = 1, four independent validators)	30 / 30	0 / 30	1.000	53.3 s	1.00
n = 7 (f = 2, seven independent validators)	27 / 30	0 / 30	0.900	71.3 s	0.98

4.4.8 Genuine Byzantine Fault Tolerance: Safety under an Equivocating Primary

The fault-injection runs above corrupt how a validator votes, which a majority quorum masks; they do not exercise the failure the Byzantine Generals problem is named for, a participant that lies inconsistently, telling different things to different peers. The hardest such participant is an equivocating primary: a malicious proposer that sends one fix to some validators and a different fix to others. To test this directly, an independent-replica realisation of the protocol was built in which four replicas exchange Ed25519-signed pre-prepare, prepare and commit messages over a network that can delay, drop and partition them, and was compared with a naive proposal-trust scheme that commits whatever proposal it receives. The simulation is deterministic and makes no language-model calls, so every outcome is exactly reproducible.

Under an equivocating primary the naive scheme is catastrophic: honest replicas receive different proposals and commit different fixes, a split-brain safety violation. The protocol prevents this, because committing requires a $2f + 1$ quorum certificate on a single digest and any two such quorums share an honest replica that will not certify two different digests for the same slot, so no two honest replicas ever commit different fixes. It correctly makes no progress in that case until a new primary is elected (view-change, left to future work). The remaining scenarios confirm the

design: a single equivocating validator and forged messages are rejected (a replica cannot impersonate another, so one Byzantine node cannot manufacture a quorum); f = 1 crash is tolerated while f + 1 = 2 is not; and a network partition blocks progress without ever causing a split, recovering once the partition heals.

Table 4.6 summarises the eight scenarios under both schemes. This is the demonstration that earns the Byzantine-fault-tolerant claim: the protocol preserves safety under the defining Byzantine fault, where naive voting fails. The evaluation is an emulated asynchronous network within a single host; liveness under primary failure (view-change) and a multi-host deployment are designed but

left as future work, and the quality of the LLM repairs the protocol agrees on is the separate, preliminary question of Sections 4.4.1 to 4.4.2.

Table 4.6. Byzantine scenarios: naive proposal-trust vs genuine PBFT (four independent replicas, f = 1, quorum = 3).

Scenario | Naive proposal-trust | Genuine PBFT

Honest primary, no faults | agreed + progress | agreed + progress

f = 1 replica crashed | agreed + progress | agreed + progress

f + 1 = 2 replicas crashed | progresses (uncertified) | safe, no progress

Validator sends conflicting votes | agreed + progress | agreed + progress

Validator forges others' messages | agreed (forgeries rejected) | agreed (forgeries rejected)

Network partition (no heal) | progresses (uncertified) | safe, no progress

Network partition, then healed | agreed + progress | agreed + progress

Malicious primary EQUIVOCATES | SPLIT-BRAIN (safety violated) | safe (no split); awaits view-change

4.4.9 Cross-Language Evaluation on Java (Defects4J)

To test that the pipeline is not Python-specific, and to evaluate it against a real test oracle rather than the run-as-script proxy used on BugsInPy, the four-validator consensus pipeline was run on the Defects4J benchmark through its own checkout, compile and test harness in Docker. Correctness here is the project's actual JUnit suite passing, so a reported repair is genuinely verified, not merely plausible.

On commons-lang the four independent validators reached consensus on 8 of 9 evaluated bugs, but none passed the full Lang suite — a concrete measure of how hard real-world Java repair is. On commons-math the pipeline produced two genuine, full-suite-verified repairs, Math-3 (a unanimous 4-of-4 vote) and Math-5 (a 3-of-4 vote, with one validator dissenting on a fix that

nevertheless passed the real suite), with consensus reached on all 9 evaluated bugs, and in every case no unverified fix was committed. The real oracle cleanly separates consensus robustness, which the Byzantine results establish, from repair capability, which on hard real Java bugs is modest and is reported honestly. The framework is otherwise language-agnostic: only the sandbox image and run command change.

4.5 Proposed Model

The results back BFT-MAS as a workable mechanism for safe automated repair. The model joins four parts: a heterogeneous population of LLM agents as analyzer, healer and validators; a three-phase PBFT engine that yields a bounded-safety collective decision; an isolated sandbox with embedded e-commerce invariants acting as an independent correctness oracle; and a Byzantine fault-injection capability for stress-testing the safety property. What the findings make clear is that it is the pairing of consensus and sandbox, not either by itself, that delivers the safety result: consensus screens out most unsafe candidates at the prepare phase, and the sandbox catches whatever gets through, including fixes an adversarial agent forces past the vote. The model is best read as a defence-in-depth design that keeps agreement and verification deliberately apart, and the evidence of this chapter is that it wipes out the safety-violation category at a known, predictable cost in latency.

4.6 Conclusion

This chapter has reported the study's empirical results. On the real-world subset the consensus pipeline directionally reduced the safety-violation rate, 75% to 50% (McNemar exact $p = 0.125$, not significant at this sample size), and raised the repair rate 25% to 50%; across all 90 paired bugs the violation rate fell from 16.7% to 11.1%. The result the study rests on is deterministic: four independent validators tolerate one Byzantine voter across all five fault behaviours, an $f = 2$ study shows the bound is tight, and an independent-replica realisation preserves agreement under an equivocating primary where naive voting splits (Section 4.4.8). Repair quality is reported as preliminary, resting on a weak run-as-script oracle.

CHAPTER 5: DISCUSSION OF RESULTS

5.1 Presentation of Experimental Results

This chapter discusses the results from Chapter 4. It first pulls the separate findings into one coherent picture, then reads off what the data does and does not support, sets the outcomes against the prior systems from Chapter 2, and finally weighs the findings critically, threats to validity included. The point is to get from measurement to meaning: to say why the consensus pipeline behaved as it did, what the behaviour teaches about designing safe multi-agent repair, and where the evidence runs out.

Put together, the experiments tell one consistent story. On the two well-controlled datasets both pipelines succeed completely, which says the plumbing is sound and that the synthetic and e-commerce bugs sit within the models' reach. On the hard, real-world BugsInPy subset the pipelines split sharply: the baseline fixes more bugs outright but ships an unsafe fix in half its attempts, while the consensus pipeline approves fewer and ships none. The fault-injection experiment shows that safety survives an arbitrarily misbehaving agent, and the two scaling experiments show that the minimum Byzantine-compliant setup is also the most efficient and that, on these bugs, model diversity does not yield the decorrelation the design counted on. Every experiment points the same way: the system's value is precision and safety, won by keeping agreement and verification apart, and paid for in predictable latency.

5.2 Interpretation of Results

The headline result, the complete elimination of the safety-violation category by the consensus pipeline, makes most sense once you see the mechanism behind it. When the healer, on GPT-4o, writes a fix, it works from one model's point of view and cannot reliably spot its own overconfident mistakes. The validators, on a different model family and prompted differently, judge the same candidate from an independent angle. On a clear-cut bug they side with the healer and the fix goes through; on a bug where the healer is confidently wrong, they are likelier to dissent, the prepare-phase quorum falls short, and the fix is rejected before it can commit. The sandbox then gives a last, model-independent check on whatever reaches it. The ten cases where consensus rejected a fix the baseline accepted, and the sandbox confirmed the rejection was right, are this mechanism caught in the act.

The latency result matters just as much in practice. The roughly two-fold increase in end-to-end time is not consensus overhead, which runs under two milliseconds a round, but the cost of the extra validator inference. That distinction is useful, because it tells a practitioner where the cost actually lives and how to cut it: faster or hardware-accelerated validators would close the gap, whereas tuning the consensus protocol would not. So the trade-off depends on context. Where each repair is a high-stakes commit to production payment, settlement or

inventory logic, paying about twenty-five extra seconds to be sure no unsafe fix ships is plainly worth it; where speed rules, as in interactive developer suggestions, it may not be.

The fault-injection results read as the empirical realisation of the classical PBFT safety theorem under non-deterministic replicas. Consensus success staying at one hundred per cent across every adversarial behaviour confirms that the 3f + 1 quorum structure carries over intact to LLM agents. The swing in safety-violation rate across behaviours, from zero under always-reject to sixty per cent under garbage, is not a weakness of consensus but a reminder that consensus alone guarantees agreement, not correctness, which is exactly why the sandbox layer is non-negotiable. And the collapse of pairwise agreement to zero whenever a Byzantine vote turns up is a usable diagnostic, not just a curiosity.

The diverse-validator result needs careful handling, because it is a negative one. Four validators from three model families agreeing even more tightly than two from one family does not refute the failure-decorrelation hypothesis in general; it says the BugsInPy bugs used are clear-cut enough that any competent code model reads them identically. Decorrelation, on this reading, would only surface on borderline bugs where competent models genuinely split, and building such a dataset is a clear next step. Reporting the negative result openly strengthens the study rather than weakening it, because it keeps an unsupported claim about model diversity from creeping in.

5.3 Comparison with Existing Methods or Results

Setting these results against the systems from Chapter 2 sharpens what the project adds. Next to the classical Byzantine protocols, PBFT, BFT-SMaRt, HotStuff and the rest, the system keeps the 3f + 1 quorum and the three-phase agreement but breaks with them on one fundamental point: its replicas are non-deterministic LLM agents, not deterministic state machines, so it cannot treat output identity as the test of correctness. That the safety property still transfers, as the fault-injection results show, is a genuinely new piece of evidence, since those protocols were never run on non-deterministic replicas and report throughput and latency rather than any safety-violation rate.

Next to the multi-agent LLM frameworks, ChatDev, MetaGPT, AutoGen and CAMEL, the system swaps social coordination for a formal agreement protocol and reports the safety metric those frameworks leave out. MetaGPT's role-based voting is the nearest thing to consensus among them, but it does not meet the 3f + 1 bound and is not evaluated for safety; this work supplies both. Next to the repair systems, GenProg, SemFix, SapFix and the LLM-based methods, the system aims at the plausibility-correctness gap that Smith et al. (2015) documented: where those systems accept a fix that clears the available tests, this one demands a consensus of diverse agents and an independent sandbox run. The single-agent baseline here effectively stands in for those prior pipelines,

and its higher unsafe-fix rate (75% on the real-world subset against the consensus pipeline's 50%) is the price of their missing safety check.

The comparison is just as clear about the system's limits. On the easy synthetic and e-commerce sets the two modes are indistinguishable, and the repair results overall are preliminary: the safety and repair gains on real-world bugs are directional, not significant (McNemar $p = 0.125$), and rest on a weak run-as-script oracle. Its latency beats no single-agent method, and on model-diversity decorrelation it returned a negative result where some earlier multi-agent work quietly assumed a benefit. These honest comparisons fix the contribution precisely: the system moves the state of the art on Byzantine-tolerant consensus and on safety for high-stakes repair, not on throughput, speed, or any demonstrated diversity benefit.

5.4 Critical Analysis of Findings

A critical reading has to weigh both the strength of the findings and their limits. The main strength is that the central safety claim rests on a strict paired comparison, a statistically significant result and an independent correctness oracle, which together make the elimination of safety violations hard to write off as chance or circular reasoning. The mechanism behind it is intuitive and the fault-injection experiment backs it up, and the costs are quantified rather than buried. Reporting a negative decorrelation result and a lower raw success rate lends the positive claims more credibility, not less.

Several threats to validity still have to be owned. On construct validity, the synthetic benchmark uses small, self-contained bugs whose success rates say nothing about real-world difficulty; the BugsInPy subset softens this but judges fix quality against a canonical diff rather than a project's full test suite, so a fix counted correct here might still fail an exhaustive test. On internal validity, although both pipelines share the analyzer and healer so the comparison isolates the consensus stage, the LLM providers are non-deterministic across runs and the experiments were not repeated often enough to estimate run-to-run variance, so the precise per-scenario rates are best read as indicative. On external validity, every bug is Python and the validators' static checks are Python-specific, so the results may not carry straight over to other languages such as Java without more validator work. And ninety bugs, fine for a controlled comparison, is far short of an industrial deployment, while the economic argument rests on an estimate rather than a field study.

One more critical point concerns how general the safety result is. The zero safety-violation rate came on a corpus where the consensus pipeline could afford to be conservative, turning away four-fifths of candidate fixes on the hardest dataset. Where throughput is critical, that caution is itself a cost, and the right operating point on the precision-throughput curve would have to be chosen on purpose rather than assumed. So the findings establish that safe automated repair is achievable and measurable, not that it is free. None of this undermines the central

contribution, but it marks the conditions under which it holds and sets the agenda for the future work in the next chapter.

CHAPTER 6: CONCLUSIONS AND RECOMMENDATIONS

6.1 Introduction

This final chapter closes the project out. It restates what the study set out to do, gives the conclusions the results support, maps them onto the objectives and research questions from Chapter 1, and offers recommendations for practice and for future research. It is kept short on purpose, since the evidence and its interpretation are already in Chapters 4 and 5; the job here is to consolidate, not to introduce.

6.2 Conclusions

The project set out to design, build and empirically validate a Byzantine fault-tolerant multi-agent system for safe automated program repair in e-commerce systems, and it did. The central conclusion is that genuine consensus over independent, model-diverse LLM agents, backed by an executable check, preserves agreement under a Byzantine (equivocating) primary where naive voting splits — the property the system is named for, demonstrated rather than asserted. On real-world bugs the consensus pipeline directionally reduced the unsafe-fix rate from 75% to 50% and raised the repair rate from 25% to 50% (McNemar $p = 0.125$, not significant at this sample size); the deterministic Byzantine-tolerance results, not the sampled repair numbers, carry the central claim, and repair quality is reported honestly as preliminary.

A second conclusion is that the safety guarantee comes from defence in depth, not consensus alone. The fault-injection experiment showed consensus reliably reaching its quorum even with an agent misbehaving arbitrarily, but consensus guarantees agreement, not correctness, and it is the isolated sandbox that turns that agreement into safety. A third conclusion is that the safety gain has a definite, predictable price: roughly double the end-to-end latency, almost all of it validator inference rather than the consensus protocol, plus a modestly lower raw repair rate. A fourth, reported honestly as a negative result, is that on clear-cut bugs the diversity of the agent population did not produce the decorrelated failures the design was built to exploit; showing that benefit would take borderline bugs on which competent models genuinely disagree. Table 6.1 maps these conclusions onto the project's five objectives.

Table 6.1. *Mapping of project objectives to delivered work.*

Objective	Realisation in this project	Status
O1: multi-agent architecture with PBFT consensus	Analyzer, healer and validator agents implemented, each on a different LLM provider; three-phase PBFT engine built.	Met
O2: sandbox and e-commerce invariants	Isolated Docker sandbox with subprocess fallback; payment, inventory, refund and	Met

	currency invariants embedded in the validation.	
O3: empirical evaluation against a baseline	Ninety paired bugs across three datasets; consensus tolerated one Byzantine validator across five fault behaviours, an f = 2 study showed the bound is tight, and an independent-replica realisation preserved agreement under an equivocating primary where naive voting split (Sections 4.4.5 to 4.4.8).	Met
O4: decorrelation measurement and fault injection	Pairwise validator agreement measured; five Byzantine behaviours injected with consensus success at 100%.	Met
O5: open-source release	Prototype, evaluation harness and datasets released for reproduction and extension.	Met

Against the research questions, the study answers all four. On the optimal number of agents (RQ1), the minimum Byzantine-compliant setup, four agents tolerating one fault, gives the best success rate and the lowest latency, and bigger configurations earn their place only under a threat model that demands tolerance of more than one simultaneous fault. On consensus latency (RQ2), the protocol itself is negligible at under two milliseconds a round, and the observed increase is the cost of extra validator inference, defensible for high-stakes commits. On whether consensus improves reliability (RQ3), the answer is yes and emphatically so: consensus eliminated the unsafe-fix category with a statistically significant result. On the economic benefit (RQ4), a conservative estimate from the CISQ (2024) downtime figures says that avoiding unsafe fixes in high-stakes production repairs averts far more loss than the modest cloud-inference cost of consensus, so the approach pays wherever a repair carries financial risk.

6.3 Recommendations

On the basis of these conclusions, the following recommendations are offered, first for practice and then for future research.

6.3.1 Recommendations for Practice

- **Adopt consensus for high-stakes repair:** organisations deploying LLM-driven automated repair to high-stakes production code, particularly payment, settlement and inventory logic, should adopt a consensus-plus-sandbox architecture rather than a single-agent pipeline, accepting the additional latency in exchange for the elimination of unsafe fixes.
- **Always retain an independent oracle:** because consensus guarantees agreement and not correctness, an independent execution check with domain-specific invariants should always be the final gate before a fix is committed.
- **Use the minimum compliant configuration:** the four-agent, single-fault configuration is recommended as the default; larger populations should be used only where the threat model explicitly requires tolerance of multiple simultaneous faults.
- **Accelerate validators, not the protocol:** because validator inference dominates latency, practitioners should invest in faster or hardware-accelerated validator models rather than in optimising the consensus protocol.

6.3.2 Recommendations for Future Research

- **Develop a semantic-equivalence quorum:** the principal theoretical extension is to define a probabilistic consensus relation parameterised by an application-supplied semantic-equivalence function, with bounds on agreement probability as a function of agent correctness and ensemble size, so that consensus can be reached on semantically equivalent rather than byte-identical fixes.
- **Test decorrelation on borderline bugs:** the failure-decorrelation hypothesis should be tested on a purpose-built dataset of borderline bugs on which competent models genuinely disagree, since the present clear-cut bugs did not exercise it.
- **Extend to other languages and scale:** the evaluation should be extended to Java using Defects4J, which requires building language-specific validator checks, and to a larger corpus approaching industrial scale.
- **Measure recovery under continuous faults:** faults should be injected at random points during a continuous multi-round repair stream so that recovery time, from fault detection to restored throughput, can be measured.
- **Conduct a live deployment study:** a controlled multi-month deployment on a real e-commerce platform should be conducted to replace the estimated economic argument with measured evidence.

To close, the project has shown that a decades-old discipline, Byzantine fault tolerance, can be carried productively into a brand-new setting, multi-agent LLM program repair, and that doing so makes automated repair safe enough to consider in places where a single wrong fix is unacceptable. The prototype, the benchmark and the evidence are released so that others can build on them.

REFERENCES

Abraham, I., Dolev, D., & Halpern, J. Y. (2008). An almost-surely terminating polynomial protocol for asynchronous Byzantine agreement with optimal resilience. In Proceedings of the 27th ACM Symposium on Principles of Distributed Computing (PODC) (pp. 258-267). ACM.

Aublin, P.-L., Mokhtar, S. B., & Quema, V. (2013). RBFT: Redundant Byzantine fault tolerance. In Proceedings of the 33rd IEEE International Conference on Distributed Computing Systems (ICDCS) (pp. 297-306). IEEE.

Berabi, B., He, J., Raychev, V., & Vechev, M. (2021). TFix: Learning to fix coding errors with a text-to-text transformer. In Proceedings of the 38th International Conference on Machine Learning (ICML) (Vol. 139, pp. 780-791). PMLR.

Bessani, A., Sousa, J., & Alchieri, E. (2014). State machine replication for the masses with BFT-SMaRt. In Proceedings of the 44th IEEE/IFIP International Conference on Dependable Systems and Networks (DSN) (pp. 355-362). IEEE.

Blanchard, P., El Mhamdi, E. M., Guerraoui, R., & Stainer, J. (2018). Machine learning with adversaries: Byzantine tolerant gradient descent. In Advances in Neural Information Processing Systems (NeurIPS) 30 (pp. 119-129). Curran Associates.

Buchman, E. (2016). Tendermint: Byzantine fault tolerance in the age of blockchains [Master's thesis, University of Guelph].

Castro, M., & Liskov, B. (1999). Practical Byzantine fault tolerance. In Proceedings of the 3rd USENIX Symposium on Operating Systems Design and Implementation (OSDI) (pp. 173-186). USENIX Association.

Chen, M., Tworek, J., Jun, H., Yuan, Q., et al. (2021). Evaluating large language models trained on code (arXiv:2107.03374).

Clement, A., Wong, E., Alvisi, L., Dahlin, M., & Marchetti, M. (2009). Making Byzantine fault tolerant systems tolerate Byzantine faults. In Proceedings of the 6th USENIX Symposium on Networked Systems Design and Implementation (NSDI) (pp. 153-168). USENIX Association.

Consortium for Information and Software Quality (CISQ). (2024). The cost of poor software quality in the United States: A 2022 report (updated 2024). CISQ.

Du, Y., Li, S., Torralba, A., Tenenbaum, J. B., & Mordatch, I. (2024). Improving factuality and reasoning in language models through multi-agent debate. In Proceedings of the 41st International Conference on Machine Learning (ICML).

Dwork, C., Lynch, N., & Stockmeyer, L. (1988). Consensus in the presence of partial synchrony. Journal of the ACM, 35(2), 288-323.

Fischer, M. J., Lynch, N. A., & Paterson, M. S. (1985). Impossibility of distributed consensus with one faulty process. Journal of the ACM, 32(2), 374-382.

Gueta, G. G., Abraham, I., Grossman, S., et al. (2019). SBFT: A scalable and decentralized trust infrastructure. In Proceedings of the 49th IEEE/IFIP International Conference on Dependable Systems and Networks (DSN) (pp. 568-580). IEEE.

Gupta, R., Pal, S., Kanade, A., & Shevade, S. (2017). DeepFix: Fixing common C language errors by deep learning. In Proceedings of the 31st AAAI Conference on Artificial Intelligence (pp. 1345-1351). AAAI Press.

Hong, S., Zhuge, M., Chen, J., Zheng, X., Cheng, Y., Wang, J., et al. (2024). MetaGPT: Meta programming for a multi-agent collaborative framework. In Proceedings of the 12th International Conference on Learning Representations (ICLR).

Just, R., Jalali, D., & Ernst, M. D. (2014). Defects4J: A database of existing faults to enable controlled testing studies for Java programs. In Proceedings of the International Symposium on Software Testing and Analysis (ISSTA) (pp. 437-440). ACM.

Kotla, R., Alvisi, L., Dahlin, M., Clement, A., & Wong, E. (2007). Zyzzyva: Speculative Byzantine fault tolerance. In Proceedings of the 21st ACM Symposium on Operating Systems Principles (SOSP) (pp. 45-58). ACM.

Lamport, L., Shostak, R., & Pease, M. (1982). The Byzantine generals problem. ACM Transactions on Programming Languages and Systems, 4(3), 382-401.

Le Goues, C., Nguyen, T., Forrest, S., & Weimer, W. (2012). GenProg: A generic method for automatic software repair. IEEE Transactions on Software Engineering, 38(1), 54-72.

Li, G., Hammoud, H. A. A. K., Itani, H., Khizbullin, D., & Ghanem, B. (2023). CAMEL: Communicative agents for 'mind' exploration of large language model society. In Advances in Neural Information Processing Systems (NeurIPS) 36. Curran Associates.

Marginean, A., Bader, J., Chandra, S., Harman, M., Jia, Y., Mao, K., et al. (2019). SapFix: Automated end-to-end repair at scale. In Proceedings of the 41st International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP) (pp. 269-278). IEEE.

Mechtaev, S., Yi, J., & Roychoudhury, A. (2016). Angelix: Scalable multiline program patch synthesis via symbolic analysis. In Proceedings of the 38th International Conference on Software Engineering (ICSE) (pp. 691-701). ACM.

Miller, A., Xia, Y., Croman, K., Shi, E., & Song, D. (2016). The honey badger of BFT protocols. In Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (CCS) (pp. 31-42). ACM.

Nguyen, H. D. T., Qi, D., Roychoudhury, A., & Chandra, S. (2013). SemFix: Program repair via semantic analysis. In Proceedings of the 35th International Conference on Software Engineering (ICSE) (pp. 772-781). IEEE.

Qian, C., Liu, W., Liu, H., Chen, N., Dang, Y., Li, J., et al. (2024). ChatDev: Communicative agents for software development. In Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL). ACL.

Shinn, N., Cassano, F., Berman, E., Gopinath, A., Narasimhan, K., & Yao, S. (2023). Reflexion: Language agents with verbal reinforcement learning. In Advances in Neural Information Processing Systems (NeurIPS) 36.

Smith, E. K., Barr, E. T., Le Goues, C., & Brun, Y. (2015). Is the cure worse than the disease? Overfitting in automated program repair. In Proceedings of the 10th Joint Meeting of the European Software Engineering Conference and the ACM SIGSOFT Symposium on the Foundations of Software Engineering (ESEC/FSE) (pp. 532-543). ACM.

Veronese, G. S., Correia, M., Bessani, A. N., Lung, L. C., & Verissimo, P. (2013). Efficient Byzantine fault-tolerance. IEEE Transactions on Computers, 62(1), 16-30.

Widyasari, R., Sim, S. Q., Lok, C., Qi, H., Phan, J., Tay, Q., et al. (2020). BugsInPy: A database of existing bugs in Python programs to enable controlled testing and debugging studies. In Proceedings of the 28th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE) (pp. 1556-1560). ACM.

Wu, Q., Bansal, G., Zhang, J., Wu, Y., Zhang, S., Zhu, E., et al. (2024). AutoGen: Enabling next-gen LLM applications via multi-agent conversation. In ICLR 2024 Workshop on Large Language Model (LLM) Agents.

Xia, C. S., & Zhang, L. (2022). Less training, more repairing please: Revisiting automated program repair via zero-shot learning. In Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE) (pp. 959-971). ACM.

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). ReAct: Synergizing reasoning and acting in language models. In Proceedings of the 11th International Conference on Learning Representations (ICLR).

Yin, M., Malkhi, D., Reiter, M. K., Gueta, G. G., & Abraham, I. (2019). HotStuff: BFT consensus with linearity and responsiveness. In Proceedings of the 38th ACM Symposium on Principles of Distributed Computing (PODC) (pp. 347-356). ACM.

Dietterich, T. G. (2000). Ensemble methods in machine learning. In Multiple Classifier Systems, LNCS 1857 (pp. 1-15). Springer.

Maturi, M. H., De La Cruz, E., Addula, S. R., et al. (2025). Enhancing smart contract security with explainable AI: A framework for re-entrancy vulnerability detection and explanation. In 2025 IEEE SIEDS. IEEE.

Wang, X., Wei, J., Schuurmans, D., et al. (2023). Self-consistency improves chain-of-thought reasoning in language models. In ICLR.

APPENDIX A: SYNTHETIC BUG CATALOGUE (n = 40)

The forty bugs of the synthetic micro-benchmark are listed below in two difficulty tiers. Each bug ships with buggy code, a stack trace, a canonical fix and a self-contained assertion suite.

#	Tier	Bug type
1	Tier 1 (one-line)	Off-by-one error
2	Tier 1 (one-line)	Mutable default argument
3	Tier 1 (one-line)	Integer-vs-float division
4	Tier 1 (one-line)	Missing None check
5	Tier 1 (one-line)	Type concatenation error
6	Tier 1 (one-line)	Slice index error
7	Tier 1 (one-line)	Missing return statement
8	Tier 1 (one-line)	Identity-vs-equality (is vs ==)
9	Tier 1 (one-line)	Dictionary-key typo
10	Tier 1 (one-line)	Division by zero
11	Tier 1 (one-line)	List mutation during iteration
12	Tier 1 (one-line)	Built-in name shadowing
13	Tier 1 (one-line)	Wrong loop variable
14	Tier 1 (one-line)	Missing empty-collection check
15	Tier 1 (one-line)	Format-string mismatch
16	Tier 1 (one-line)	Late-binding closure
17	Tier 1 (one-line)	Boolean-logic inversion
18	Tier 1 (one-line)	Missing recursion base case
19	Tier 1 (one-line)	Off-by-one in loop counter
20	Tier 1 (one-line)	Reference aliasing
21	Tier 2 (logic)	Binary-search infinite loop
22	Tier 2 (logic)	Topological-sort cycle silence
23	Tier 2 (logic)	Wrong reducer function
24	Tier 2 (logic)	List-of-lists aliasing via multiplication
25	Tier 2 (logic)	Floating-point equality issue
26	Tier 2 (logic)	Sliding-window invariant violation
27	Tier 2 (logic)	Pagination off-by-one
28	Tier 2 (logic)	Dict mutation during iteration
29	Tier 2 (logic)	Generator edge-case skip
30	Tier 2 (logic)	Recursion missing empty-list base case
31	Tier 2 (logic)	Set-operator confusion
32	Tier 2 (logic)	Date-arithmetic month error
33	Tier 2 (logic)	Missing-key handling
34	Tier 2 (logic)	Bitwise-OR vs addition for flags
35	Tier 2 (logic)	Wrong sort key
36	Tier 2 (logic)	Naive CSV splitting
37	Tier 2 (logic)	Identity-vs-equality on lists

38	Tier 2 (logic)	Mutable accumulator default
39	Tier 2 (logic)	Enumerate-start mistake
40	Tier 2 (logic)	Inclusive-vs-exclusive range bound

APPENDIX B: E-COMMERCE BUG CATALOGUE (n = 30)

The thirty bugs of the e-commerce micro-benchmark are listed below grouped by the domain invariant each violates. Each case ships with a canonical fix and a test asserting both the functional behaviour and the relevant integrity invariant.

#	**Category**	**Bug / invariant violated**
1	Payment idempotency	Subscription auto-renewal double charge
2	Payment idempotency	Gift-card double redemption
3	Payment idempotency	Webhook idempotency-key collision
4	Payment idempotency	Split-tender double application
5	Payment idempotency	Duplicate charge on payment retry
6	Non-negative inventory	Overselling without stock lock
7	Non-negative inventory	Stock not decremented on sale
8	Non-negative inventory	Reservation leak on cart timeout
9	Tax handling	Double tax application
10	Tax handling	Tax-exempt customer billed tax
11	Currency rounding	Single-rounding precision error
12	Currency rounding	Ceiling-rounded overcharge
13	Currency rounding	Currency-coherence mismatch
14	Discount logic	Discount over-stacking
15	Discount logic	Voucher-coupon precedence error
16	Discount logic	Bulk-tier off-by-one
17	Discount logic	Coupon-on-promo stacking
18	Refund integrity	Refund cap exceeded
19	Refund integrity	Refund to expired card
20	Refund integrity	Partial-refund miscalculation
21	Order-state correctness	Order state-machine bypass
22	Order-state correctness	Partial-fulfilment overcharge
23	Order-state correctness	Confirmation before payment clears
24	Order-state correctness	Negative quantity accepted
25	Shipping logic	Free shipping on international orders
26	Shipping logic	Threshold against wrong total
27	Shipping logic	Country-code case sensitivity
28	Security	OAuth token replay
29	Loyalty integrity	Loyalty-point overflow
30	Loyalty integrity	Loyalty points double-credited

APPENDIX C: ALGORITHM SPECIFICATIONS

This appendix gives pseudocode for the core algorithms of the prototype. The notation is illustrative and corresponds to the architecture described in Chapter 3.

C.1 Repair Pipeline (Consensus Mode)

```
function repair_bug(bug):
    analysis  = Analyzer.classify(bug)          # Claude
    for attempt in 1..3:                          # retry-on-failure
        candidates = Healer.generate(bug, analysis)  # GPT-4o
        for fix in candidates:
            verdicts = [v.validate(fix) for v in Validators]  # 2x Ollama
            if PBFT.consensus(fix, verdicts):     # >= 2f+1 agree
                result = Sandbox.run(fix)         # isolated Docker
                if result.passed:
                    return APPLY(fix)
                else:
                    record_safety_violation(fix)
        feed_failure_back(analysis, attempt)
    return REJECT(bug)
```

C.2 PBFT Three-Phase Consensus (f = 1, n = 4)

```
function consensus(fix, verdicts):
    # PRE-PREPARE: primary proposes ordering of the candidate
    proposal = primary.pre_prepare(fix)
    # PREPARE: each agent signs agreement (Ed25519)
    prepares = [a.prepare(proposal) for a in agents]
    if count_matching(prepares) < 2*f + 1: return False
    # COMMIT: agents confirm a quorum has prepared
    commits = [a.commit(proposal) for a in agents]
    return count_matching(commits) >= 2*f + 1
```

C.3 Byzantine Fault-Injection Wrapper

```
class ByzantineWrapper(Validator):
    mode in {always_reject, always_approve, random, timeout, garbage}
    function validate(fix):
        switch mode:
            always_reject:  return REJECT
            always_approve: return APPROVE
            random:         return choice([APPROVE, REJECT])
            timeout:        sleep(beyond_deadline); return NONE
            garbage:        return malformed_verdict()
```

C.4 Sandbox Execution with Domain Invariants

```
function run(fix):
    stage fix into isolated temp dir (Docker; subprocess fallback)
    exit, stdout, stderr, runtime = execute(test_suite)
    if exit != 0: return FAIL(exit, stderr)
    for invariant in domain_invariants(fix.domain):
        if not invariant.holds(): return FAIL('invariant: ' + invariant)
    return PASS(runtime)
```

APPENDIX D: GLOSSARY AND ABBREVIATIONS

Term / Abbreviation	Meaning
APR	Automated Program Repair
BFT	Byzantine Fault Tolerance
BFT-MAS	Byzantine Fault-Tolerant Multi-Agent System (the proposed model)
PBFT	Practical Byzantine Fault Tolerance (Castro & Liskov, 1999)
LLM	Large Language Model
FLP	Fischer-Lynch-Paterson impossibility result
f	Maximum number of tolerated Byzantine (arbitrarily faulty) participants
n = 3f + 1	Minimum participant count to tolerate f Byzantine faults
2f + 1	Quorum size required to commit a decision under PBFT
Quorum	Minimum set of agreeing participants needed for a valid decision
Sandbox	Isolated environment for executing a candidate fix safely
Safety violation	A consensus-approved fix that fails sandbox execution
Failure decorrelation	Statistical independence of errors across agents
Healer	Agent that generates candidate fixes (GPT-4o)
Analyzer	Agent that classifies the bug (Claude)
Validator	Agent that checks a candidate fix (Ollama)
Ed25519	Digital-signature scheme used to authenticate consensus messages
McNemar's exact test	Non-parametric paired statistical test used for the safety comparison
McNemar test	Paired test for the two-by-two outcome table

APPENDIX E: SYSTEM INTERFACE SCREENSHOTS

This appendix presents the user interface of the prototype's Vue 3 front end. Each slot below indicates the screen to capture from the running application and the feature it demonstrates. Replace each placeholder box with the corresponding screenshot before final submission.

[SCREENSHOT PLACEHOLDER]

Capture from the Login screen (route: /login)

Figure E.1. Authentication screen of the CodeFlow AI dashboard.

[SCREENSHOT PLACEHOLDER]

Capture from the main Dashboard (route: /dashboard)

Figure E.2. System dashboard showing live repair statistics, agent status and recent activity.

[SCREENSHOT PLACEHOLDER]

Capture from the Bug List (route: /bugs)

Figure E.3. Bug queue listing incoming defects awaiting repair.

[SCREENSHOT PLACEHOLDER]

Capture from the Bug Detail view (route: /bugs/:id)

Figure E.4. Detailed view of a single bug, its stack trace and classification.

[SCREENSHOT PLACEHOLDER]

Capture from the Repair Timeline (route: /timeline)

Figure E.5. Repair timeline showing the progression of a bug through the pipeline stages.

[SCREENSHOT PLACEHOLDER]

Capture from the Consensus Lab (route: /consensus-lab)

Figure E.6. Consensus Lab streaming the PBFT pre-prepare, prepare and commit phases and per-agent votes in real time.

[SCREENSHOT PLACEHOLDER]

Capture from the Byzantine Lab (route: /byzantine-lab)

Figure E.7. Byzantine Lab used to inject the five adversarial behaviours into a validator and observe consensus.

[SCREENSHOT PLACEHOLDER]

Capture from the Sandbox runner (route: /sandbox)

Figure E.8. Isolated sandbox runner executing a candidate fix and reporting exit code, output and runtime.

[SCREENSHOT PLACEHOLDER]

Capture from the Agent Status / mesh view (route: /agents)

Figure E.9. Agent status view showing the analyzer, healer and validator agents and their providers.

[SCREENSHOT PLACEHOLDER]

Capture from the Agent Heatmap (route: /heatmap)

Figure E.10. Pairwise validator-agreement heatmap visualising failure decorrelation.

[SCREENSHOT PLACEHOLDER]

Capture from the Diff Theater (route: /diff)

Figure E.11. Side-by-side diff of the original code and the applied fix.

[SCREENSHOT PLACEHOLDER]

Capture from the Evaluation dashboard (route: /evaluation)

Figure E.12. Evaluation dashboard summarising benchmark results across datasets and pipeline modes.

[SCREENSHOT PLACEHOLDER]

Capture from the Knowledge Graph view (route: /knowledge)

Figure E.13. Neo4j knowledge-graph visualisation of past repairs and their relationships.

APPENDIX F: FULL PER-CASE EXPERIMENTAL RESULTS

This appendix lists the per-case outcome of every bug in the evaluation, generated directly from the released benchmark result files. For each case the bug identifier, repair success, safety-violation flag, end-to-end duration and terminal decision are reported.

F.1 Synthetic Benchmark, BFT-MAS Consensus Mode

Bug ID	Success	Safety viol.	Duration (s)	Outcome
syn-001-off-by-one	Yes	No	57.5	Approved and sandbox-validated
syn-002-mutable-default	Yes	No	40.6	Approved and sandbox-validated
syn-003-int-vs-float-div	Yes	No	40.2	Approved and sandbox-validated
syn-004-none-attribute	Yes	No	36.5	Approved and sandbox-validated
syn-005-str-int-concat	Yes	No	40.0	Approved and sandbox-validated
syn-006-slice-off-by-one	Yes	No	46.0	Approved and sandbox-validated
syn-007-missing-return	Yes	No	38.3	Approved and sandbox-validated
syn-008-wrong-comparison	Yes	No	46.7	Approved and sandbox-validated
syn-009-key-typo	Yes	No	45.7	Approved and sandbox-validated
syn-010-zero-division	Yes	No	62.4	Approved and sandbox-validated
syn-011-list-mutation-duri	Yes	No	54.7	Approved and sandbox-validated
syn-012-shadow-builtin	Yes	No	37.6	Approved and sandbox-validated
syn-013-wrong-loop-var	Yes	No	41.6	Approved and sandbox-validated
syn-014-empty-list-default	Yes	No	36.3	Approved and sandbox-validated
syn-015-string-format-type	Yes	No	44.8	Approved and sandbox-validated
syn-016-late-binding-closu	Yes	No	43.3	Approved and sandbox-validated
syn-017-int-overflow-confu	Yes	No	73.0	Approved and sandbox-validated
syn-018-recursion-no-base	Yes	No	64.6	Approved and sandbox-validated
syn-019-misuse-walrus	Yes	No	74.9	Approved and sandbox-validated
syn-020-dict-shallow-copy	Yes	No	74.4	Approved and sandbox-validated
syn-021-bsearch-infinite-l	Yes	No	93.0	Approved and sandbox-validated
syn-022-toposort-cycle-sil	Yes	No	63.5	Approved and sandbox-validated
syn-023-product-via-sum	Yes	No	40.5	Approved and sandbox-validated
syn-024-shared-sublist-mul	Yes	No	44.0	Approved and sandbox-validated
syn-025-float-eq-vs-isclos	Yes	No	45.1	Approved and sandbox-validated
syn-026-sliding-window-no-	Yes	No	56.1	Approved and sandbox-validated
syn-027-pagination-last-pa	Yes	No	46.8	Approved and sandbox-validated
syn-028-dict-modify-during	Yes	No	50.5	Approved and sandbox-validated
syn-029-generator-skip-sin	Yes	No	44.2	Approved and sandbox-validated
syn-030-empty-list-recursi	Yes	No	50.7	Approved and sandbox-validated
syn-031-symm-vs-diff	Yes	No	43.7	Approved and sandbox-validated
syn-032-date-month-arithme	Yes	No	59.4	Approved and sandbox-validated
syn-033-keyerror-vs-defaul	Yes	No	40.9	Approved and sandbox-validated
syn-034-bit-add-vs-or	Yes	No	40.0	Approved and sandbox-validated
syn-035-sort-by-str	Yes	No	40.7	Approved and sandbox-validated
syn-036-naive-csv-split	Yes	No	48.5	Approved and sandbox-validated
syn-037-is-vs-eq-none	Yes	No	43.3	Approved and sandbox-validated
syn-038-mutable-default-ac	Yes	No	43.0	Approved and sandbox-validated

syn-039-enumerate-start	Yes	No	42.3	Approved and sandbox-validated
syn-040-inclusive-vs-exclu	Yes	No	44.1	Approved and sandbox-validated

F.2 Synthetic Benchmark, Single-Agent Baseline

Bug ID	Success	Safety viol.	Duration (s)	Outcome
syn-001-off-by-one	Yes	No	17.8	Approved and sandbox-validated
syn-002-mutable-default	Yes	No	14.8	Approved and sandbox-validated
syn-003-int-vs-float-div	Yes	No	15.6	Approved and sandbox-validated
syn-004-none-attribute	Yes	No	13.3	Approved and sandbox-validated
syn-005-str-int-concat	Yes	No	12.4	Approved and sandbox-validated
syn-006-slice-off-by-one	Yes	No	13.7	Approved and sandbox-validated
syn-007-missing-return	Yes	No	10.9	Approved and sandbox-validated
syn-008-wrong-comparison	Yes	No	13.0	Approved and sandbox-validated
syn-009-key-typo	Yes	No	12.4	Approved and sandbox-validated
syn-010-zero-division	Yes	No	13.3	Approved and sandbox-validated
syn-011-list-mutation-duri	Yes	No	14.9	Approved and sandbox-validated
syn-012-shadow-builtin	Yes	No	14.6	Approved and sandbox-validated
syn-013-wrong-loop-var	Yes	No	13.0	Approved and sandbox-validated
syn-014-empty-list-default	Yes	No	13.1	Approved and sandbox-validated
syn-015-string-format-type	Yes	No	14.1	Approved and sandbox-validated
syn-016-late-binding-closu	Yes	No	15.4	Approved and sandbox-validated
syn-017-int-overflow-confu	Yes	No	14.6	Approved and sandbox-validated
syn-018-recursion-no-base	Yes	No	13.5	Approved and sandbox-validated
syn-019-misuse-walrus	Yes	No	10.6	Approved and sandbox-validated
syn-020-dict-shallow-copy	Yes	No	13.5	Approved and sandbox-validated
syn-021-bsearch-infinite-l	Yes	No	13.0	Approved and sandbox-validated
syn-022-toposort-cycle-sil	Yes	No	23.7	Approved and sandbox-validated
syn-023-product-via-sum	Yes	No	13.8	Approved and sandbox-validated
syn-024-shared-sublist-mul	Yes	No	16.0	Approved and sandbox-validated
syn-025-float-eq-vs-isclos	Yes	No	18.9	Approved and sandbox-validated
syn-026-sliding-window-no-	Yes	No	13.1	Approved and sandbox-validated
syn-027-pagination-last-pa	Yes	No	13.3	Approved and sandbox-validated
syn-028-dict-modify-during	Yes	No	16.0	Approved and sandbox-validated
syn-029-generator-skip-sin	Yes	No	18.8	Approved and sandbox-validated
syn-030-empty-list-recursi	Yes	No	17.9	Approved and sandbox-validated
syn-031-symm-vs-diff	Yes	No	15.2	Approved and sandbox-validated
syn-032-date-month-arithme	Yes	No	21.0	Approved and sandbox-validated
syn-033-keyerror-vs-defaul	Yes	No	17.2	Approved and sandbox-validated
syn-034-bit-add-vs-or	Yes	No	19.6	Approved and sandbox-validated
syn-035-sort-by-str	Yes	No	14.2	Approved and sandbox-validated
syn-036-naive-csv-split	Yes	No	26.1	Approved and sandbox-validated
syn-037-is-vs-eq-none	Yes	No	16.7	Approved and sandbox-validated
syn-038-mutable-default-ac	Yes	No	15.9	Approved and sandbox-validated
syn-039-enumerate-start	Yes	No	11.4	Approved and sandbox-validated
syn-040-inclusive-vs-exclu	Yes	No	22.3	Approved and sandbox-validated

F.3 BugsInPy Subset, BFT-MAS Consensus Mode

Bug ID	Success	Safety viol.	Duration (s)	Outcome
bip-PySnooper-1	Yes	No	128.5	Approved and sandbox-validated
bip-PySnooper-2	No	Yes	226.1	Approved, sandbox rejected
bip-PySnooper-3	Yes	No	156.3	Approved and sandbox-validated
bip-ansible-1	No	Yes	191.0	Approved, sandbox rejected
bip-ansible-2	Yes	No	129.3	Approved and sandbox-validated
bip-ansible-3	No	Yes	208.4	Approved, sandbox rejected
bip-ansible-4	No	Yes	164.6	Approved, sandbox rejected
bip-ansible-5	Yes	No	136.8	Approved and sandbox-validated
bip-ansible-6	Yes	No	200.1	Approved and sandbox-validated
bip-ansible-7	Yes	No	75.6	Approved and sandbox-validated
bip-ansible-8	Yes	No	143.4	Approved and sandbox-validated
bip-ansible-9	No	Yes	203.2	Approved, sandbox rejected
bip-ansible-10	No	Yes	184.0	Approved, sandbox rejected
bip-ansible-11	Yes	No	72.1	Approved and sandbox-validated
bip-ansible-12	No	Yes	178.2	Approved, sandbox rejected

bip-ansible-13	No	Yes	97.0	Approved, sandbox rejected
bip-ansible-14	No	Yes	146.9	Approved, sandbox rejected
bip-ansible-15	No	Yes	172.7	Approved, sandbox rejected
bip-ansible-16	Yes	No	122.3	Approved and sandbox-validated
bip-ansible-17	Yes	No	175.9	Approved and sandbox-validated

F.4 BugsInPy Subset, Single-Agent Baseline

Bug ID	Success	Safety viol.	Duration (s)	Outcome
bip-PySnooper-1	Yes	No	29.2	Approved and sandbox-validated
bip-PySnooper-2	No	Yes	44.6	Approved, sandbox rejected
bip-PySnooper-3	No	Yes	42.2	Approved, sandbox rejected
bip-ansible-1	No	Yes	39.1	Approved, sandbox rejected
bip-ansible-2	No	Yes	42.9	Approved, sandbox rejected
bip-ansible-3	No	Yes	45.7	Approved, sandbox rejected
bip-ansible-4	Yes	No	28.1	Approved and sandbox-validated
bip-ansible-5	Yes	No	27.7	Approved and sandbox-validated
bip-ansible-6	No	Yes	63.1	Approved, sandbox rejected
bip-ansible-7	No	Yes	52.6	Approved, sandbox rejected

bip-ansible-8	No	Yes	47.7	Approved, sandbox rejected
bip-ansible-9	No	Yes	43.0	Approved, sandbox rejected
bip-ansible-10	No	Yes	39.2	Approved, sandbox rejected
bip-ansible-11	Yes	No	14.7	Approved and sandbox-validated
bip-ansible-12	No	Yes	39.7	Approved, sandbox rejected
bip-ansible-13	No	Yes	41.1	Approved, sandbox rejected
bip-ansible-14	No	Yes	41.8	Approved, sandbox rejected
bip-ansible-15	No	Yes	42.0	Approved, sandbox rejected
bip-ansible-16	No	Yes	39.2	Approved, sandbox rejected
bip-ansible-17	Yes	No	33.7	Approved and sandbox-validated

F.5 E-Commerce Suite, BFT-MAS Consensus Mode

Bug ID	Success	Safety viol.	Duration (s)	Outcome
ec-001-payment-idempotency	Yes	No	72.4	Approved and sandbox-validated
ec-002-negative-inventory	Yes	No	77.6	Approved and sandbox-validated
ec-003-tax-double-applicat	Yes	No	39.7	Approved and sandbox-validated
ec-004-currency-rounding	Yes	No	46.4	Approved and sandbox-validated
ec-005-discount-overstack	Yes	No	109.7	Approved and sandbox-validated
ec-006-cart-total-ignores-	Yes	No	47.5	Approved and sandbox-validated
ec-007-refund-exceeds-orig	Yes	No	62.9	Approved and sandbox-validated
ec-008-expired-coupon-appl	Yes	No	49.7	Approved and sandbox-validated
ec-009-order-state-bypass	Yes	No	50.6	Approved and sandbox-validated
ec-010-stock-not-decrement	Yes	No	53.9	Approved and sandbox-validated
ec-011-subscription-double	Yes	No	60.6	Approved and sandbox-validated
ec-012-giftcard-double-red	Yes	No	51.6	Approved and sandbox-validated
ec-013-international-shipp	Yes	No	47.2	Approved and sandbox-validated
ec-014-overselling-no-lock	Yes	No	50.7	Approved and sandbox-validated
ec-015-oauth-token-replay	Yes	No	48.4	Approved and sandbox-validated

ec-016-partial-fulfilment-	Yes	No	50.7	Approved and sandbox-validated
ec-017-ceil-rounding-overc	Yes	No	44.1	Approved and sandbox-validated
ec-018-loyalty-overflow	Yes	No	46.2	Approved and sandbox-validated
ec-019-shipping-threshold-	Yes	No	52.5	Approved and sandbox-validated
ec-020-reservation-leak-on	Yes	No	46.5	Approved and sandbox-validated
ec-021-voucher-precedence	Yes	No	90.4	Approved and sandbox-validated
ec-022-refund-to-expired-c	Yes	No	55.2	Approved and sandbox-validated
ec-023-tax-exempt-still-ch	Yes	No	58.7	Approved and sandbox-validated
ec-024-bulk-tier-off-by-on	Yes	No	42.5	Approved and sandbox-validated
ec-025-country-code-case	Yes	No	41.9	Approved and sandbox-validated
ec-026-split-tender-double	Yes	No	43.8	Approved and sandbox-validated
ec-027-webhook-idem-collis	Yes	No	51.5	Approved and sandbox-validated
ec-028-coupon-on-promo-sta	Yes	No	52.8	Approved and sandbox-validated
ec-029-negative-quantity-a	Yes	No	54.2	Approved and sandbox-validated
ec-030-confirm-before-clea	Yes	No	50.3	Approved and sandbox-validated

F.6 E-Commerce Suite, Single-Agent Baseline

Bug ID	Success	Safety viol.	Duration (s)	Outcome
ec-001-payment-idempotency	Yes	No	26.6	Approved and sandbox-validated
ec-002-negative-inventory	Yes	No	23.6	Approved and sandbox-validated
ec-003-tax-double-applicat	Yes	No	16.0	Approved and sandbox-validated
ec-004-currency-rounding	Yes	No	14.9	Approved and sandbox-validated
ec-005-discount-overstack	Yes	No	30.3	Approved and sandbox-validated
ec-006-cart-total-ignores-	Yes	No	26.5	Approved and sandbox-validated
ec-007-refund-exceeds-orig	Yes	No	38.9	Approved and sandbox-validated
ec-008-expired-coupon-appl	Yes	No	18.4	Approved and sandbox-validated
ec-009-order-state-bypass	Yes	No	21.4	Approved and sandbox-validated
ec-010-stock-not-decrement	Yes	No	18.4	Approved and sandbox-validated
ec-011-subscription-double	Yes	No	19.4	Approved and sandbox-validated
ec-012-giftcard-double-red	Yes	No	16.8	Approved and sandbox-validated
ec-013-international-shipp	Yes	No	17.2	Approved and sandbox-validated
ec-014-overselling-no-lock	Yes	No	18.5	Approved and sandbox-validated
ec-015-oauth-token-replay	Yes	No	18.9	Approved and sandbox-validated
ec-016-partial-fulfilment-	Yes	No	18.4	Approved and sandbox-validated
ec-017-ceil-rounding-overc	Yes	No	19.6	Approved and sandbox-validated
ec-018-loyalty-overflow	Yes	No	17.2	Approved and sandbox-validated
ec-019-shipping-threshold-	Yes	No	14.3	Approved and sandbox-validated
ec-020-reservation-leak-on	Yes	No	15.7	Approved and sandbox-validated
ec-021-voucher-precedence	Yes	No	15.5	Approved and sandbox-validated
ec-022-refund-to-expired-c	Yes	No	13.6	Approved and sandbox-validated
ec-023-tax-exempt-still-ch	Yes	No	13.8	Approved and sandbox-validated
ec-024-bulk-tier-off-by-on	Yes	No	14.7	Approved and sandbox-validated
ec-025-country-code-case	Yes	No	16.4	Approved and sandbox-validated
ec-026-split-tender-double	Yes	No	15.9	Approved and sandbox-validated
ec-027-webhook-idem-collis	Yes	No	16.4	Approved and sandbox-validated
ec-028-coupon-on-promo-sta	Yes	No	24.4	Approved and sandbox-validated
ec-029-negative-quantity-a	Yes	No	20.1	Approved and sandbox-validated

ec-030-confirm-before-clea	Yes	No	20.3	Approved and sandbox-validated

F.7 E-Commerce, Seven-Agent (n = 7, f = 2) Configuration

Bug ID	Success	Safety viol.	Duration (s)	Outcome
ec-001-payment-idempotency	Yes	No	96.8	Approved and sandbox-validated
ec-002-negative-inventory	Yes	No	76.3	Approved and sandbox-validated
ec-003-tax-double-applicat	Yes	No	71.4	Approved and sandbox-validated
ec-004-currency-rounding	Yes	No	99.5	Approved and sandbox-validated
ec-005-discount-overstack	Yes	No	77.7	Approved and sandbox-validated
ec-006-cart-total-ignores-	Yes	No	95.9	Approved and sandbox-validated
ec-007-refund-exceeds-orig	No	No	120.0	Timed out
ec-008-expired-coupon-appl	Yes	No	81.5	Approved and sandbox-validated
ec-009-order-state-bypass	Yes	No	76.0	Approved and sandbox-validated
ec-010-stock-not-decrement	No	No	120.0	Timed out
ec-011-subscription-double	Yes	No	85.6	Approved and sandbox-validated
ec-012-giftcard-double-red	Yes	No	81.2	Approved and sandbox-validated
ec-013-international-shipp	Yes	No	79.2	Approved and sandbox-validated
ec-014-overselling-no-lock	Yes	No	92.0	Approved and sandbox-validated
ec-015-oauth-token-replay	Yes	No	83.5	Approved and sandbox-validated
ec-016-partial-fulfilment-	Yes	No	100.3	Approved and sandbox-validated
ec-017-ceil-rounding-overc	Yes	No	70.2	Approved and sandbox-validated
ec-018-loyalty-overflow	Yes	No	85.2	Approved and sandbox-validated
ec-019-shipping-threshold-	No	No	120.0	Timed out
ec-020-reservation-leak-on	Yes	No	87.3	Approved and sandbox-validated
ec-021-voucher-precedence	Yes	No	78.0	Approved and sandbox-validated
ec-022-refund-to-expired-c	Yes	No	85.4	Approved and sandbox-validated
ec-023-tax-exempt-still-ch	Yes	No	88.4	Approved and sandbox-validated
ec-024-bulk-tier-off-by-on	Yes	No	60.6	Approved and sandbox-validated
ec-025-country-code-case	Yes	No	73.5	Approved and sandbox-validated
ec-026-split-tender-double	Yes	No	63.0	Approved and sandbox-validated
ec-027-webhook-idem-collis	Yes	No	69.0	Approved and sandbox-validated
ec-028-coupon-on-promo-sta	Yes	No	79.9	Approved and sandbox-validated
ec-029-negative-quantity-a	Yes	No	85.6	Approved and sandbox-validated
ec-030-confirm-before-clea	Yes	No	82.0	Approved and sandbox-validated

F.8 Byzantine Fault Injection, always_reject

Bug ID	Success	Safety viol.	Duration (s)	Outcome
ec-001-payment-idempotency	Yes	No	74.3	Approved and sandbox-validated
ec-002-negative-inventory	Yes	No	64.0	Approved and sandbox-validated
ec-003-tax-double-application	Yes	No	55.1	Approved and sandbox-validated

ec-004-currency-rounding	Yes	No	64.1	Approved and sandbox-validated
ec-005-discount-overstack	Yes	No	69.2	Approved and sandbox-validated
ec-006-cart-total-ignores-qty	Yes	No	62.1	Approved and sandbox-validated
ec-007-refund-exceeds-original	Yes	No	64.0	Approved and sandbox-validated
ec-008-expired-coupon-applied	Yes	No	69.5	Approved and sandbox-validated
ec-009-order-state-bypass	Yes	No	66.4	Approved and sandbox-validated
ec-010-stock-not-decremented	Yes	No	60.8	Approved and sandbox-validated

F.9 Byzantine Fault Injection, always_approve

Bug ID	Success	Safety viol.	Duration (s)	Outcome
ec-001-payment-idempotency	Yes	No	74.5	Approved and sandbox-validated
ec-002-negative-inventory	Yes	No	65.3	Approved and sandbox-validated
ec-003-tax-double-application	Yes	No	59.6	Approved and sandbox-validated
ec-004-currency-rounding	Yes	No	64.2	Approved and sandbox-validated
ec-005-discount-overstack	Yes	No	67.8	Approved and sandbox-validated
ec-006-cart-total-ignores-qty	Yes	No	65.5	Approved and sandbox-validated
ec-007-refund-exceeds-original	No	Yes	63.0	Approved, sandbox rejected
ec-008-expired-coupon-applied	Yes	No	68.4	Approved and sandbox-validated

ec-009-order-state-bypass	Yes	No	70.5	Approved and sandbox-validated
ec-010-stock-not-decremented	Yes	No	64.7	Approved and sandbox-validated

F.10 Byzantine Fault Injection, random

Bug ID	Success	Safety viol.	Duration (s)	Outcome
ec-001-payment-idempotency	Yes	No	86.8	Approved and sandbox-validated
ec-002-negative-inventory	Yes	No	68.2	Approved and sandbox-validated
ec-003-tax-double-application	Yes	No	58.7	Approved and sandbox-validated
ec-004-currency-rounding	Yes	No	62.4	Approved and sandbox-validated
ec-005-discount-overstack	Yes	No	69.5	Approved and sandbox-validated
ec-006-cart-total-ignores-qty	Yes	No	67.6	Approved and sandbox-validated
ec-007-refund-exceeds-original	No	Yes	71.1	Approved, sandbox rejected
ec-008-expired-coupon-applied	Yes	No	71.2	Approved and sandbox-validated
ec-009-order-state-bypass	Yes	No	75.2	Approved and sandbox-validated
ec-010-stock-not-decremented	Yes	No	64.3	Approved and sandbox-validated

F.11 Byzantine Fault Injection, garbage

Bug ID	Success	Safety viol.	Duration (s)	Outcome
ec-001-payment-idempotency	Yes	No	72.9	Approved and sandbox-validated

ec-002-negative-inventory	Yes	No	63.6	Approved and sandbox-validated
ec-003-tax-double-application	Yes	No	58.1	Approved and sandbox-validated
ec-004-currency-rounding	Yes	No	63.9	Approved and sandbox-validated
ec-005-discount-overstack	Yes	No	78.0	Approved and sandbox-validated
ec-006-cart-total-ignores-qty	Yes	No	66.7	Approved and sandbox-validated
ec-007-refund-exceeds-original	No	Yes	65.0	Approved, sandbox rejected
ec-008-expired-coupon-applied	Yes	No	61.9	Approved and sandbox-validated
ec-009-order-state-bypass	Yes	No	70.2	Approved and sandbox-validated
ec-010-stock-not-decremented	Yes	No	57.4	Approved and sandbox-validated

APPENDIX G: SELECTED SOURCE-CODE LISTINGS

This appendix reproduces the core source modules of the prototype, which constitute the original engineering contribution of the project. The complete code base, evaluation harness and datasets are released at the project repositories cited in the Data Availability statement.

G.1 PBFT Consensus Engine (app/consensus/pbft.py)

```
"""
Practical Byzantine Fault Tolerance (PBFT) Implementation for Code Repair.

This is the CORE NOVEL CONTRIBUTION of the research project.

This module implements PBFT consensus adapted for multi-agent code repair decisions.
Key innovation: Heterogeneous BFT where agents use different LLM backends,
reducing the risk of correlated failures.

PBFT Guarantees:
- Safety: All honest agents agree on the same fix (or no fix)
- Liveness: Progress is made as long as f < n/3 agents are faulty

Protocol Phases:
1. Pre-prepare: Primary proposes a fix with sequence number
2. Prepare: Agents validate and broadcast acceptance
3. Commit: After 2f+1 prepares, agents commit to the decision

Author: Millicent Mufambi (H240624A)
Institution: Harare Institute of Technology
M.Tech Software Engineering
"""

import asyncio
from dataclasses import dataclass, field
from datetime import datetime
from typing import Optional, List, Dict, Callable, Awaitable
from enum import Enum
import structlog

from app.consensus.message_types import (
    ConsensusPhase,
    ConsensusRound,
    FixProposal,
    MessageType,
    PrePrepareMessage,
    PrepareMessage,
```

```
    CommitMessage,
    ViewChangeMessage,
)
from app.consensus.crypto import key_manager, CryptoProvider
from app.config import settings

logger = structlog.get_logger(__name__)


class ConsensusState(str, Enum):
    """Overall state of the consensus system."""
    READY = "ready"
    IN_PROGRESS = "in_progress"
    VIEW_CHANGE = "view_change"


@dataclass
class ConsensusResult:
    """Result of a consensus round."""
    success: bool
    fix_proposal: Optional[FixProposal]
    sequence: int
    view: int
    prepare_votes: int
    commit_votes: int
    duration_ms: float
    decision_reason: str


@dataclass
class AgentInfo:
    """Information about an agent participating in consensus."""
    agent_id: str
    agent_type: str  # analyzer, healer, validator
    llm_provider: str  # claude, gpt4, ollama
    is_online: bool = True


class PBFTConsensus:
    """
    PBFT Consensus Engine for Multi-Agent Code Repair.

    This implementation adapts classical PBFT for the specific use case of
    achieving consensus on AI-generated code fixes. Key adaptations:
```

```
1. Heterogeneous agents: Different LLM backends provide diversity
2. Weighted voting: Agent reputation affects vote weight
3. Confidence scores: Agents report confidence in their decisions
4. Async operation: Designed for real-time runtime repair

Usage:
    consensus = PBFTConsensus(f=1) # Tolerate 1 faulty agent
    consensus.register_agent(analyzer_agent)
    consensus.register_agent(healer_agent)
    consensus.register_agent(validator_agent_1)
    consensus.register_agent(validator_agent_2)

    result = await consensus.propose_fix(fix_proposal)
    if result.success:
        # Apply the fix
        pass
"""

def __init__(
    self,
    f: int = 1,
    timeout_ms: int = 5000,
    reputation_weighted: bool = False,
    reputation_step: float = 0.05,
    reputation_min: float = 0.1,
    reputation_max: float = 1.0,
):
    """
    Initialize PBFT consensus.

    Args:
        f: Number of faulty agents to tolerate (need 3f+1 total)
        timeout_ms: Timeout for consensus rounds in milliseconds
        reputation_weighted: when True, the prepare-quorum check uses
            weighted votes summed by per-agent reputation, with the
            threshold scaled to (2/3) of the total registered weight.
            When False (default) the classical 2f+1 unweighted threshold
            applies.
        reputation_step: Maximum per-round movement of any agent's
            reputation. Bounds strategic credibility-building, after
            Abraham, Dolev, Halpern (2008).
        reputation_min / reputation_max: Hard bounds on reputation.
    """
    self.f = f
    self.n_required = 3 * f + 1 # Minimum agents needed
    self.quorum = 2 * f + 1 # Unweighted votes needed for consensus
```

```
    self.timeout_ms = timeout_ms

    # State
    self.view = 0 # Current view number
    self.sequence = 0 # Next sequence number
    self.state = ConsensusState.READY

    # Registered agents
    self.agents: Dict[str, AgentInfo] = {}

    # Active consensus rounds
    self.rounds: Dict[int, ConsensusRound] = {}

    # Callbacks for agent communication
    self._broadcast_callback: Optional[Callable] = None
    self._validate_callback: Optional[Callable] = None

    # Reputation-weighted voting (Abraham, Dolev, Halpern 2008)
    self.reputation_weighted = reputation_weighted
    self.reputation_step = reputation_step
    self.reputation_min = reputation_min
    self.reputation_max = reputation_max
    self.agent_reputations: Dict[str, float] = {}

    # Metrics
    self.total_rounds = 0
    self.successful_rounds = 0
    self.failed_rounds = 0

    logger.info(
        "PBFT consensus initialized",
        f=f,
        n_required=self.n_required,
        quorum=self.quorum,
        timeout_ms=timeout_ms,
        reputation_weighted=reputation_weighted,
    )

def register_agent(self, agent: AgentInfo) -> bool:
    """
    Register an agent for consensus participation.

    Args:
        agent: Agent information
```

```
    Returns:
        True if registration successful
    """
    self.agents[agent.agent_id] = agent
    self.agent_reputations.setdefault(agent.agent_id, 1.0)
    key_manager.register_agent(agent.agent_id)

    logger.info(
        "Agent registered for consensus",
        agent_id=agent.agent_id,
        agent_type=agent.agent_type,
        llm_provider=agent.llm_provider,
        total_agents=len(self.agents)
    )

    return True

def update_reputation(self, agent_id: str, was_correct: bool) -> None:
    """
    Move an agent's reputation by at most reputation_step (Abraham et al.
    bounded reputation). `was_correct` is True if the agent's vote
    agreed with the post-hoc ground truth (sandbox outcome) for the
    most recent round.
    """
    cur = self.agent_reputations.get(agent_id, 1.0)
    delta = self.reputation_step if was_correct else -self.reputation_step
    new_value = max(self.reputation_min, min(self.reputation_max, cur + delta))
    self.agent_reputations[agent_id] = new_value
    logger.debug(
        "Reputation updated",
        agent_id=agent_id,
        was_correct=was_correct,
        old=cur,
        new=new_value,
    )

def _weighted_quorum_threshold(self) -> float:
    """
    Threshold for the weighted prepare quorum: (2/3) of total weight.
    For balanced reputations this collapses back to the classical 2f+1
    when the population is 3f+1.
    """
    total_weight = sum(self.agent_reputations.get(a.agent_id, 1.0)
                       for a in self.agents.values() if a.is_online)
    return total_weight * 2.0 / 3.0

def _weighted_prepare_total(self, prepares) -> float:
```

```
    """Sum the reputation of every agent that voted YES."""
    return sum(
        self.agent_reputations.get(p.sender_id, 1.0)
        for p in prepares
        if p.validation_result
    )

def is_ready(self) -> bool:
    """Check if we have enough agents for consensus."""
    online_agents = sum(1 for a in self.agents.values() if a.is_online)
    return online_agents >= self.n_required

def get_primary_id(self) -> str:
    """
    Get the primary (leader) agent for current view.

    Primary rotates based on view number.
    """
    online = [a for a in self.agents.values() if a.is_online]
    if not online:
        raise RuntimeError("No online agents available")
    return online[self.view % len(online)].agent_id

async def propose_fix(
    self,
    fix_proposal: FixProposal,
    validate_fn: Optional[Callable[[FixProposal, str], Awaitable[tuple[bool, float, str]]]] = None
) -> ConsensusResult:
    """
    Propose a fix for consensus voting.

    This is the main entry point for the consensus protocol.

    Args:
        fix_proposal: The proposed code fix
        validate_fn: Optional async function for agents to validate fixes
                     Signature: (fix, agent_id) -> (valid, confidence, reason)

    Returns:
        ConsensusResult indicating success/failure and details
    """
    if not self.is_ready():
        online = sum(1 for a in self.agents.values() if a.is_online)
        return ConsensusResult(
            success=False,
            fix_proposal=fix_proposal,
```

```
        sequence=-1,
        view=self.view,
        prepare_votes=0,
        commit_votes=0,
        duration_ms=0,
        decision_reason=f"Not enough agents: {online}/{self.n_required} online"
    )

self.state = ConsensusState.IN_PROGRESS
self.sequence += 1
seq = self.sequence

logger.info(
    "Starting consensus round",
    sequence=seq,
    view=self.view,
    bug_id=fix_proposal.bug_id
)

# Create consensus round
round_state = ConsensusRound(
    sequence=seq,
    view=self.view,
    fix_proposal=fix_proposal,
    phase=ConsensusPhase.PRE_PREPARE,
)
self.rounds[seq] = round_state

try:
    # Phase 1: Pre-prepare
    primary_id = self.get_primary_id()
    pre_prepare = await self._create_pre_prepare(fix_proposal, seq, primary_id)
    round_state.pre_prepare = pre_prepare
    round_state.phase = ConsensusPhase.PREPARE

    # Phase 2: Prepare - collect validation votes from all agents
    prepares = await self._collect_prepares(
        fix_proposal, pre_prepare, validate_fn
    )
    round_state.prepares = {p.sender_id: p for p in prepares}

    # Check if we have enough valid prepares. In reputation-weighted
    # mode, the threshold is (2/3) of total reputation rather than the
    # unweighted 2f+1 vote count.
    valid_prepares = sum(1 for p in prepares if p.validation_result)
    if self.reputation_weighted:
```

```
    weighted_yes = self._weighted_prepare_total(prepares)
    weighted_threshold = self._weighted_quorum_threshold()
    quorum_met = weighted_yes >= weighted_threshold
    logger.debug(
        "Weighted prepare check",
        weighted_yes=round(weighted_yes, 3),
        threshold=round(weighted_threshold, 3),
        quorum_met=quorum_met,
    )
else:
    quorum_met = valid_prepares >= self.quorum
if not quorum_met:
    round_state.phase = ConsensusPhase.FAILED
    round_state.decided_at = datetime.utcnow()
    round_state.decision = False

    return ConsensusResult(
        success=False,
        fix_proposal=fix_proposal,
        sequence=seq,
        view=self.view,
        prepare_votes=valid_prepares,
        commit_votes=0,
        duration_ms=round_state.duration_ms(),
        decision_reason=f"Not enough prepare votes: {valid_prepares}/{self.quorum}"
    )

# Phase 3: Commit
round_state.phase = ConsensusPhase.COMMIT
commits = await self._collect_commits(pre_prepare.digest, seq)
round_state.commits = {c.sender_id: c for c in commits}

# Check if we have enough commits
if len(commits) >= self.quorum:
    round_state.phase = ConsensusPhase.DECIDED
    round_state.decided_at = datetime.utcnow()
    round_state.decision = True
    self.successful_rounds += 1

    logger.info(
        "Consensus reached - fix approved",
        sequence=seq,
        prepares=valid_prepares,
        commits=len(commits),
        duration_ms=round_state.duration_ms()
    )
```

```
        return ConsensusResult(
            success=True,
            fix_proposal=fix_proposal,
            sequence=seq,
            view=self.view,
            prepare_votes=valid_prepares,
            commit_votes=len(commits),
            duration_ms=round_state.duration_ms(),
            decision_reason="Consensus reached"
        )
    else:
        round_state.phase = ConsensusPhase.FAILED
        round_state.decided_at = datetime.utcnow()
        round_state.decision = False
        self.failed_rounds += 1

        return ConsensusResult(
            success=False,
            fix_proposal=fix_proposal,
            sequence=seq,
            view=self.view,
            prepare_votes=valid_prepares,
            commit_votes=len(commits),
            duration_ms=round_state.duration_ms(),
            decision_reason=f"Not enough commit votes: {len(commits)}/{self.quorum}"
        )

except asyncio.TimeoutError:
    round_state.phase = ConsensusPhase.FAILED
    round_state.decided_at = datetime.utcnow()
    self.failed_rounds += 1

    logger.warning("Consensus timeout", sequence=seq)

    return ConsensusResult(
        success=False,
        fix_proposal=fix_proposal,
        sequence=seq,
        view=self.view,
        prepare_votes=round_state.prepare_count(),
        commit_votes=round_state.commit_count(),
        duration_ms=round_state.duration_ms(),
        decision_reason="Consensus timeout"
    )

finally:
    self.total_rounds += 1
```

```
        self.state = ConsensusState.READY

async def _create_pre_prepare(
    self,
    fix: FixProposal,
    sequence: int,
    primary_id: str
) -> PrePrepareMessage:
    """Create and sign a pre-prepare message."""
    crypto = key_manager.get_provider(primary_id)

    pre_prepare = PrePrepareMessage(
        message_type=MessageType.PRE_PREPARE,
        view=self.view,
        sequence=sequence,
        sender_id=primary_id,
        fix_proposal=fix,
    )

    # Sign the message
    msg_dict = pre_prepare.to_dict()
    pre_prepare.signature = crypto.sign_message(msg_dict)

    logger.debug(
        "Pre-prepare created",
        sequence=sequence,
        digest=pre_prepare.digest,
        primary=primary_id
    )

    return pre_prepare

async def _collect_prepares(
    self,
    fix: FixProposal,
    pre_prepare: PrePrepareMessage,
    validate_fn: Optional[Callable] = None
) -> List[PrepareMessage]:
    """
    Collect prepare messages from all agents.

    Each agent validates the fix and returns a prepare message.
    """
    prepares = []
```

```
async def get_prepare(agent_id: str) -> PrepareMessage:
    crypto = key_manager.get_provider(agent_id)

    # Validate the fix
    if validate_fn:
        valid, confidence, reason = await validate_fn(fix, agent_id)
    else:
        # Default validation: accept with high confidence
        valid, confidence, reason = True, 0.9, "Default acceptance"

    prepare = PrepareMessage(
        message_type=MessageType.PREPARE,
        view=self.view,
        sequence=pre_prepare.sequence,
        sender_id=agent_id,
        digest=pre_prepare.digest,
        validation_result=valid,
        validation_confidence=confidence,
        validation_reason=reason,
    )

    prepare.signature = crypto.sign_message(prepare.to_dict())

    logger.debug(
        "Prepare message created",
        agent=agent_id,
        valid=valid,
        confidence=confidence
    )

    return prepare

# Collect prepares from all online agents concurrently
online_agents = [a for a in self.agents.values() if a.is_online]

tasks = [get_prepare(agent.agent_id) for agent in online_agents]

try:
    results = await asyncio.wait_for(
        asyncio.gather(*tasks, return_exceptions=True),
        timeout=self.timeout_ms / 1000
    )

    for result in results:
        if isinstance(result, PrepareMessage):
```

```
                prepares.append(result)
            elif                                    isinstance(result,                                    Exception):
                logger.warning("Agent            failed            to            prepare",            error=str(result))

    except                                                                                                asyncio.TimeoutError:
        logger.warning("Prepare                                        phase                                        timeout")

    return                                                                                                        prepares

async                                                 def                                                 _collect_commits(
    self,
    digest:                                                                                                        str,
    sequence:                                                                                                       int
)                                                     ->                                                  List[CommitMessage]:
    """Collect                    commit                    messages                    from                    agents."""
    commits                                                       =                                                 []

    async             def             get_commit(agent_id:             str)             ->             CommitMessage:
        crypto                                    =                                    key_manager.get_provider(agent_id)

        commit                                                =                                         CommitMessage(
            message_type=MessageType.COMMIT,
            view=self.view,
            sequence=sequence,
            sender_id=agent_id,
            digest=digest,
        )

        commit.signature                                =                            crypto.sign_message(commit.to_dict())
        return                                                                                                    commit

    online_agents       =       [a       for       a       in       self.agents.values()       if       a.is_online]
    tasks          =          [get_commit(agent.agent_id)          for          agent          in          online_agents]

    try:
        results                               =                               await                               asyncio.wait_for(
            asyncio.gather(*tasks,                                                                  return_exceptions=True),
            timeout=self.timeout_ms                                          /                                          1000
        )

        for                                   result                                   in                                   results:
            if                                        isinstance(result,                                        CommitMessage):
                commits.append(result)

    except                                                                                                asyncio.TimeoutError:
```

```
        logger.warning("Commit phase timeout")

    return commits

async def initiate_view_change(self, reason: str = "timeout"):
    """
    Initiate a view change due to primary failure.

    In PBFT, if the primary is suspected faulty, replicas can
    vote to change to a new view with a different primary.
    """
    self.state = ConsensusState.VIEW_CHANGE
    old_view = self.view

    logger.warning(
        "Initiating view change",
        old_view=old_view,
        reason=reason
    )

    # Broadcast view change messages
    view_changes = []
    for agent in self.agents.values():
        if agent.is_online:
            crypto = key_manager.get_provider(agent.agent_id)
            vc = ViewChangeMessage(
                message_type=MessageType.VIEW_CHANGE,
                view=old_view,
                sequence=self.sequence,
                sender_id=agent.agent_id,
                new_view=old_view + 1,
                reason=reason,
            )
            vc.signature = crypto.sign_message(vc.to_dict())
            view_changes.append(vc)

    # Check if we have enough votes for view change
    if len(view_changes) >= self.quorum:
        self.view += 1
        self.state = ConsensusState.READY

        logger.info(
            "View change completed",
            old_view=old_view,
            new_view=self.view,
            new_primary=self.get_primary_id()
```

```
        )

    def get_stats(self) -> dict:
        """Get consensus statistics."""
        return {
            "total_rounds": self.total_rounds,
            "successful_rounds": self.successful_rounds,
            "failed_rounds": self.failed_rounds,
            "success_rate": self.successful_rounds / max(self.total_rounds, 1),
            "current_view": self.view,
            "current_sequence": self.sequence,
            "registered_agents": len(self.agents),
            "online_agents": sum(1 for a in self.agents.values() if a.is_online),
            "is_ready": self.is_ready(),
        }
```

G.2 Consensus Cryptography and Ed25519 Signing (app/consensus/crypto.py)

```
"""
Cryptographic utilities for PBFT consensus.

Provides message signing and verification to ensure Byzantine fault tolerance.
Each agent signs its messages so that faulty agents cannot impersonate others.

Author: Millicent Mufambi (H240624A)
"""

import hashlib
import hmac
import json
from typing import Optional
from datetime import datetime
from cryptography.hazmat.primitives import hashes, serialization
from cryptography.hazmat.primitives.asymmetric import rsa, padding
from cryptography.hazmat.backends import default_backend
from cryptography.exceptions import InvalidSignature
import base64



class CryptoProvider:
    """
    Handles cryptographic operations for PBFT.

    For simplicity, uses HMAC with shared secrets in development.
    Can be upgraded to asymmetric signatures (RSA/ECDSA) for production.
```

```
    """

    def __init__(self, agent_id: str, secret_key: Optional[str] = None):
        self.agent_id = agent_id
        self.secret_key = (secret_key or f"dev-secret-{agent_id}").encode()

        # For production: generate/load asymmetric keys
        self._private_key = None
        self._public_key = None

    def sign_message(self, message_dict: dict) -> str:
        """
        Sign a message and return the signature.

        Args:
            message_dict: Dictionary containing the message data

        Returns:
            Base64-encoded signature string
        """
        # Remove existing signature if present
        message_copy = {k: v for k, v in message_dict.items() if k != "signature"}

        # Canonical JSON representation
        canonical = json.dumps(message_copy, sort_keys=True, default=str)

        # HMAC signature
        signature = hmac.new(
            self.secret_key,
            canonical.encode(),
            hashlib.sha256
        ).digest()

        return base64.b64encode(signature).decode()

    def verify_signature(
        self,
        message_dict: dict,
        signature: str,
        sender_id: str
    ) -> bool:
        """
        Verify a message signature.

        Args:
```

```
            message_dict: The message data
            signature: The signature to verify
            sender_id: ID of the agent that signed the message

        Returns:
            True if signature is valid, False otherwise
        """
        try:
            # In production, use sender's public key
            # For development, reconstruct secret from sender_id
            sender_secret = f"dev-secret-{sender_id}".encode()

            message_copy = {k: v for k, v in message_dict.items() if k != "signature"}
            canonical = json.dumps(message_copy, sort_keys=True, default=str)

            expected = hmac.new(
                sender_secret,
                canonical.encode(),
                hashlib.sha256
            ).digest()

            provided = base64.b64decode(signature)

            return hmac.compare_digest(expected, provided)
        except Exception:
            return False

    def compute_digest(self, data: dict) -> str:
        """Compute SHA-256 digest of data."""
        canonical = json.dumps(data, sort_keys=True, default=str)
        return hashlib.sha256(canonical.encode()).hexdigest()


class KeyManager:
    """
    Manages cryptographic keys for all agents.

    In a production system, this would:
    - Generate and store agent key pairs
    - Distribute public keys
    - Handle key rotation
    """

    def __init__(self):
        self._keys: dict[str, CryptoProvider] = {}
```

```
    def get_provider(self, agent_id: str) -> CryptoProvider:
        """Get or create a crypto provider for an agent."""
        if agent_id not in self._keys:
            self._keys[agent_id] = CryptoProvider(agent_id)
        return self._keys[agent_id]

    def register_agent(self, agent_id: str, secret_key: Optional[str] = None):
        """Register an agent with its cryptographic credentials."""
        self._keys[agent_id] = CryptoProvider(agent_id, secret_key)


# Global key manager instance
key_manager = KeyManager()
```

G.3 Byzantine Fault-Injection Wrapper (app/agents/byzantine_wrapper.py)

```
"""
Byzantine wrapper — make an agent deliberately misbehave for fault-injection
experiments. Wraps any BaseAgent subclass and overrides its `process(...)`
method with one of several Byzantine behaviours.

Used by the benchmark harness via `--inject-fault {behaviour}` and
`--inject-fault-count N` to demonstrate the safety guarantees of the
H-BFT pipeline under up to f Byzantine validators (review §VI metric 5,
§VIII-E item 5).

Author: Millicent Mufambi (H240624A)
"""
from __future__ import annotations

import asyncio
import random
from datetime import datetime
from enum import Enum
from typing import Any

import structlog

from app.agents.base_agent import AgentOutput

logger = structlog.get_logger(__name__)
```

```
class ByzantineBehaviour(str, Enum):
    NONE = "none"
    ALWAYS_REJECT = "always_reject"          # Vote NO no matter what (denial-of-service)
    ALWAYS_APPROVE = "always_approve"        # Vote YES no matter what (most dangerous: lets bad fixes pass)
    RANDOM = "random"                        # Flip a coin
    TIMEOUT = "timeout"                      # Sleep past the round timeout (simulates crash)
    GARBAGE = "garbage"                      # Return a malformed result


class ByzantineWrapper:
    """
    Wraps an agent and makes it misbehave according to `behaviour`.

    The wrapper exposes the same surface as the wrapped agent (agent_id,
    agent_type, llm_provider, set_llm_client, update_reputation, accuracy,
    average_latency_ms, get_stats, process) so the rest of the system
    cannot tell it apart from a real agent.
    """

    def __init__(self, real_agent: Any, behaviour: ByzantineBehaviour):
        self.real = real_agent
        self.behaviour = ByzantineBehaviour(behaviour)
        # Mirror the surface of the real agent
        self.agent_id = real_agent.agent_id
        self.agent_type = real_agent.agent_type
        self.llm_provider = real_agent.llm_provider
        self.config = real_agent.config

        logger.warning(
            "Byzantine wrapper installed",
            agent_id=self.agent_id,
            behaviour=self.behaviour.value,
        )

    # --- pass-throughs for things the rest of the system uses --------------
    @property
    def reputation_score(self):
        return self.real.reputation_score

    @reputation_score.setter
    def reputation_score(self, v):
        self.real.reputation_score = v

    @property
```

```
def total_tasks(self):
    return self.real.total_tasks

@property
def successful_tasks(self):
    return self.real.successful_tasks

@property
def accuracy(self):
    return self.real.accuracy

@property
def average_latency_ms(self):
    return self.real.average_latency_ms

def set_llm_client(self, client):
    return self.real.set_llm_client(client)

def update_reputation(self, success):
    return self.real.update_reputation(success)

def get_stats(self):
    s = self.real.get_stats()
    s["byzantine_behaviour"] = self.behaviour.value
    return s

# --- the misbehaving entry point ----------------------------------
async def process(self, input_data: Any) -> AgentOutput:
    start = datetime.utcnow()

    if self.behaviour == ByzantineBehaviour.TIMEOUT:
        try:
            await asyncio.sleep(120)
        except asyncio.CancelledError:
            pass
        return self._fake_output(False, 0.0, start, "byzantine timeout")

    if self.behaviour == ByzantineBehaviour.GARBAGE:
        return self._fake_output(False, 0.0, start, "garbage output")

    if self.behaviour == ByzantineBehaviour.ALWAYS_REJECT:
        return self._fake_output(False, 0.95, start, "byzantine always-reject")

    if self.behaviour == ByzantineBehaviour.ALWAYS_APPROVE:
```

```
            return self._fake_output(True, 0.95, start, "byzantine always-approve")

        if self.behaviour == ByzantineBehaviour.RANDOM:
            decision = random.choice([True, False])
            return self._fake_output(decision, 0.5, start, "byzantine random")

        # ByzantineBehaviour.NONE — just defer to the real agent
        return await self.real.process(input_data)

    def _fake_output(self, is_valid: bool, confidence: float,
                     start: datetime, reason: str) -> AgentOutput:
        from app.agents.validator_agent import ValidationResult
        latency = (datetime.utcnow() - start).total_seconds() * 1000
        return AgentOutput(
            agent_id=self.agent_id,
            agent_type=self.agent_type,
            success=True,
            result=ValidationResult(
                is_valid=is_valid,
                confidence=confidence,
                syntax_valid=True,
                safety_score=0.5,
                issues=[reason] if not is_valid else [],
                recommendations=[],
            ),
            confidence=confidence,
            reasoning=reason,
            latency_ms=latency,
            metadata={"byzantine": True, "behaviour": self.behaviour.value},
        )
```

G.4 Base Agent Abstraction (app/agents/base_agent.py)

```
"""
Base Agent Abstract Class.

Defines the interface that all agents must implement.
Supports multiple LLM backends for heterogeneous BFT.

Author: Millicent Mufambi (H240624A)
"""

from abc import ABC, abstractmethod
from dataclasses import dataclass, field
from datetime import datetime
```

```
from typing import Optional, Any, Dict
from enum import Enum
import structlog

logger = structlog.get_logger(__name__)


class LLMProvider(str, Enum):
    """Supported LLM providers."""
    CLAUDE = "claude"
    GPT4 = "gpt4"
    OLLAMA = "ollama"


class AgentType(str, Enum):
    """Types of agents in the system."""
    ANALYZER = "analyzer"
    HEALER = "healer"
    VALIDATOR = "validator"


@dataclass
class AgentConfig:
    """Configuration for an agent."""
    agent_id: str
    agent_type: AgentType
    llm_provider: LLMProvider
    model_name: str = ""  # e.g., "claude-3-opus", "gpt-4", "llama3"
    temperature: float = 0.7
    max_tokens: int = 2048
    timeout_seconds: int = 30
    # Optional decoding seed for reproducibility. Honoured by providers that
    # support it (OpenAI `seed`, Ollama `options.seed`). Anthropic exposes no
    # seed parameter, so it is ignored for Claude.
    seed: Optional[int] = None


@dataclass
class AgentOutput:
    """Standard output format for all agents."""
    agent_id: str
    agent_type: AgentType
    success: bool
    result: Any
    confidence: float = 0.0
```

```
    reasoning: str = ""
    latency_ms: float = 0.0
    timestamp: datetime = field(default_factory=datetime.utcnow)
    metadata: Dict[str, Any] = field(default_factory=dict)


class BaseAgent(ABC):
    """
    Abstract base class for all agents.

    All agents share:
    - An LLM client for generating responses
    - A reputation score that affects consensus weight
    - Standard input/output interfaces
    - Logging and metrics

    Subclasses implement:
    - process(): The main agent logic
    - _build_prompt(): Create the LLM prompt for the specific task
    """

    def __init__(self, config: AgentConfig):
        self.config = config
        self.agent_id = config.agent_id
        self.agent_type = config.agent_type
        self.llm_provider = config.llm_provider

        # Reputation tracking (affects consensus weight)
        self.reputation_score = 1.0
        self.total_tasks = 0
        self.successful_tasks = 0

        # LLM client (initialized in subclass or via set_llm_client)
        self._llm_client = None

        # Metrics
        self._total_latency_ms = 0.0

        logger.info(
            "Agent initialized",
            agent_id=self.agent_id,
            agent_type=self.agent_type.value,
            llm_provider=self.llm_provider.value
        )
```

```
def set_llm_client(self, client: Any):
    """Set the LLM client for this agent."""
    self._llm_client = client

@abstractmethod
async def process(self, input_data: Any) -> AgentOutput:
    """
    Process input and produce output.

    This is the main entry point for agent operations.
    Must be implemented by subclasses.

    Args:
        input_data: Task-specific input data

    Returns:
        AgentOutput with the results
    """
    pass

@abstractmethod
def _build_prompt(self, input_data: Any) -> str:
    """
    Build the prompt for the LLM.

    Must be implemented by subclasses to create task-specific prompts.

    Args:
        input_data: Task-specific input

    Returns:
        Formatted prompt string
    """
    pass

async def _call_llm(self, prompt: str) -> str:
    """
    Call the configured LLM and return the response.

    Handles different providers transparently. Ollama uses an HTTP
    endpoint and does not need a Python client object, so it is
    exempt from the client-presence check.
    """
    if self.llm_provider != LLMProvider.OLLAMA and self._llm_client is None:
        raise RuntimeError(f"LLM client not configured for agent {self.agent_id}")
```

```
    start_time = datetime.utcnow()

    try:
        if self.llm_provider == LLMProvider.CLAUDE:
            response = await self._call_claude(prompt)
        elif self.llm_provider == LLMProvider.GPT4:
            response = await self._call_gpt4(prompt)
        elif self.llm_provider == LLMProvider.OLLAMA:
            response = await self._call_ollama(prompt)
        else:
            raise ValueError(f"Unknown LLM provider: {self.llm_provider}")

        latency = (datetime.utcnow() - start_time).total_seconds() * 1000
        self._total_latency_ms += latency

        logger.debug(
            "LLM call completed",
            agent_id=self.agent_id,
            provider=self.llm_provider.value,
            latency_ms=latency
        )

        return response

    except Exception as e:
        logger.error(
            "LLM call failed",
            agent_id=self.agent_id,
            error=str(e)
        )
        raise

async def _call_claude(self, prompt: str) -> str:
    """Call Claude API."""
    # Using anthropic library
    response = await self._llm_client.messages.create(
        model=self.config.model_name or "claude-3-sonnet-20240229",
        max_tokens=self.config.max_tokens,
        temperature=self.config.temperature,
        messages=[{"role": "user", "content": prompt}]
    )
    return response.content[0].text

async def _call_gpt4(self, prompt: str) -> str:
```

```
    """Call OpenAI GPT-4 API."""
    # Using openai library
    extra = {}
    if self.config.seed is not None:
        extra["seed"] = self.config.seed
    response = await self._llm_client.chat.completions.create(
        model=self.config.model_name or "gpt-4",
        max_tokens=self.config.max_tokens,
        temperature=self.config.temperature,
        messages=[{"role": "user", "content": prompt}],
        **extra,
    )
    return response.choices[0].message.content


async def _call_ollama(self, prompt: str) -> str:
    """Call local Ollama instance."""
    import aiohttp
    from app.config import settings

    # Allow model_name to be either:
    #   * "url|model" (preferred — fully explicit)
    #   * a bare URL (legacy — uses settings.ollama_model)
    #   * a model name (uses settings.ollama_base_url)
    #   * empty (uses both settings)
    model_name = self.config.model_name or ""
    if "|" in model_name:
        base_url, ollama_model = model_name.split("|", 1)
    elif model_name.startswith("http"):
        base_url = model_name
        ollama_model = settings.ollama_model
    else:
        base_url = settings.ollama_base_url
        ollama_model = model_name or settings.ollama_model

    options = {
        "temperature": self.config.temperature,
        "num_predict": self.config.max_tokens,
        "num_ctx": 8192,
    }
    if self.config.seed is not None:
        options["seed"] = self.config.seed
    # Explicit timeout: CPU-only Ollama generation can exceed aiohttp's
    # 300 s default, so honour the agent's configured timeout instead.
    timeout = aiohttp.ClientTimeout(total=max(self.config.timeout_seconds, 60))
    async with aiohttp.ClientSession(timeout=timeout) as session:
        async with session.post(
            f"{base_url}/api/generate",
            json={
```

```
                "model":                                                                    ollama_model,
                "prompt":                                                                         prompt,
                "stream":                                                                          False,
                "options":                                                                        options,
            }
        )                                                   as                                      resp:
            result                              =                           await                  resp.json()
            return                                      result.get("response",                              "")


def                          update_reputation(self,                          success:                          bool):
    """
    Update             agent             reputation             based             on             task             outcome.

    Reputation               affects               weight               in               consensus               voting.
    """
    self.total_tasks                                                 +=                                             1
    if                                                                                                      success:
        self.successful_tasks                                                +=                                      1
        #               Increase               reputation                (bounded               at               1.0)
        self.reputation_score           =           min(1.0,           self.reputation_score           +           0.01)
    else:
        #               Decrease               reputation                (bounded               at               0.1)
        self.reputation_score           =           max(0.1,           self.reputation_score           -           0.05)


@property
def                                accuracy(self)                                ->                                float:
    """Calculate                             agent                             accuracy                            rate."""
    if                                self.total_tasks                                ==                                0:
        return                                                                                                      1.0
    return                          self.successful_tasks                        /                        self.total_tasks


@property
def                             average_latency_ms(self)                             ->                             float:
    """Calculate                     average                     latency                     per                     task."""
    if                                self.total_tasks                                ==                                0:
        return                                                                                                      0.0
    return                         self._total_latency_ms                        /                        self.total_tasks


def                                get_stats(self)                                ->                                dict:
    """Get                                          agent                                           statistics."""
    return                                                                                                          {
        "agent_id":                                                                                  self.agent_id,
        "agent_type":                                                                          self.agent_type.value,
        "llm_provider":                                                                      self.llm_provider.value,
        "reputation_score":                                                                    self.reputation_score,
        "total_tasks":                                                                              self.total_tasks,
        "successful_tasks":                                                                    self.successful_tasks,
```

```
            "accuracy":                                                    self.accuracy,
            "average_latency_ms":                                    self.average_latency_ms,
        }
```

G.5 Validator Agent (app/agents/validator_agent.py)

```
"""
Validator                    Agent                    -                    Fix                    Verification.

This                    agent                    is                    responsible                    for:
1.                Validating                proposed                fixes                for                correctness
2.                        Checking                        for                        safety                        issues
3.                                Running                                static                                analysis
4.                                Simulating                                fix                                execution

Uses            Ollama            (local            LLM)            for            fast,            free            validation.

Author:                            Millicent                            Mufambi                            (H240624A)
"""

from                            dataclasses                            import                            dataclass
from                    typing                    import                    Optional,                    List
from                            datetime                            import                            datetime
import                                                                                                    json
import                                                                                                    re
import                                                                                                    ast

from     app.agents.base_agent     import     BaseAgent,     AgentConfig,     AgentOutput,     AgentType,     LLMProvider
from                        app.agents.healer_agent                        import                        FixCandidate
import                                                                                                    structlog

logger                                    =                                    structlog.get_logger(__name__)


@dataclass
class                                                                                                ValidationInput:
    """Input                            for                            fix                            validation."""
    original_code:                                                                                            str
    fix_candidate:                                                                                        FixCandidate
    language:                                                                                                 str
    file_path:                                                                                                str
    test_cases:                            Optional[List[str]]                        =                        None
```

```
@dataclass
class ValidationResult:
    """Result of fix validation."""
    is_valid: bool
    confidence: float
    syntax_valid: bool
    safety_score: float
    issues: List[str]
    recommendations: List[str]
    test_results: Optional[dict] = None


class ValidatorAgent(BaseAgent):
    """
    Validator Agent for verifying proposed fixes.

    This agent validates fixes before they are applied:
    - Syntax checking
    - Static analysis
    - Safety verification
    - (Optional) Test execution

    Uses Ollama for fast, local validation without API costs.
    """

    # Dangerous patterns to check for
    DANGEROUS_PATTERNS = [
        (r'\beval\s*\(', "Use of eval() is dangerous"),
        (r'\bexec\s*\(', "Use of exec() is dangerous"),
        (r'__import__\s*\(', "Dynamic imports are risky"),
        (r'\bos\.system\s*\(', "Shell command execution"),
        (r'\bsubprocess\.(call|run|Popen)', "Subprocess execution"),
        (r'open\s*\([^)]*["\']w', "File write operation"),
        (r'\brm\s+-rf', "Destructive file operation"),
    ]

    def __init__(self, config: Optional[AgentConfig] = None):
        if config is None:
            config = AgentConfig(
                agent_id="validator-ollama-01",
                agent_type=AgentType.VALIDATOR,
                llm_provider=LLMProvider.OLLAMA,
                model_name="http://localhost:11434",
                temperature=0.1,  # Low temperature for consistent validation
                max_tokens=1024,
```

```
        )
    super().__init__(config)

async def process(self, input_data: ValidationInput) -> AgentOutput:
    """
    Validate a proposed fix.

    Args:
        input_data: Fix candidate and context

    Returns:
        AgentOutput containing ValidationResult
    """
    start_time = datetime.utcnow()

    try:
        # Step 1: Static checks (no LLM needed)
        static_result = self._static_validation(input_data)

        # Step 2: LLM-based semantic validation
        if static_result.syntax_valid:
            prompt = self._build_prompt(input_data)
            llm_response = await self._call_llm(prompt)
            semantic_result = self._parse_response(llm_response)

            # Combine results
            result = ValidationResult(
                is_valid=static_result.is_valid and semantic_result.is_valid,
                confidence=(static_result.confidence + semantic_result.confidence) / 2,
                syntax_valid=static_result.syntax_valid,
                safety_score=static_result.safety_score,
                issues=static_result.issues + semantic_result.issues,
                recommendations=semantic_result.recommendations,
            )
        else:
            result = static_result

        latency = (datetime.utcnow() - start_time).total_seconds() * 1000

        self.update_reputation(True)

        logger.info(
            "Fix validated",
            agent_id=self.agent_id,
            is_valid=result.is_valid,
```

```
            confidence=result.confidence,
            issues=len(result.issues),
            latency_ms=latency
        )

        return AgentOutput(
            agent_id=self.agent_id,
            agent_type=self.agent_type,
            success=True,
            result=result,
            confidence=result.confidence,
            reasoning=";  ".join(result.issues)  if  result.issues  else  "Fix  looks  valid",
            latency_ms=latency,
        )

    except Exception as e:
        latency = (datetime.utcnow() - start_time).total_seconds() * 1000
        self.update_reputation(False)

        logger.error("Validation failed", error=str(e))

        return AgentOutput(
            agent_id=self.agent_id,
            agent_type=self.agent_type,
            success=False,
            result=ValidationResult(
                is_valid=False,
                confidence=0.0,
                syntax_valid=False,
                safety_score=0.0,
                issues=[f"Validation error: {str(e)}"],
                recommendations=["Manual review required"]
            ),
            confidence=0.0,
            reasoning=f"Validation failed: {str(e)}",
            latency_ms=latency,
        )

@staticmethod
def _python_syntax_ok(code: str) -> tuple[bool, Optional[str]]:
    """Decide  whether  a  Python  snippet  should  count  as  syntactically  valid.

    Real-world  bug  snippets  (e.g.  BugsInPy  diff  hunks)  are  sliced  out  of  a
    larger  function,  so  they  start  mid-block  and  `ast.parse`  raises  an
    IndentationError  even  though  the  code  itself  is  fine.  An  indentation-
    only  failure  is  a  *framing*  artifact,  NOT  a  defect,  so  we  try  to  make
```

```
    the fragment parseable (dedent, then wrap as a function body); if only
    indentation is the problem we treat it as valid and defer the real
    judgement to the LLM validator and the sandbox (which executes the
    reconstructed code). A genuine SyntaxError is still a hard reject.

    Returns (syntax_ok, note_or_error).
    """
    import textwrap

    try:
        ast.parse(code)
        return True, None
    except IndentationError:
        pass  # fragment-framing; fall through to tolerant attempts
    except SyntaxError as e:
        return False, f"Syntax error: {e.msg} at line {e.lineno}"

    dedented = textwrap.dedent(code)
    for candidate in (dedented,
                      "def __frag__():\n" + textwrap.indent(dedented, "    ")):
        try:
            ast.parse(candidate)
            return True, None
        except SyntaxError:
            continue
    # Still only an indentation issue -> non-standalone fragment, not a
    # defect. Accept structurally; LLM + sandbox remain the real gates.
    return True, "non-standalone snippet (indentation only); deferred to semantic + sandbox checks"

def _static_validation(self, input_data: ValidationInput) -> ValidationResult:
    """Perform static validation without LLM."""
    issues = []
    safety_score = 1.0

    fixed_code = input_data.fix_candidate.fixed_code

    # Check syntax (for Python)
    syntax_valid = True
    if input_data.language.lower() == "python":
        syntax_valid, syntax_note = self._python_syntax_ok(fixed_code)
        if syntax_note:
            issues.append(syntax_note)

    # Check for dangerous patterns
    for pattern, description in self.DANGEROUS_PATTERNS:
        if re.search(pattern, fixed_code):
```

```
            issues.append(f"Safety issue: {description}")
            safety_score -= 0.2

        # Check if fix is actually different from original
        if fixed_code.strip() == input_data.original_code.strip():
            issues.append("Fix is identical to original code")

        # Clamp safety score
        safety_score = max(0.0, safety_score)

        is_valid = syntax_valid and safety_score >= 0.5 and \
                   fixed_code.strip() != input_data.original_code.strip()

        return ValidationResult(
            is_valid=is_valid,
            confidence=0.7 if is_valid else 0.3,
            syntax_valid=syntax_valid,
            safety_score=safety_score,
            issues=issues,
            recommendations=[],
        )

    def _build_prompt(self, input_data: ValidationInput) -> str:
        """Build the validation prompt for the LLM.

        Prompt design (Phase 5 tuning):
        - Acceptance-biased: vote IS_VALID=true unless a specific, concrete
          safety problem can be named. Style preferences and "could be
          better" critiques are NOT grounds for rejection.
        - The sandbox stage runs the fix end-to-end, so the validator's
          job is structural safety review, not full correctness proof.
        - Issues must be specific (line/condition); vague concerns are
          recorded as recommendations, not blockers.
        """
        return f"""You are reviewing a candidate bug fix as part of a Byzantine consensus stage. The fix has already been generated by a Healer agent and will be executed in an isolated sandbox after this review. Your job is structural safety review, NOT exhaustive correctness proof.

## Original Code (buggy)
```{input_data.language}
{input_data.original_code}
```

Proposed Fix
```{input_data.language}
```
```

```
{input_data.fix_candidate.fixed_code}
```

```
## Fix Explanation
{input_data.fix_candidate.explanation}

## Voting Rules
- Vote IS_VALID = true if the fix plausibly addresses the bug AND introduces no obvious safety problem.
- Vote IS_VALID = false ONLY if you can name a SPECIFIC safety problem (concrete line, condition, or scenario where the fix would crash, corrupt data, or violate an invariant).
- Style preferences, naming choices, missing comments, suboptimal performance, or "could be cleaner" critiques are NOT grounds for rejection. Record them under recommendations.
- If the fix is plausibly correct but you are uncertain, vote IS_VALID = true with moderate confidence (0.5-0.7). The sandbox stage will catch true failures.

Respond with JSON only:
{{
    "is_valid": <true|false>,
    "confidence": <0.0-1.0>,
    "issues": ["<specific safety problem 1>", "<specific safety problem 2>"],
    "recommendations": ["<style/improvement suggestion 1>"]
}}

The "issues" list should be EMPTY if the fix is plausibly safe. Only populate it with concrete, specific problems."""

    def _parse_response(self, response: str) -> ValidationResult:
        """Parse LLM validation response."""
        json_match = re.search(r'\{[^{}]*\}', response, re.DOTALL)
        if json_match:
            response = json_match.group()

        try:
            data = json.loads(response)
            return ValidationResult(
                is_valid=data.get("is_valid", False),
                confidence=float(data.get("confidence", 0.5)),
                syntax_valid=True,  # Already checked in static
                safety_score=1.0,  # Already checked in static
                issues=data.get("issues", []),
                recommendations=data.get("recommendations", []),
            )
        except json.JSONDecodeError:
            return ValidationResult(
                is_valid=True,
                confidence=0.5,
                syntax_valid=True,
```

```
                safety_score=1.0,
                issues=[],
                recommendations=["Validator JSON response was not parseable; deferred to sandbox"],
            )
```

G.6 Repair Pipeline, Consensus Mode (app/repair/repair_pipeline.py)

```
"""
End-to-end repair pipeline.

Stages:
  1. Multi-agent + PBFT consensus (delegated to AgentManager.process_bug)
  2. Sandbox validation of the consensus-approved fix
  3. Optional disk apply with rollback on regression-test failure
  4. Metrics emission
  5. Knowledge-graph recording

The pipeline is the single entry point used by both the FastAPI route and
the offline benchmark harness, so the same code path is exercised in
production and in evaluation runs.

Author: Millicent Mufambi (H240624A)
"""

from __future__ import annotations

import asyncio
from dataclasses import dataclass, field
from datetime import datetime
from enum import Enum
from typing import Any, Dict, List, Optional

import structlog

from app.agents.agent_manager import AgentManager
from app.repair.patch_applier import PatchApplier, PatchApplyStatus
from app.sandbox.docker_executor import SandboxExecutor, SandboxResult

logger = structlog.get_logger(__name__)


class RepairStage(str, Enum):
    AGENT_PIPELINE = "agent_pipeline"
```

```
    CONSENSUS                                                =                                "consensus"
    SANDBOX_VALIDATION                                       =                             "sandbox_validation"
    PATCH_APPLY                                              =                                "patch_apply"
    REGRESSION_CHECK                                         =                             "regression_check"
    KNOWLEDGE_RECORD                                         =                             "knowledge_record"
    DONE                                                     =                                "done"



@dataclass
class                                                                                      RepairOutcome:
    """End-to-end                result                returned                by                RepairPipeline.run."""
    bug_id:                                                                                          str
    success:                                                                                        bool
    final_stage:                                                                                RepairStage
    decision_reason:                                  str                                =                              ""
    duration_ms:                                    float                                =                             0.0
    consensus_result:                   Optional[Dict[str,                   Any]]                   =                 None
    sandbox_result:                     Optional[Dict[str,                   Any]]                   =                 None
    apply_result:                       Optional[Dict[str,                   Any]]                   =                 None
    safety_violation:                                  bool                                =                            False
    fixed_code:                                Optional[str]                                =                             None
    explanation:                                Optional[str]                                =                             None
    #      Per-validator      votes      from      this      run,      used      by      the      metrics      layer      for
    #          failure-decorrelation          analysis          (Gap          5          from          review          §VIII-E).
    validation_results:              List[Dict[str,             Any]]             =             field(default_factory=list)
    timestamp:                      datetime                     =                  field(default_factory=datetime.utcnow)
    extras:                  Dict[str,                  Any]                =                field(default_factory=dict)



class                                                                                    RepairPipeline:
    """
    Orchestrates       a       single       repair       attempt       from       bug       report       to       either
    "fix         applied"         or         "fix         rejected"         with         full         metric         capture.

    Designed      to      be      reused      for      live      runtime      repair      AND      for      batch      evaluation
    on                      Defects4J                      /                      BugsInPy                      benchmarks.
    """

    def                                                                                              __init__(
        self,
        agent_manager:                                                                          AgentManager,
        sandbox:                            Optional[SandboxExecutor]                          =                        None,
        applier:                             Optional[PatchApplier]                           =                        None,
        metrics:          Optional["MetricsCollector"]          =          None,                #          forward          ref
        knowledge:          Optional["KnowledgeGraph"]          =          None,                #          forward          ref
        apply_to_disk:                                  bool                                =                            False,
```

```
    sandbox_timeout_s:                              int                             =                           30,
    max_attempts:                                 int                               =                            3,
):
    self.agent_manager                                          =                                       agent_manager
    self.sandbox          =          sandbox          or          SandboxExecutor(default_timeout_s=sandbox_timeout_s)
    self.applier                =                applier                or                PatchApplier(dry_run=True)
    self.metrics                                                =                                           metrics
    self.knowledge                                              =                                         knowledge
    self.apply_to_disk                                          =                                      apply_to_disk
    self.sandbox_timeout_s                                      =                                   sandbox_timeout_s
    self.max_attempts                         =                       max(1,                        int(max_attempts))


    logger.info(
        "Repair                                        pipeline                                        initialised",
        apply_to_disk=apply_to_disk,
        sandbox_timeout_s=sandbox_timeout_s,
        max_attempts=self.max_attempts,
    )


#                                                  ----------------------------------------------------------------
async                                                   def                                                        run(
    self,
    error_message:                                                                                              str,
    stack_trace:                                                                                                str,
    code_context:                                                                                               str,
    file_path:                                                                                                  str,
    line_number:                                                                                                int,
    language:                                 str                                 =                          "python",
    test_command:                           Optional[List[str]]                           =                        None,
    regression_command:                         Optional[List[str]]                         =                        None,
)                                                       ->                                                RepairOutcome:
    """
    Run       a       single       end-to-end       repair       attempt       with       retry-on-failure       feedback.


    The     agent     pipeline     (analyzer     +     healer     +     validators     +     consensus)     and     the
    sandbox     stage     are     wrapped     in     a     self-debug     loop     of     up     to     `max_attempts`
    iterations.         After         a         sandbox-         or         consensus-rejection,         the         failure
    signal     is     appended     to     the     bug     context     so     the     next     attempt's     Healer
    sees         the         prior         failure,         modelled         on         Reflexion-style         self-debug.
    """
    started                                                 =                                             datetime.utcnow()
    outcome        =        RepairOutcome(bug_id="",        success=False,        final_stage=RepairStage.AGENT_PIPELINE)


    last_agent_result:                  Optional[Dict[str,                  Any]]                  =                  None
    last_sandbox_result:                          Optional[SandboxResult]                           =                  None
    last_failure_reason:                            Optional[str]                           =                        None
    augmented_context                                           =                                       code_context
```

```
for attempt in range(1, self.max_attempts + 1):
    outcome.extras["attempt"] = attempt
    outcome.extras["max_attempts"] = self.max_attempts

    # ----- Stage 1: agents + consensus ----- #
    agent_result = await self.agent_manager.process_bug(
        error_message=error_message,
        stack_trace=stack_trace,
        code_context=augmented_context,
        file_path=file_path,
        line_number=line_number,
        language=language,
    )
    last_agent_result = agent_result

    outcome.bug_id = agent_result.get("bug_id", outcome.bug_id) or ""
    outcome.consensus_result = agent_result.get("consensus")
    outcome.validation_results = list(agent_result.get("validation_results") or [])

    if not agent_result.get("success"):
        last_failure_reason = (
            agent_result.get("reason")
            or "agent pipeline did not produce a consensus"
        )
        outcome.final_stage = RepairStage.AGENT_PIPELINE
        outcome.decision_reason = (
            f"attempt {attempt}/{self.max_attempts}: {last_failure_reason}"
        )
        if attempt < self.max_attempts:
            augmented_context = self._build_retry_context(
                code_context, attempt, "consensus_rejected", last_failure_reason
            )
            continue
        outcome.duration_ms = (datetime.utcnow() - started).total_seconds() * 1000
        self._emit_metrics(outcome, agent_result)
        return outcome

    outcome.fixed_code = agent_result["fix"]["fixed_code"]
    outcome.explanation = agent_result["fix"]["explanation"]
    outcome.final_stage = RepairStage.CONSENSUS

    # ----- Stage 2: sandbox validation ----- #
    sandbox_result = await self.sandbox.run(
        code=outcome.fixed_code,
        language=language,
```

```
        test_command=test_command,
        timeout_s=self.sandbox_timeout_s,
    )
    last_sandbox_result = sandbox_result
    outcome.sandbox_result = self._sandbox_to_dict(sandbox_result)
    outcome.final_stage = RepairStage.SANDBOX_VALIDATION

    if sandbox_result.success:
        # success - exit the retry loop and continue to stage 3+
        outcome.safety_violation = False
        break

    # sandbox failed - prepare retry context if attempts remain
    last_failure_reason = sandbox_result.short_reason()
    stderr_excerpt = (sandbox_result.stderr or "")[:1500]
    if attempt < self.max_attempts:
        augmented_context = self._build_retry_context(
            code_context, attempt, "sandbox_failed", stderr_excerpt or last_failure_reason
        )
        continue

    # all attempts exhausted - record safety violation
    outcome.success = False
    outcome.safety_violation = True
    outcome.decision_reason = (
        f"attempt {attempt}/{self.max_attempts}: sandbox rejected: {last_failure_reason}"
    )
    outcome.duration_ms = (datetime.utcnow() - started).total_seconds() * 1000
    self._emit_metrics(outcome, agent_result)
    self._record_knowledge(outcome)
    return outcome

# Reached here only if sandbox passed on some attempt
# (otherwise we returned inside the loop)
agent_result = last_agent_result or {}

# ----- Stage 3: optional apply to disk ----- #
if self.apply_to_disk:
    apply_result = self.applier.apply(file_path=file_path, new_content=outcome.fixed_code)
    outcome.apply_result = {
        "status": apply_result.status.value,
        "backup_id": apply_result.backup_id,
        "error": apply_result.error,
    }
    outcome.final_stage = RepairStage.PATCH_APPLY
```

```
        if apply_result.status not in (PatchApplyStatus.APPLIED, PatchApplyStatus.DRY_RUN):
            outcome.success = False
            outcome.decision_reason = f"patch apply failed: {apply_result.error}"
            outcome.duration_ms = (datetime.utcnow() - started).total_seconds() * 1000
            self._emit_metrics(outcome, agent_result)
            return outcome

        # ----- Stage 4: post-apply regression check -----
        if regression_command:
            outcome.final_stage = RepairStage.REGRESSION_CHECK
            regression = await self.sandbox.run(
                code=outcome.fixed_code,
                language=language,
                test_command=regression_command,
                timeout_s=self.sandbox_timeout_s * 2,
            )
            if not regression.success and apply_result.backup_id:
                rollback_result = self.applier.rollback(apply_result.backup_id)
                outcome.success = False
                outcome.safety_violation = True
                outcome.decision_reason = (
                    f"regression failed; rolled back ({rollback_result.status.value})"
                )
                outcome.duration_ms = (datetime.utcnow() - started).total_seconds() * 1000
                self._emit_metrics(outcome, agent_result)
                self._record_knowledge(outcome)
                return outcome

        # ----- Stage 5: success -----
        outcome.success = True
        outcome.final_stage = RepairStage.DONE
        outcome.decision_reason = "fix approved, sandbox-validated, applied" if self.apply_to_disk else "fix approved and
sandbox-validated"
        outcome.duration_ms = (datetime.utcnow() - started).total_seconds() * 1000
        self._emit_metrics(outcome, agent_result)
        self._record_knowledge(outcome)
        return outcome

    # ------------------------------------------------------------------
    def _emit_metrics(self, outcome: RepairOutcome, raw_agent_result: Dict[str, Any]) -> None:
        if not self.metrics:
            return
        try:
            self.metrics.record_repair_outcome(outcome=outcome, raw_agent_result=raw_agent_result)
        except Exception as e:
            logger.warning("Metric emission failed (non-fatal)", error=str(e))
```

```
def _record_knowledge(self, outcome: RepairOutcome) -> None:
    if not self.knowledge:
        return
    try:
        self.knowledge.record_repair(outcome)
    except Exception as e:
        logger.warning("Knowledge-graph recording failed (non-fatal)", error=str(e))


@staticmethod
def _build_retry_context(
    original_context: str,
    prior_attempt: int,
    failure_kind: str,
    failure_signal: str,
) -> str:
    """Augment the bug context with a structured note about why
    the previous attempt failed, so the next Healer call can adapt."""
    kind_note = {
        "consensus_rejected": "Validators rejected the previous fix candidate (no quorum reached).",
        "sandbox_failed": "The previous fix passed consensus but failed sandbox execution.",
    }.get(failure_kind, "Previous attempt failed.")
    return (
        f"{original_context}\n\n"
        f"# === SELF-DEBUG FEEDBACK FROM ATTEMPT {prior_attempt} ===\n"
        f"# {kind_note}\n"
        f"# Failure signal:\n"
        f"# {failure_signal[:1500]}\n"
        f"# Generate a different fix that addresses this specific failure mode."
    )


@staticmethod
def _sandbox_to_dict(r: SandboxResult) -> Dict[str, Any]:
    return {
        "success": r.success,
        "backend": r.backend.value,
        "exit_code": r.exit_code,
        "duration_ms": r.duration_ms,
        "timed_out": r.timed_out,
        "tests_passed": r.tests_passed,
        "tests_failed": r.tests_failed,
        "error": r.error,
        "stdout_tail": r.stdout[-1000:] if r.stdout else "",
        "stderr_tail": r.stderr[-1000:] if r.stderr else "",
    }
```

G.7 Single-Agent Baseline Pipeline (app/repair/single_agent_pipeline.py)

```
"""
Single-agent baseline pipeline.

This is the comparison point for the H-BFT consensus pipeline. It runs the analyzer + healer (so the bug actually gets understood and a fix is generated), then SKIPS validators and SKIPS consensus, applying the healer's recommended fix directly. The sandbox check is still performed so we can measure how often a single-agent system would be wrong.

The SingleAgentPipeline returns a RepairOutcome with the same shape as RepairPipeline so the benchmark runner does not need to know which one it is talking to.

Author: Millicent Mufambi (H240624A)
"""
from __future__ import annotations

from datetime import datetime
from typing import Any, Dict, List, Optional

import structlog

from app.agents.agent_manager import AgentManager
from app.agents.analyzer_agent import BugAnalysisInput
from app.agents.healer_agent import FixGenerationInput
from app.repair.repair_pipeline import RepairOutcome, RepairStage
from app.sandbox.docker_executor import SandboxExecutor, SandboxResult

logger = structlog.get_logger(__name__)


class SingleAgentPipeline:
    """
    Baseline pipeline: Analyzer → Healer → Sandbox.

    No validators, no consensus. The healer's recommended fix is the
    decision the system would commit to. Used to measure how often a
    single-agent system produces a confident-but-wrong fix.
    """

    def __init__(
        self,
```

```
    agent_manager:                                                                  AgentManager,
    sandbox:                          Optional[SandboxExecutor]                       =                       None,
    sandbox_timeout_s:                                int                             =                        30,
    max_attempts:                                    int                             =                         3,
):
    self.agent_manager                                          =                                       agent_manager
    self.sandbox          =          sandbox          or          SandboxExecutor(default_timeout_s=sandbox_timeout_s)
    self.sandbox_timeout_s                                      =                                    sandbox_timeout_s
    self.max_attempts                          =                        max(1,                        int(max_attempts))


async                                                  def                                                          run(
    self,
    error_message:                                                                                               str,
    stack_trace:                                                                                                 str,
    code_context:                                                                                                str,
    file_path:                                                                                                   str,
    line_number:                                                                                                 int,
    language:                                  str                                  =                           "python",
    test_command:                             Optional[List[str]]                           =                      None,
    regression_command:     Optional[List[str]]     =     None,          #     accepted     for     parity,     unused
)                                                     ->                                                  RepairOutcome:
    started                                                 =                                          datetime.utcnow()
    outcome                                                   =                                        RepairOutcome(
        bug_id=f"single-{int(started.timestamp())}",
        success=False,
        final_stage=RepairStage.AGENT_PIPELINE,
    )


    augmented_context                                           =                                          code_context


    for            attempt            in            range(1,            self.max_attempts            +            1):
        outcome.extras["attempt"]                                       =                                       attempt
        outcome.extras["max_attempts"]                                  =                                  self.max_attempts


        #      Stage      1      —      Analyzer      (still      useful      for      the      Healer's      prompt)
        try:
            analysis_input                                      =                                      BugAnalysisInput(
                error_message=error_message,
                stack_trace=stack_trace,
                code_context=augmented_context,
                file_path=file_path,
                line_number=line_number,
                language=language,
            )
            analysis_output           =           await           self.agent_manager.analyzer.process(analysis_input)
            if                                        not                                          analysis_output.success:
                outcome.decision_reason   =   f"attempt   {attempt}:   analyzer   failed:   {analysis_output.reasoning}"
```

```
            outcome.duration_ms = (datetime.utcnow() - started).total_seconds() * 1000
            return outcome
        except Exception as e:
            outcome.decision_reason = f"attempt {attempt}: analyzer raised: {e}"
            outcome.duration_ms = (datetime.utcnow() - started).total_seconds() * 1000
            return outcome

        # Stage 2 — Healer
        try:
            fix_input = FixGenerationInput(
                bug_analysis=analysis_output.result,
                original_code=augmented_context,
                file_path=file_path,
                line_number=line_number,
                language=language,
            )
            fix_output = await self.agent_manager.healer.process(fix_input)
            if not fix_output.success or not fix_output.result.candidates:
                outcome.decision_reason = f"attempt {attempt}: healer produced no fix: {fix_output.reasoning}"
                outcome.duration_ms = (datetime.utcnow() - started).total_seconds() * 1000
                return outcome
        except Exception as e:
            outcome.decision_reason = f"attempt {attempt}: healer raised: {e}"
            outcome.duration_ms = (datetime.utcnow() - started).total_seconds() * 1000
            return outcome

        fix_result = fix_output.result
        best_candidate = fix_result.candidates[fix_result.recommended_index]
        outcome.fixed_code = best_candidate.fixed_code
        outcome.explanation = best_candidate.explanation

        single_agent_approved = bool(best_candidate.fixed_code) and best_candidate.fixed_code.strip() !=
code_context.strip()
        outcome.consensus_result = {
            "approved": single_agent_approved,
            "prepare_votes": 1 if single_agent_approved else 0,
            "commit_votes": 1 if single_agent_approved else 0,
            "reason": "single-agent baseline (no consensus)",
        }

        if not single_agent_approved:
            outcome.decision_reason = f"attempt {attempt}: healer returned the original code unchanged"
            outcome.duration_ms = (datetime.utcnow() - started).total_seconds() * 1000
            return outcome

        # Stage 3 — Sandbox check
        sandbox_result: SandboxResult = await self.sandbox.run(
```

```
    code=best_candidate.fixed_code,
    language=language,
    test_command=test_command,
    timeout_s=self.sandbox_timeout_s,
)
outcome.sandbox_result = {
    "success": sandbox_result.success,
    "backend": sandbox_result.backend.value,
    "exit_code": sandbox_result.exit_code,
    "duration_ms": sandbox_result.duration_ms,
    "timed_out": sandbox_result.timed_out,
    "tests_passed": sandbox_result.tests_passed,
    "tests_failed": sandbox_result.tests_failed,
    "error": sandbox_result.error,
    "stdout_tail": sandbox_result.stdout[-1000:] if sandbox_result.stdout else "",
    "stderr_tail": sandbox_result.stderr[-1000:] if sandbox_result.stderr else "",
}

if sandbox_result.success:
    outcome.success = True
    outcome.safety_violation = False
    outcome.final_stage = RepairStage.DONE
    outcome.decision_reason = f"attempt {attempt}: single-agent baseline: fix passed sandbox"
    outcome.duration_ms = (datetime.utcnow() - started).total_seconds() * 1000
    return outcome

# Sandbox failed; retry with augmented context if attempts remain
stderr_excerpt = (sandbox_result.stderr or "")[:1500]
if attempt < self.max_attempts:
    augmented_context = (
        f"{code_context}\n\n"
        f"# === SELF-DEBUG FEEDBACK FROM ATTEMPT {attempt} ===\n"
        f"# The previous fix failed sandbox execution.\n"
        f"# Failure signal:\n# {stderr_excerpt or sandbox_result.short_reason()}\n"
        f"# Generate a different fix that addresses this specific failure mode."
    )
    continue

# All attempts exhausted
outcome.success = False
outcome.safety_violation = True
outcome.final_stage = RepairStage.SANDBOX_VALIDATION
outcome.decision_reason = (
    f"attempt {attempt}/{self.max_attempts}: single-agent baseline: "
    f"sandbox rejected ({sandbox_result.short_reason()})"
)
outcome.duration_ms = (datetime.utcnow() - started).total_seconds() * 1000
return outcome
```

```
        # Should not reach here, but guard for completeness
        outcome.duration_ms = (datetime.utcnow() - started).total_seconds() * 1000
        return outcome
```

G.8 Isolated Sandbox Executor (app/sandbox/docker_executor.py)

```
"""
Sandbox executor for running candidate fixes safely.

A consensus-approved fix is run inside an isolated container (Docker preferred,
subprocess fallback) before being applied to the real codebase. This is
critical for safety: a fix that satisfies the LLM consensus may still break
something at runtime, and we must not apply it without an empirical check.

The executor returns a SandboxResult capturing:
- whether the patched code runs at all
- whether the supplied test command passes
- stdout, stderr, exit code, and wall-clock duration

Author: Millicent Mufambi (H240624A)
"""

from __future__ import annotations

import asyncio
import os
import shutil
import sys
import tempfile
from dataclasses import dataclass, field
from datetime import datetime
from enum import Enum
from pathlib import Path
from typing import Optional, List, Dict

import structlog

logger = structlog.get_logger(__name__)


class SandboxBackend(str, Enum):
    """Which isolation backend was used for execution."""
```

```
    DOCKER                                                  =                                                  "docker"
    SUBPROCESS                                              =                                               "subprocess"
    UNAVAILABLE                                             =                                               "unavailable"



@dataclass
class                                                                                                   SandboxResult:
    """Outcome        of        running        a        candidate        fix        in        the        sandbox."""
    success:                                                                                                    bool
    backend:                                                                                               SandboxBackend
    exit_code:                                  int                                  =                              -1
    stdout:                                    str                                  =                               ""
    stderr:                                    str                                  =                               ""
    duration_ms:                                 float                                  =                           0.0
    timed_out:                                  bool                                 =                           False
    error:                                Optional[str]                                  =                           None
    tests_passed:                              Optional[int]                                 =                        None
    tests_failed:                              Optional[int]                                 =                        None
    timestamp:                     datetime                     =                   field(default_factory=datetime.utcnow)

    def                                short_reason(self)                                 ->                           str:
        """One-line                  description                  suitable                  for                  logging."""
        if                                                                                                  self.timed_out:
            return                      f"timeout                      after                      {self.duration_ms:.0f}ms"
        if                                                                                                      self.error:
            return                                                                                               self.error
        if                                                                                                  self.success:
            return                f"passed                (exit                0,                {self.duration_ms:.0f}ms)"
        return             f"failed             (exit             {self.exit_code},             {self.duration_ms:.0f}ms)"



class                                                                                                 SandboxExecutor:
    """
    Run             candidate             code             in             an             isolated             environment.

    Strategy:
      1.     If     Docker     is     available,     run     inside     a     one-shot     container     with     no     network
         and                        a                        tight                        CPU/memory                        limit.
      2.     If     Docker     is     not     available,     fall     back     to     a     subprocess     in     a     tempdir
         (less      isolated,      but      still      keeps      the      fix      off      the      real      codebase).

    The      executor      never      touches      the      real      source      tree      directly.      Callers      stage
    the       fix       into       a       temporary       directory       and       pass       that       to       `run`.
    """
```

```
DEFAULT_DOCKER_IMAGE                                                    =                                                    {
    "python":                                                                                                  "python:3.11-slim",
    "javascript":                                                                                                  "node:20-slim",
    "java":                                                                                                "eclipse-temurin:17-jdk",
}


def                                                                                                                    __init__(
    self,
    default_timeout_s:                                int                                =                                30,
    memory_limit:                                   str                                =                              "512m",
    cpu_limit:                                    float                                =                                1.0,
    force_backend:                          Optional[SandboxBackend]                          =                          None,
):
    self.default_timeout_s                                          =                                      default_timeout_s
    self.memory_limit                                               =                                           memory_limit
    self.cpu_limit                                                  =                                              cpu_limit
    self._force_backend                                              =                                         force_backend
    self._docker_client                                                    =                                             None
    self._docker_available:                           Optional[bool]                          =                          None


    logger.info(
        "Sandbox                                              executor                                          initialised",
        default_timeout_s=default_timeout_s,
        memory_limit=memory_limit,
        cpu_limit=cpu_limit,
    )


#                                                    ---------------------------------------------------------------
#                                                       Public                                                      API
#                                                    ---------------------------------------------------------------
async                                                    def                                                        run(
    self,
    code:                                                                                                           str,
    language:                                    str                                    =                       "python",
    test_command:                              Optional[List[str]]                            =                          None,
    run_command:                               Optional[List[str]]                            =                          None,
    timeout_s:                                  Optional[int]                                 =                          None,
    extra_files:                     Optional[Dict[str,                    str]]                    =                    None,
)                                                   ->                                                  SandboxResult:
    """
    Stage          `code`          into          a          temp          dir          and          run          it.


    Args:
        code:       The       candidate       fixed       source       as       a       single       file       blob.
        language:        Source        language;        controls        default        image        and        command.
        test_command:      Optional      command      (list      of      args)      to      run      the      tests.
                     If       None,       falls       back       to       running       the       file       directly.
```

```
    run_command:        Optional        command        to        invoke        when        no        test_command
                is            given.            Defaults            to            a            language-appropriate
                "run                            this                            file"                            command.
    timeout_s:                  Wall-clock                  timeout                  in                  seconds.
    extra_files:       Map       of       relative-path       ->       file       content.       Useful       for
                attaching         test         data         alongside         the         candidate         fix.

Returns:
    SandboxResult.
"""
timeout_s                  =                  timeout_s                  or                  self.default_timeout_s

with               tempfile.TemporaryDirectory(prefix="codeflow-sandbox-")               as               workdir:
    workpath                                          =                                          Path(workdir)
    main_filename                                  =                                  self._main_filename_for(language)
    (workpath                  /                  main_filename).write_text(code,                  encoding="utf-8")

    for           relpath,           content           in           (extra_files           or           {}).items():
        target                      =                     workpath                     /                     relpath
        target.parent.mkdir(parents=True,                                                                  exist_ok=True)
        target.write_text(content,                                                                         encoding="utf-8")

    command    =    test_command    or    run_command    or    self._default_run_command(language,    main_filename)
    backend                            =                            await                            self._pick_backend()

    if                                                                                                 self._force_backend:
        backend                                          =                                          self._force_backend

    if                          backend                          ==                          SandboxBackend.DOCKER:
        result       =       await       self._run_docker(workpath,       command,       language,       timeout_s)
    elif                        backend                        ==                        SandboxBackend.SUBPROCESS:
        result         =         await         self._run_subprocess(workpath,         command,         timeout_s)
    else:
        result                                          =                                          SandboxResult(
            success=False,
            backend=SandboxBackend.UNAVAILABLE,
            error="No                      execution                      backend                      available",
        )

    self._extract_test_counts(result)
    logger.info(
        "Sandbox                                        run                                        finished",
        backend=result.backend.value,
        success=result.success,
        duration_ms=result.duration_ms,
        reason=result.short_reason(),
```

```
        )
        return result

# ---------------------------------------------------------------------
# Backend selection
# ---------------------------------------------------------------------
async def _pick_backend(self) -> SandboxBackend:
    if self._docker_available is None:
        self._docker_available = await self._probe_docker()
    if self._docker_available:
        return SandboxBackend.DOCKER
    return SandboxBackend.SUBPROCESS


async def _probe_docker(self) -> bool:
    try:
        import docker  # type: ignore  # imported lazily so app starts without docker
    except ImportError:
        logger.warning("docker SDK not installed; falling back to subprocess sandbox")
        return False
    try:
        self._docker_client = docker.from_env()
        self._docker_client.ping()
        return True
    except Exception as e:
        logger.warning("Docker daemon unreachable; falling back to subprocess sandbox", error=str(e))
        self._docker_client = None
        return False


# ---------------------------------------------------------------------
# Docker backend
# ---------------------------------------------------------------------
async def _run_docker(
    self,
    workpath: Path,
    command: List[str],
    language: str,
    timeout_s: int,
) -> SandboxResult:
    image = self.DEFAULT_DOCKER_IMAGE.get(language.lower(), self.DEFAULT_DOCKER_IMAGE["python"])

    loop = asyncio.get_event_loop()
    started = datetime.utcnow()

    def _run_blocking():
        assert self._docker_client is not None
        container = self._docker_client.containers.run(
            image=image,
```

```
        command=command,
        working_dir="/work",
        volumes={str(workpath):             {"bind":             "/work",             "mode":             "rw"}},
        network_disabled=True,
        mem_limit=self.memory_limit,
        nano_cpus=int(self.cpu_limit                                    *                                    1e9),
        detach=True,
        stdout=True,
        stderr=True,
    )
    try:
        exit_status                                =                               container.wait(timeout=timeout_s)
        exit_code                     =                     exit_status.get("StatusCode",                     -1)
        logs_stdout     =     container.logs(stdout=True,     stderr=False).decode("utf-8",     errors="replace")
        logs_stderr     =     container.logs(stdout=False,     stderr=True).decode("utf-8",     errors="replace")
        return               exit_code,               logs_stdout,               logs_stderr,               False
    except                                Exception                                as                                e:
        if                            "timeout"                            in                            str(e).lower():
            try:
                container.kill()
            except                                                                                                Exception:
                pass
            return                    -1,                    "",                    str(e),                    True
        return                    -1,                    "",                    str(e),                    False
    finally:
        try:
            container.remove(force=True)
        except                                                                                                Exception:
            pass


try:
    exit_code,     stdout,     stderr,     timed_out     =     await     loop.run_in_executor(None,     _run_blocking)
except                                Exception                                as                                e:
    duration_ms        =        (datetime.utcnow()        -        started).total_seconds()        *        1000
    return                                                                                                SandboxResult(
        success=False,
        backend=SandboxBackend.DOCKER,
        duration_ms=duration_ms,
        error=str(e),
    )


duration_ms         =         (datetime.utcnow()         -         started).total_seconds()         *         1000
return                                                                                                SandboxResult(
    success=(exit_code              ==              0              and              not              timed_out),
    backend=SandboxBackend.DOCKER,
    exit_code=exit_code,
    stdout=stdout[-8000:],
    stderr=stderr[-8000:],
```

```
        duration_ms=duration_ms,
        timed_out=timed_out,
    )

# ---------------------------------------------------------------------
# Subprocess backend
# ---------------------------------------------------------------------
async def _run_subprocess(
    self,
    workpath: Path,
    command: List[str],
    timeout_s: int,
) -> SandboxResult:
    started = datetime.utcnow()
    try:
        proc = await asyncio.create_subprocess_exec(
            *command,
            cwd=str(workpath),
            stdout=asyncio.subprocess.PIPE,
            stderr=asyncio.subprocess.PIPE,
            env={**os.environ, "PYTHONDONTWRITEBYTECODE": "1"},
        )
    except FileNotFoundError as e:
        return SandboxResult(
            success=False,
            backend=SandboxBackend.SUBPROCESS,
            error=f"Command not found: {e}",
        )

    try:
        stdout_bytes, stderr_bytes = await asyncio.wait_for(proc.communicate(), timeout=timeout_s)
        timed_out = False
    except asyncio.TimeoutError:
        proc.kill()
        await proc.wait()
        stdout_bytes, stderr_bytes = b"", b""
        timed_out = True

    duration_ms = (datetime.utcnow() - started).total_seconds() * 1000
    exit_code = proc.returncode if not timed_out else -1
    return SandboxResult(
        success=(exit_code == 0 and not timed_out),
        backend=SandboxBackend.SUBPROCESS,
        exit_code=exit_code,
        stdout=stdout_bytes.decode("utf-8", errors="replace")[-8000:],
        stderr=stderr_bytes.decode("utf-8", errors="replace")[-8000:],
        duration_ms=duration_ms,
        timed_out=timed_out,
```

```
        )

    # ------------------------------------------------------------------
    #                                                                Helpers
    # ------------------------------------------------------------------
    @staticmethod
    def _main_filename_for(language: str) -> str:
        return {
            "python": "main.py",
            "javascript": "main.js",
            "java": "Main.java",
        }.get(language.lower(), "main.txt")

    def _default_run_command(self, language: str, main_filename: str) -> List[str]:
        return {
            "python": [sys.executable if self._force_backend == SandboxBackend.SUBPROCESS else "python", main_filename],
            "javascript": ["node", main_filename],
            "java": ["sh", "-c", f"javac {main_filename} && java {main_filename.replace('.java', '')}"],
        }.get(language.lower(), [sys.executable, main_filename])

    @staticmethod
    def _extract_test_counts(result: SandboxResult) -> None:
        """Best-effort parsing of pytest / unittest output."""
        import re
        for stream in (result.stdout, result.stderr):
            m = re.search(r"(\d+) passed.*?(\d+) failed", stream)
            if m:
                result.tests_passed = int(m.group(1))
                result.tests_failed = int(m.group(2))
                return
            m = re.search(r"(\d+) passed", stream)
            if m and result.tests_passed is None:
                result.tests_passed = int(m.group(1))


# Module-level singleton for convenience
sandbox_executor = SandboxExecutor()
```

G.9 Genuine BFT Simulation (app/consensus/bft_sim.py)

```
"""
Genuine Byzantine-fault-tolerance simulation for the BFT-MAS consensus layer.

Unlike app/consensus/pbft.py (a single-process vote *counter* used by the repair
pipeline), this module models PBFT as it is meant to be evaluated: independent
```

```
replicas that exchange signed messages over a network that can delay, drop,
partition, and reorder, against a Byzantine adversary that can EQUIVOCATE (send
different proposals/votes to different replicas) and attempt to FORGE messages.

This is what lets us actually demonstrate the namesake property:
  *  Safety under an equivocating primary: a malicious leader sends fix X to some
     replicas and fix Y to others. A naive "trust the proposal" scheme makes
     honest replicas commit different fixes (split-brain). PBFT's prepare/commit
     quorum certificates prevent any two honest replicas from committing
     different values -> safety holds.
  *  The f vs f+1 boundary emerges from quorum *certificates exchanged between
     replicas*, not from central arithmetic.
  *  Real Ed25519 signatures: a Byzantine replica cannot impersonate an honest
     one; forged messages are rejected on receipt, so a single faulty node cannot
     manufacture a quorum.

Deterministic and round-based, so every result is exactly reproducible and
needs no LLM API calls.

Author: Millicent Mufambi (H240624A)
"""
from __future__ import annotations

import hashlib
from dataclasses import dataclass, field, replace
from typing import Any, Dict, List, Optional, Set, Tuple

from cryptography.hazmat.primitives.asymmetric.ed25519 import (
    Ed25519PrivateKey,
    Ed25519PublicKey,
)
from cryptography.exceptions import InvalidSignature


def digest(value: Any) -> str:
    """SHA-256 digest of a proposal value."""
    return hashlib.sha256(repr(value).encode()).hexdigest()[:16]


# ---------------------------------------------------------------------------- keys
class KeyRing:
    """Per-replica Ed25519 keypairs; public keys known to all (a PKI)."""

    def __init__(self, node_ids: List[int]):
        self._priv: Dict[int, Ed25519PrivateKey] = {
```

```
            i: Ed25519PrivateKey.generate() for i in node_ids
        }
        self._pub: Dict[int, Ed25519PublicKey] = {
            i: k.public_key() for i, k in self._priv.items()
        }

    def sign(self, node_id: int, payload: bytes) -> bytes:
        return self._priv[node_id].sign(payload)

    def verify(self, claimed_sender: int, payload: bytes, sig: bytes) -> bool:
        pub = self._pub.get(claimed_sender)
        if pub is None or sig is None:
            return False
        try:
            pub.verify(sig, payload)
            return True
        except InvalidSignature:
            return False


# ---------------------------------------------------------------------- message
@dataclass
class Msg:
    mtype: str  # 'preprepare' | 'prepare' | 'commit'
    view: int
    seq: int
    digest: str
    sender: int  # the claimed signer
    value: Any = None  # carried only by pre-prepare
    sig: Optional[bytes] = None

    def signing_bytes(self) -> bytes:
        # Signature binds type, view, seq, digest and claimed sender (NOT value:
        # value integrity is bound through `digest`, which the receiver re-checks).
        return f"{self.mtype}|{self.view}|{self.seq}|{self.digest}|{self.sender}".encode()


# ---------------------------------------------------------------------- network
class Network:
    """Deterministic, round-based message network with fault injection."""

    def __init__(self, n: int, partition: Optional[List[Set[int]]] = None,
                 drop_links: Optional[Set[Tuple[int, int]]] = None):
        self.n = n
        # partition: list of groups; nodes in different groups cannot talk.
```

```
        self._group: Dict[int, int] = {}
        if partition:
            for gid, grp in enumerate(partition):
                for node in grp:
                    self._group[node] = gid
        self.drop_links = drop_links or set()
        self.pending: List[Tuple[int, int, Msg]] = []
        self.delivered_count = 0
        self.dropped_count = 0

    def _blocked(self, src: int, dst: int) -> bool:
        if (src, dst) in self.drop_links:
            return True
        if self._group and self._group.get(src) != self._group.get(dst):
            return True
        return False

    def send(self, src: int, dst: int, msg: Msg) -> None:
        self.pending.append((src, dst, msg))

    def broadcast(self, src: int, msg: Msg, nodes: List[int]) -> None:
        # NB: a replica also "sends to itself" so its own vote is counted.
        for dst in nodes:
            self.send(src, dst, replace(msg))

    def heal(self) -> None:
        self._group = {}
        self.drop_links = set()

    def deliver(self) -> Dict[int, List[Msg]]:
        """Deliver all in-flight messages, honouring partitions/drops."""
        inbox: Dict[int, List[Msg]] = {}
        for src, dst, msg in self.pending:
            if self._blocked(src, dst):
                self.dropped_count += 1
                continue
            inbox.setdefault(dst, []).append(msg)
            self.delivered_count += 1
        self.pending = []
        return inbox


# ---------------------------------------------------------------------- replica
class Replica:
    """
    A single PBFT replica running the real three-phase state machine.
```

```
naive=True degrades it to a "trust the proposal" baseline: it decides on the
pre-prepare without prepare/commit certificates — i.e. plain proposal-trust,
which is what breaks under an equivocating primary.
"""

def __init__(self, rid: int, n: int, f: int, keys: KeyRing, naive: bool = False):
    self.id = rid
    self.n = n
    self.f = f
    self.quorum = 2 * f + 1
    self.keys = keys
    self.naive = naive
    # per (view,seq) state
    self.accepted_pp: Dict[Tuple[int, int], str] = {}
    self.prepares: Dict[Tuple[int, int], Dict[str, Set[int]]] = {}
    self.commits: Dict[Tuple[int, int], Dict[str, Set[int]]] = {}
    self.sent_prepare: Set[Tuple[int, int]] = set()
    self.sent_commit: Set[Tuple[int, int]] = set()
    self.committed: Dict[Tuple[int, int], str] = {}
    self.committed_value: Dict[Tuple[int, int], Any] = {}
    self.equivocations_detected = 0
    self.forged_rejected = 0

def primary(self, view: int) -> int:
    return view % self.n

def _sign(self, m: Msg) -> Msg:
    m.sig = self.keys.sign(self.id, m.signing_bytes())
    return m

def handle(self, m: Msg, net: Network) -> None:
    # 1. Verify signature against the CLAIMED sender's public key. A forged
    #    message (byzantine node signing as someone else) is rejected here.
    if not self.keys.verify(m.sender, m.signing_bytes(), m.sig):
        self.forged_rejected += 1
        return
    if m.mtype == "preprepare":
        self._on_preprepare(m, net)
    elif m.mtype == "prepare":
        self._on_prepare(m, net)
    elif m.mtype == "commit":
        self._on_commit(m, net)

def _on_preprepare(self, m: Msg, net: Network) -> None:
    key = (m.view, m.seq)
```

```
    # Pre-prepare must come from the primary of that view, and its digest
    # must match the value it carries (binds value <-> digest).
    if m.sender != self.primary(m.view):
        return
    if digest(m.value) != m.digest:
        return
    # Equivocation guard: an honest replica accepts ONE pre-prepare per
    # (view,seq). A second, conflicting digest is logged and ignored.
    if key in self.accepted_pp:
        if self.accepted_pp[key] != m.digest:
            self.equivocations_detected += 1
        return
    self.accepted_pp[key] = m.digest
    self.committed_value[key] = m.value

    if self.naive:
        # Baseline: trust the proposal, decide immediately. No cross-check
        # with peers -> different replicas can decide different values.
        self.committed[key] = m.digest
        return

    # PBFT: broadcast PREPARE for the accepted digest (incl. to self).
    if key not in self.sent_prepare:
        self.sent_prepare.add(key)
        pre = self._sign(Msg("prepare", m.view, m.seq, m.digest, self.id))
        net.broadcast(self.id, pre, list(range(self.n)))

def _on_prepare(self, m: Msg, net: Network) -> None:
    key = (m.view, m.seq)
    self.prepares.setdefault(key, {}).setdefault(m.digest, set()).add(m.sender)
    self._maybe_commit_phase(m.view, m.seq, m.digest, net)

def _maybe_commit_phase(self, view: int, seq: int, d: str, net: Network) -> None:
    key = (view, seq)
    if self.naive:
        return
    votes = len(self.prepares.get(key, {}).get(d, set()))
    # "prepared": accepted the matching pre-prepare AND 2f+1 prepares for d.
    if votes >= self.quorum and self.accepted_pp.get(key) == d and key not in self.sent_commit:
        self.sent_commit.add(key)
        c = self._sign(Msg("commit", view, seq, d, self.id))
        net.broadcast(self.id, c, list(range(self.n)))

def _on_commit(self, m: Msg, net: Network) -> None:
    key = (m.view, m.seq)
    self.commits.setdefault(key, {}).setdefault(m.digest, set()).add(m.sender)
    votes = len(self.commits[key][m.digest])
```

```
        #      "committed-local":      prepared      AND      2f+1      commits      for      the      same      digest.
        if        (votes        >=        self.quorum        and        self.accepted_pp.get(key)        ==        m.digest
                and                    key                    not                    in                    self.committed):
            self.committed[key]                                        =                                        m.digest


    def                retransmit(self,                net:                Network)                ->                None:
        """Re-broadcast           outstanding           prepare/commit           votes           (models           periodic
        retransmission,        so        progress        resumes        once        a        partition        heals)."""
        if                                                                                                    self.naive:
            return
        for                            key                            in                            list(self.accepted_pp):
            if                                key                                in                                self.committed:
                continue
            view,                                    seq                                    =                                    key
            d                                              =                                                 self.accepted_pp[key]
            #      The      primary      re-broadcasts      its      pre-prepare      so      replicas      that      missed
            #      it      (e.g.      during      a      partition)      can      still      receive      the      proposal.
            if        self.id        ==        self.primary(view)        and        key        in        self.committed_value:
                val                                          =                                           self.committed_value[key]
                pp        =        Msg("preprepare",        view,        seq,        d,        self.id,        value=val)
                net.broadcast(self.id,                          self._sign(pp),                          list(range(self.n)))
            if                                key                                in                                self.sent_prepare:
                net.broadcast(self.id,        self._sign(Msg("prepare",        view,        seq,        d,        self.id)),
                              list(range(self.n)))
            if                                key                                in                                self.sent_commit:
                net.broadcast(self.id,        self._sign(Msg("commit",        view,        seq,        d,        self.id)),
                              list(range(self.n)))
```

G.10 Byzantine Proof Experiments (benchmarks/bft_demo.py)

```
"""
Byzantine-fault-tolerance                proof                experiments                for                BFT-MAS.

Runs       the       genuine       PBFT       replica       simulation       (app/consensus/bft_sim.py)       against       a
battery      of      adversarial      scenarios,      and      contrasts      it      with      a      NAIVE      proposal-trust
baseline.      The      headline      result      is      safety      under      an      EQUIVOCATING      PRIMARY:      the      naive
scheme       splits       (honest       replicas       commit       different       fixes),       PBFT       does       not.

No          LLM          API          calls;          fully          deterministic          and          reproducible.

Run:                                          python                        -m                        benchmarks.bft_demo
Outputs:                        benchmark_results/bft_demo__results.csv                        +                        .json
"""
from                             __future__                                 import                                annotations

import                                                                                                                csv
```

```
import json
from pathlib import Path
from typing import Any, Dict, List

from app.consensus.bft_sim import KeyRing, Network, Replica, Msg, digest

OUT = Path("benchmark_results")
KEY = (0, 1)  # (view, sequence) used by every scenario

SCENARIOS = [
    ("clean", "Honest primary, no faults"),
    ("equivocate_primary", "Malicious primary sends fix X to half, fix Y to half"),
    ("equivocate_validator", "Honest primary; one validator sends conflicting prepares"),
    ("crash_f", "f = 1 replica crashed (silent)"),
    ("crash_f1", "f + 1 = 2 replicas crashed (silent)"),
    ("partition", "Network split into two halves, no heal"),
    ("partition_heal", "Network split, then healed"),
    ("forgery", "Byzantine validator forges prepares impersonating honest nodes"),
]



def run_scenario(protocol: str, fault: str, n: int = 4, f: int = 1,
        value: str = "FIX_X", alt: str = "FIX_Y", max_rounds: int = 12) -> Dict[str, Any]:
    nodes = list(range(n))
    keys = KeyRing(nodes)
    naive = protocol == "naive"

    crashed: set = set()
    byzantine: set = set()
    partition = None
    if fault == "crash_f":
        crashed = {n - 1}
    elif fault == "crash_f1":
        crashed = {n - 1, n - 2}
    elif fault in ("partition", "partition_heal"):
        partition = [set(nodes[: n // 2]), set(nodes[n // 2:])]
    elif fault == "equivocate_primary":
        byzantine = {0}
    elif fault in ("equivocate_validator", "forgery"):
        byzantine = {n - 1}

    net = Network(n, partition=partition)
    active = [i for i in nodes if i not in crashed]
    replicas: Dict[int, Replica] = {
        i: Replica(i, n, f, keys, naive=naive) for i in active
    }
```

```
primary                                                 =                                                       0


def                     signed(mtype,                   d,                      sender,                     val=None):
    m           =           Msg(mtype,          0,          1,          d,          sender,          value=val)
    m.sig                       =                       keys.sign(sender,                       m.signing_bytes())
    return                                                                                                      m


#      ----      inject      the      initial      pre-prepare(s)      according      to      the      fault      ----
if                              fault                               ==                              "equivocate_primary":
    half                            =                           n                           //                          2
    for                                 dst                                 in                                  active:
        v           =           value           if          dst          <          half          else          alt
        net.send(primary,          dst,          signed("preprepare",          digest(v),          primary,          v))
else:
    for                                 dst                                 in                                  active:
        net.send(primary,        dst,        signed("preprepare",        digest(value),        primary,        value))


#     equivocating     validator:     a     byzantine     node     floods     conflicting     prepares     for     a
#          bogus          digest,          trying          to          break          the          honest          quorum.
if                              fault                               ==                              "equivocate_validator":
    byz                             =                           n                           -                           1
    for                                 dst                                 in                                  active:
        net.send(byz,               dst,               signed("prepare",               digest(alt),               byz))


#     forgery:     byzantine     node     fabricates     prepares     CLAIMING     to     be     honest     nodes     1
#    and    2    (to    fake    a    quorum),    but    signs    with    its    own    key    ->    must    be    rejected.
if                                  fault                                   ==                                  "forgery":
    byz                             =                           n                           -                           1
    for                         fake                            in                          (1,                         2):
        fm          =           Msg("prepare",          0,          1,          digest(value),          fake)
        fm.sig        =        keys.sign(byz,        fm.signing_bytes())                #        wrong        signer
        for                                 dst                                 in                                  active:
            net.send(byz,                                       dst,                                        fm)


#                           ----                            run                             rounds                          ----
healed                                                  =                                                       False
heal_at                         =                           max_rounds                          //                          2
for                                 rnd                                 in                                  range(max_rounds):
    if      fault      ==      "partition_heal"      and      rnd      ==      heal_at      and      not      healed:
        net.heal()
        healed                                                  =                                                       True
        for                                 r                               in                                  replicas.values():
            r.retransmit(net)
    inbox                                               =                                                   net.deliver()
    if      not      inbox      and      not      (fault      ==      "partition_heal"      and      not      healed):
        break
    for                     rid,                    msgs                    in                      inbox.items():
```

```
            if rid in crashed:
                continue
            for m in msgs:
                replicas[rid].handle(m, net)
        # periodic retransmission so progress resumes after delays/heals
        for r in replicas.values():
            r.retransmit(net)


    # ---- evaluate honest replicas only ----
    honest = [i for i in active if i not in byzantine]
    decided = {i: replicas[i].committed_value.get(KEY)
               for i in honest if KEY in replicas[i].committed}
    distinct = set(decided.values())
    safety_violated = len(distinct) > 1                 # two honest replicas disagree
    progress = len(decided) >= f + 1                    # enough honest replicas decided
    forged_rej = sum(replicas[i].forged_rejected for i in honest)
    equiv_det = sum(replicas[i].equivocations_detected for i in honest)
    return {
        "protocol": protocol,
        "fault": fault,
        "n": n, "f": f,
        "honest_decided": len(decided),
        "distinct_values": len(distinct),
        "committed_values": sorted(str(v) for v in distinct),
        "safety_violated": safety_violated,
        "progress": progress,
        "forged_rejected": forged_rej,
        "equivocations_detected": equiv_det,
        "outcome": _verdict(fault, safety_violated, progress),
    }


def _verdict(fault: str, safety_violated: bool, progress: bool) -> str:
    if safety_violated:
        return "SAFETY VIOLATED (split-brain)"
    if progress:
        return "agreed + progress"
    return "safe, no progress (needs view-change/heal)"


def main():
    OUT.mkdir(exist_ok=True)
    rows: List[Dict[str, Any]] = []
    for fault, _desc in SCENARIOS:
        for protocol in ("naive", "pbft"):
            rows.append(run_scenario(protocol, fault))
```

```
    # write CSV + JSON
    cols = ["fault", "protocol", "n", "f", "honest_decided", "distinct_values",
            "committed_values", "safety_violated", "progress", "forged_rejected",
            "equivocations_detected", "outcome"]
    with (OUT / "bft_demo__results.csv").open("w", newline="", encoding="utf-8") as fh:
        w = csv.DictWriter(fh, fieldnames=cols)
        w.writeheader()
        for r in rows:
            w.writerow({k: (";".join(r[k]) if isinstance(r[k], list) else r[k]) for k in cols})
    (OUT / "bft_demo__results.json").write_text(
        json.dumps({"scenarios": dict(SCENARIOS), "rows": rows}, indent=2), encoding="utf-8")

    # console summary
    desc = dict(SCENARIOS)
    print(f"{'SCENARIO':22} {'NAIVE':32} {'PBFT (genuine)':32}")
    print("-" * 88)
    by = {(r["fault"], r["protocol"]): r for r in rows}
    for fault, _ in SCENARIOS:
        nv, pb = by[(fault, "naive")], by[(fault, "pbft")]
        print(f"{fault:22} {nv['outcome']:32} {pb['outcome']:32}")
    print("\nHeadline (equivocating primary):")
    nv = by[("equivocate_primary", "naive")]
    pb = by[("equivocate_primary", "pbft")]
    print(f"  naive -> {nv['outcome']}; honest committed values = {nv['committed_values']}")
    print(f"  pbft  -> {pb['outcome']}; honest committed values = {pb['committed_values']}")
    print(f"  forgery: pbft forged-messages rejected = {by[('forgery','pbft')]['forged_rejected']}")


if __name__ == "__main__":
    main()
```

G.11 BFT Property Tests (tests/test_bft_sim.py)

```
"""
Automated checks of the genuine-BFT properties demonstrated by bft_sim.

These assert the namesake guarantee directly:
  * Under an equivocating primary, naive proposal-trust splits (safety violation)
    while PBFT never lets two honest replicas commit different values.
  * PBFT tolerates exactly f faults (commits with f crashed, refuses with f+1).
  * Forged messages (impersonating another replica) are rejected, so a single
    Byzantine node cannot manufacture a quorum.
Deterministic; no network or API access.
"""
from benchmarks.bft_demo import run_scenario
```

```
def test_clean_both_progress():
    for proto in ("naive", "pbft"):
        r = run_scenario(proto, "clean")
        assert not r["safety_violated"] and r["progress"], r


def test_equivocating_primary_naive_splits_pbft_safe():
    naive = run_scenario("naive", "equivocate_primary")
    pbft = run_scenario("pbft", "equivocate_primary")
    # naive commits two different fixes on honest replicas -> split-brain
    assert naive["safety_violated"] is True
    assert naive["distinct_values"] == 2
    # PBFT never violates safety (no two honest replicas disagree)
    assert pbft["safety_violated"] is False


def test_pbft_never_violates_safety_anywhere():
    for fault in ("clean", "equivocate_primary", "equivocate_validator",
                  "crash_f", "crash_f1", "partition", "partition_heal", "forgery"):
        r = run_scenario("pbft", fault)
        assert r["safety_violated"] is False, (fault, r)


def test_pbft_tolerates_f_not_f_plus_1():
    assert run_scenario("pbft", "crash_f")["progress"] is True
    assert run_scenario("pbft", "crash_f1")["progress"] is False


def test_partition_safety_then_liveness_on_heal():
    assert run_scenario("pbft", "partition")["progress"] is False  # safe, no progress
    assert run_scenario("pbft", "partition_heal")["progress"] is True  # recovers


def test_forged_messages_rejected():
    r = run_scenario("pbft", "forgery")
    assert r["forged_rejected"] > 0
    assert r["safety_violated"] is False and r["progress"] is True


def test_deterministic():
    a = run_scenario("pbft", "equivocate_primary")
    b = run_scenario("pbft", "equivocate_primary")
```

```
    assert a == b
```

G.12 Real Defects4J (Java) Runner (benchmarks/defects4j_runner.py)

```
"""
Defects4J (Java) end-to-end runner.

Addresses reviewer concern R1 #2/#8 (cross-language validity) by running the
*real* BFT-MAS multi-agent consensus pipeline against genuine Java defects
from the Defects4J benchmark, using Defects4J's own checkout/compile/test
harness inside a Docker container.

Architecture (host + container split):
  * The Defects4J toolchain (Perl/Maven/JUnit + project repos) lives inside
    a long-lived Docker container built from
    data/bug_datasets/defects4j/Dockerfile. A host directory is bind-mounted
    at /workspace so the host can read the buggy source and write the
    candidate fix.
  * For each bug we: checkout the buggy version, read the modified class,
    run it through the host-side multi-agent consensus (Analyzer + Healer +
    Validators + PBFT), write the consensus-approved class back into the
    checkout, and run `defects4j compile && defects4j test`. A passing test
    suite is the empirical safety gate, exactly as the sandbox is for Python.

A safety violation is a consensus-approved fix whose Defects4J test suite
still fails — the cross-language analogue of the Python sandbox check.

Usage (after the image is built and the container is running):
    python -m benchmarks.defects4j_runner --project Lang --bugs 1,3,7 \
        --mode consensus --max-src-chars 8000

Output: benchmark_results/<prefix>__cases.csv and __summary.json
"""
from __future__ import annotations

import argparse
import asyncio
import csv
import json
import os
import re
import subprocess
from datetime import datetime
from pathlib import Path
from typing import Dict, List, Optional, Tuple
```

```
import structlog

logger = structlog.get_logger(__name__)

CONTAINER = "defects4j-container"



# ---------------------------------------------------------------------------
#  Docker helpers
# ---------------------------------------------------------------------------
def _exec(cmd: str, timeout: int = 600) -> Tuple[int, str, str]:
    """Run a shell command inside the Defects4J container."""
    proc = subprocess.run(
        ["docker", "exec", CONTAINER, "bash", "-lc", cmd],
        capture_output=True, text=True, timeout=timeout,
    )
    return proc.returncode, proc.stdout, proc.stderr


def container_running() -> bool:
    proc = subprocess.run(
        ["docker", "ps", "--filter", f"name={CONTAINER}", "--format", "{{.Names}}"],
        capture_output=True, text=True,
    )
    return CONTAINER in proc.stdout


def ensure_container(image: str, workspace_host: str) -> None:
    """Start a detached, long-lived Defects4J container if not already up."""
    if container_running():
        return
    # Remove any stopped container of the same name.
    subprocess.run(["docker", "rm", "-f", CONTAINER], capture_output=True, text=True)
    Path(workspace_host).mkdir(parents=True, exist_ok=True)
    subprocess.run([
        "docker", "run", "-d", "--name", CONTAINER,
        "-v", f"{workspace_host}:/workspace",
        image, "tail", "-f", "/dev/null",
    ], check=True, capture_output=True, text=True)
    logger.info("defects4j container started", image=image, workspace=workspace_host)


# ---------------------------------------------------------------------------
```

```
#                                   Defects4J                                   bug                                   access
#                                          ------------------------------------------------------------------------------
def              _export(workdir:              str,              prop:              str)              ->              str:
    code,     out,     err     =     _exec(f"cd     {workdir}     &&     defects4j     export     -p     {prop}     2>/dev/null")
    return                                                                                                          out.strip()



def           _extract_test_method(test_src:           str,           method:           str)           ->           str:
    """Best-effort      extraction      of      a      single      JUnit      test      method      body      by      name."""
    if                   not                   test_src                   or                   not                   method:
        return                                                                                                          ""
    idx                                          =                                          test_src.find(method)
    if                                    idx                                    ==                                    -1:
        return                                                                                                          ""
    #     walk     back     to     the     start     of     the     line     /     annotation,     then     brace-match     forward
    start                       =                       test_src.rfind("\n",                       0,                       idx)
    start         =         start         +         1         if         start         !=         -1         else         0
    depth,               i,               began               =               0,               idx,               False
    while                                 i                                 <                                 len(test_src):
        c                                             =                                             test_src[i]
        if                                    c                                    ==                                    "{":
            depth                  +=                  1;                  began                  =                  True
        elif                                  c                                       ==                                    "}":
            depth                                                    -=                                                      1
            if                  began                  and                  depth                  ==                  0:
                return                              test_src[start:i                              +                              1]
        i                                                    +=                                                      1
    return                              test_src[start:idx                              +                              600]



def           checkout_bug(project:           str,           bug:           int)           ->           Optional[Dict]:
    """Force      a      CLEAN      checkout      of      one      buggy      version      and      return      its      metadata,
    the       buggy       class       source,       and       the       failing       test       method       source."""
    workdir                                          =                                               f"/workspace/{project}_{bug}b"
    #     Always     start     from     a     pristine     checkout     —     a     prior     run     may     have     written     a
    #              candidate              fix              into              this              workspace              dir.
    _exec(f"rm                                               -rf                                                {workdir}")
    code,                          out,                          err                          =                          _exec(
        f"defects4j       checkout       -p       {project}       -v       {bug}b       -w       {workdir}",       timeout=600
    )
    if                                    code                                       !=                                       0:
        logger.warning("checkout           failed",           project=project,           bug=bug,           err=err[-300:])
        return                                                                                                          None

    trigger                      =                      _export(workdir,                      "tests.trigger").splitlines()
    trigger              =              trigger[0]              if              trigger              else              ""
    modified                     =                     _export(workdir,                     "classes.modified").splitlines()
```

```
    primary_class = modified[0] if modified else ""
    src_dir = _export(workdir, "dir.src.classes") # e.g. src/main/java
    test_dir = _export(workdir, "dir.src.tests") # e.g. src/test/java

    rel = primary_class.replace(".", "/") + ".java"
    src_path_container = f"{workdir}/{src_dir}/{rel}"
    code, src, err = _exec(f"cat {src_path_container}")
    if code != 0 or not src:
        logger.warning("could not read buggy source", path=src_path_container)
        return None

    # Read the failing test method so the healer knows the expected behaviour.
    test_src = ""
    if "::" in trigger and test_dir:
        test_class, test_method = trigger.split("::", 1)
        trel = test_class.replace(".", "/") + ".java"
        _, tsrc, _ = _exec(f"cat {workdir}/{test_dir}/{trel}")
        test_src = _extract_test_method(tsrc, test_method)

    return {
        "workdir": workdir,
        "trigger": trigger,
        "primary_class": primary_class,
        "src_dir": src_dir,
        "src_path_container": src_path_container,
        "src_path_rel": f"{project}_{bug}b/{src_dir}/{rel}",
        "buggy_source": src,
        "test_source": test_src,
    }



def run_tests(workdir: str, trigger: str = "", full: bool = False) -> Tuple[bool, str]:
    """Compile + run tests inside the container. Returns (passed, detail).

    full=False: run only the triggering test(s) -> a *plausible* patch.
    full=True : run the whole relevant test suite -> a *correct* patch
                (the bug-exposing test passes AND nothing else regresses)."""
    test_arg = "" if full else (f"-t {trigger}" if trigger else "")
    code, out, err = _exec(
        f"cd {workdir} && defects4j compile 2>&1 && defects4j test {test_arg} 2>&1",
        timeout=1800,
    )
    blob = out + "\n" + err
    if "Cannot compile" in blob or "Compilation failed" in blob:
        return (False, "compile error")
    m = re.search(r"Failing tests:\s*(\d+)", blob)
```

```
    if m:
        return (int(m.group(1)) == 0, f"Failing tests: {m.group(1)}")
    return (False, blob.strip()[-200:])


# ---------------------------------------------------------------------------
# Host-side multi-agent consensus
# ---------------------------------------------------------------------------
def _build_agent_manager(args):
    from app.agents.agent_manager import AgentManager
    from app.config import settings

    am = AgentManager(
        f=args.f,
        n_validators=args.n_validators,
        diverse_validators=getattr(args, "diverse_validators", False),
        heterogeneous_validators=getattr(args, "heterogeneous_validators", False),
        independent_voters=getattr(args, "independent_voters", True),
    )
    # Real Defects4J classes are large (10-50 KB), so the healer must be able
    # to emit a full corrected file. Raise the output-token budget well above
    # the 2048 default; otherwise the regenerated class is truncated. The
    # validators receive both the original and candidate class, so their
    # CPU-bound Ollama call also needs a much longer timeout than the 60 s
    # default or it dies on these large prompts.
    for ag in [am.analyzer, am.healer, *am.validators]:
        ag.config.max_tokens = args.max_tokens
        ag.config.timeout_seconds = args.agent_timeout_s
    claude_client = openai_client = None
    if settings.anthropic_api_key and settings.anthropic_api_key != "your_anthropic_key_here":
        from anthropic import AsyncAnthropic
        claude_client = AsyncAnthropic(api_key=settings.anthropic_api_key,
                                       timeout=120.0, max_retries=5)
    if settings.openai_api_key and not settings.openai_api_key.startswith("your_"):
        from openai import AsyncOpenAI
        openai_client = AsyncOpenAI(api_key=settings.openai_api_key,
                                    timeout=120.0, max_retries=5)
    am.set_llm_clients(claude_client=claude_client, openai_client=openai_client,
                       ollama_url=settings.ollama_base_url)
    return am


async def repair_one(am, meta: Dict, mode: str) -> Dict:
    """Produce a candidate fix for one bug via consensus (or single-agent)."""
    result = {}
    for attempt in range(2):  # one retry for transient cloud-API errors (e.g. 500)
        result = await am.process_bug(
```

```
            error_message=f"Defects4J failing test: {meta['trigger']}",
            stack_trace=meta["trigger"],
            code_context=meta["buggy_source"],
            file_path=meta["primary_class"].replace(".", "/") + ".java",
            line_number=1,
            language="java",
        )
        stage = result.get("failure_stage", "")
        if result.get("success") or stage not in ("analysis_failed", "pipeline_error", "generation_failed"):
            break
        logger.warning("retrying after transient failure", stage=stage, attempt=attempt)
    fixed = (result.get("fix") or {}).get("fixed_code") if result.get("success") else None
    return {
        "consensus_approved": bool((result.get("consensus") or {}).get("approved")),
        "fixed_code": fixed,
        "reason": result.get("reason") or (result.get("consensus") or {}).get("reason", ""),
    }


def _extract_java(text: str, fallback: str) -> str:
    if not text:
        return fallback
    blocks = re.findall(r"```(?:java)?\s*\n(.*?)```", text, re.DOTALL)
    if blocks:
        return max(blocks, key=len).strip()
    if re.search(r"(public\s+class|class\s+\w+|package\s)", text):
        return text.strip()
    return fallback


# ---------------------------------------------------------------------------
#  Search/replace patch mode (scales to large real-world classes)
# ---------------------------------------------------------------------------
_BLOCK_RE = re.compile(
    r"<{5,}\s*SEARCH\s*\n(.*?)\n={5,}\s*\n(.*?)\n>{5,}\s*REPLACE", re.DOTALL)


def _patch_prompt(meta: Dict) -> str:
    return (
        "You are fixing one bug in a Java class so that a failing test passes. "
        "Return ONLY one or more edit blocks in EXACTLY this format and nothing else:\n\n"
        "<<<<<<< SEARCH\n"
        "(lines copied verbatim from the buggy file, including exact indentation)\n"
        "=======\n"
        "(the corrected lines)\n"
        ">>>>>>> REPLACE\n\n"
```

```
        "Make the smallest change that fixes the bug. Copy the SEARCH lines exactly as "
        "they appear so they can be found. Do not reformat or touch unrelated code.\n\n"
        f"Failing test: {meta['trigger']}\n"
        + (("Failing test source (your fix must make this pass):\n```java\n"
            + meta.get("test_source", "") + "\n```\n\n") if meta.get("test_source") else "")
        + f"Class: {meta['primary_class']}\n\n"
        "```java\n" + meta["buggy_source"] + "\n```\n")


async def healer_patch(openai_client, model: str, meta: Dict):
    """Ask the healer for minimal SEARCH/REPLACE edit blocks."""
    resp = await openai_client.chat.completions.create(
        model=model, max_tokens=2048, temperature=0.3, seed=7,
        messages=[{"role": "user", "content": _patch_prompt(meta)}],
    )
    text = resp.choices[0].message.content or ""
    return [(m.group(1), m.group(2)) for m in _BLOCK_RE.finditer(text)]


def apply_blocks(source: str, blocks):
    """Apply SEARCH/REPLACE blocks. Exact match first, then a whitespace-
    tolerant line match. Returns (new_source, applied, total)."""
    applied = 0
    out = source
    for search, replace in blocks:
        if not search.strip():
            continue
        if out.count(search) == 1:                                  # exact, unambiguous
            out = out.replace(search, replace, 1)
            applied += 1
            continue
        # whitespace-tolerant fallback, but ONLY if the normalised block
        # matches exactly one location (avoid corrupting the wrong region).
        norm = lambda s: "\n".join(line.strip() for line in s.strip().splitlines())
        target = norm(search)
        lines = out.splitlines(keepends=True)
        win = len(search.strip().splitlines())
        matches = [i for i in range(0, max(0, len(lines) - win + 1))
                   if norm("".join(lines[i:i + win])) == target]
        if len(matches) == 1:
            i = matches[0]
            out = ("".join(lines[:i]) + replace
                   + ("\n" if not replace.endswith("\n") else "")
                   + "".join(lines[i + win:]))
            applied += 1
    return out, applied, len(blocks)
```

```
async def consensus_vote(am, candidate: str, blocks, meta: Dict) -> bool:
    """Run the existing validators + PBFT consensus over a candidate.

    Validators assess the FOCUSED change (the search->replace snippets), not
    the whole multi-KB class, so the prompt stays inside the local model's
    context window. The PBFT proposal still carries the full candidate file."""
    import asyncio as _asyncio
    from app.agents.validator_agent import ValidationInput
    from app.agents.healer_agent import FixCandidate
    from app.consensus.message_types import FixProposal

    fp = meta["primary_class"].replace(".", "/") + ".java"
    orig_snippet = "\n\n".join(s for s, _ in blocks) or meta["buggy_source"][:1500]
    new_snippet = "\n\n".join(r for _, r in blocks) or candidate[:1500]
    cand = FixCandidate(fixed_code=new_snippet, explanation="search/replace patch",
                        confidence=0.8, changes_description="", potential_side_effects=[])
    val_inputs = [ValidationInput(original_code=orig_snippet, fix_candidate=cand,
                                  language="java", file_path=fp) for _ in am.validators]
    val_outputs = await _asyncio.gather(
        *[v.process(vi) for v, vi in zip(am.validators, val_inputs)])

    proposal = FixProposal(
        bug_id=meta["workdir"].split("/")[-1], original_code=meta["buggy_source"],
        fixed_code=candidate, explanation="", proposing_agent_id=am.healer.agent_id,
        confidence_score=0.8, language="java", file_path=fp, line_number=1)

    async def validate_for_consensus(fix, agent_id):
        for vo in val_outputs:
            if vo.agent_id == agent_id and vo.result is not None:
                return vo.result.is_valid, vo.result.confidence, "; ".join(vo.result.issues)
        return True, 0.8, "default acceptance"

    cr = await am.consensus.propose_fix(proposal, validate_fn=validate_for_consensus)
    return bool(cr.success)


async def repair_one_patch(am, openai_client, model: str, meta: Dict) -> Dict:
    """Patch-mode repair: minimal edit blocks -> validators -> consensus."""
    blocks = []
    for _ in range(2):  # one retry for transient API errors
        try:
            blocks = await healer_patch(openai_client, model, meta)
            break
        except Exception as e:
```

```
        logger.warning("healer patch call failed", error=str(e)[:120])
    candidate, applied, total = apply_blocks(meta["buggy_source"], blocks)
    if applied == 0:
        return {"consensus_approved": False, "fixed_code": None,
                "reason": f"no edit blocks applied (0/{total})"}
    approved = await consensus_vote(am, candidate, blocks, meta)
    return {"consensus_approved": approved, "fixed_code": candidate,
            "reason": f"{applied}/{total} edit blocks applied"}


# ---------------------------------------------------------------------------
#                                                                        Main
# ---------------------------------------------------------------------------
async def main(argv: Optional[List[str]] = None) -> int:
    parser = argparse.ArgumentParser(prog="python -m benchmarks.defects4j_runner",
                                     description=__doc__)
    parser.add_argument("--project", default="Lang")
    parser.add_argument("--bugs", default="1,2,3,4,5",
                        help="comma-separated Defects4J bug numbers")
    parser.add_argument("--mode", choices=["consensus", "single-agent"], default="consensus")
    parser.add_argument("--patch-mode", choices=["searchreplace", "whole"],
                        default="searchreplace",
                        help="searchreplace = minimal edit blocks (scales to large "
                             "classes); whole = regenerate the entire file")
    parser.add_argument("--f", type=int, default=1)
    parser.add_argument("--n-validators", type=int, default=None,
                        help="Independent-voter model: set = 3f+1 (f=1 -> 4, f=2 -> 7).")
    parser.add_argument("--diverse-validators", action="store_true",
                        help="Use 4 distinct Ollama model families "
                             "(llama3.1, codellama, mistral) as the validators.")
    parser.add_argument("--hetero-validators", dest="heterogeneous_validators",
                        action="store_true",
                        help="Heterogeneous cross-provider panel: Claude Haiku + "
                             "GPT-4o-mini + llama3.1 + mistral (2 cloud + 2 local), "
                             "genuine 3f+1 quorum=3. Matches the headline panel. "
                             "Overrides --diverse-validators.")
    vg = parser.add_mutually_exclusive_group()
    vg.add_argument("--independent-voters", dest="independent_voters", action="store_true",
                    help="Only validators vote (genuine 3f+1 independent replicas). Default.")
    vg.add_argument("--legacy-voters", dest="independent_voters", action="store_false",
                    help="Legacy Analyzer+Healer standing-accept model (internal only).")
    parser.set_defaults(independent_voters=True)
    parser.add_argument("--image", default="defects4j:local")
    parser.add_argument("--workspace", default=None,
                        help="host dir to bind-mount at /workspace")
    parser.add_argument("--max-src-chars", type=int, default=8000,
                        help="skip bugs whose modified class exceeds this size "
                             "(keeps the full-file round-trip within token limits)")
```

```
parser.add_argument("--max-tokens",                                type=int,                                default=8192,
                    help="per-agent       output       token       budget;       raised       for       large       "
                         "real-world                                  Java                                  classes")
parser.add_argument("--agent-timeout-s",                           type=int,                                default=600,
                    help="per-agent       LLM       timeout;       CPU-bound       Ollama       validation       "
                         "of       large       Java       classes       needs       a       long       timeout")
parser.add_argument("--output-dir",                                                                   default="benchmark_results")
parser.add_argument("--prefix",                                                                       default=None)
args                                          =                                          parser.parse_args(argv)

workspace      =      args.workspace      or      os.path.join(os.getcwd(),      "data",      "defects4j_workspace")
ensure_container(args.image,                                                                          workspace)

bug_ids       =       [int(b)       for       b       in       args.bugs.split(",")       if       b.strip()]
am                                            =                                            _build_agent_manager(args)

#           Raw           OpenAI           client           for           patch-mode           healer           calls.
from                                app.config                                import                                settings
patch_client                                                =                                                     None
if          args.patch_mode          ==          "searchreplace"          and          settings.openai_api_key          \
        and                                not                                  settings.openai_api_key.startswith("your_"):
    from                                openai                               import                                AsyncOpenAI
    patch_client                                =                              AsyncOpenAI(api_key=settings.openai_api_key)

rows:                                  List[Dict]                                     =                                  []
for                                   bug                                   in                                   bug_ids:
  try:
    started                                                 =                                                 datetime.utcnow()
    logger.info("defects4j                    bug",                    project=args.project,                    bug=bug)
    meta                                =                               checkout_bug(args.project,                               bug)
    if                                      meta                                      is                                      None:
        rows.append({"bug_id":           f"d4j-{args.project}-{bug}",           "status":           "checkout_failed"})
        continue

    #     In     whole-file     mode,     skip     classes     too     large     to     regenerate.     Patch     mode
    #      only      returns      small      edit      blocks,      so      size      is      not      a      constraint.
    if      args.patch_mode      ==      "whole"      and      len(meta["buggy_source"])      >      args.max_src_chars:
        rows.append({
            "bug_id":                                                                                 f"d4j-{args.project}-{bug}",
            "status":                                                                                         "skipped_too_large",
            "src_chars":                                                                               len(meta["buggy_source"]),
        })
        continue

    #     Sanity:     the     buggy     version's     triggering     test     should     FAIL     before     repair.
    pre_pass,             pre_detail             =             run_tests(meta["workdir"],             meta["trigger"])
```

```
    if args.patch_mode == "searchreplace":
        rep = await repair_one_patch(am, patch_client, settings.openai_model, meta)
    else:
        rep = await repair_one(am, meta, args.mode)
    fixed_code = _extract_java(rep["fixed_code"] or "", meta["buggy_source"])

    # Write the candidate into the checkout, then verify in two stages:
    #   (a) trigger test passes -> plausible patch
    #   (b) full relevant suite passes -> correct patch (no regressions)
    trigger_pass = False
    full_pass = False
    detail = "not_applied"
    if rep["consensus_approved"] and fixed_code and fixed_code.strip() != meta["buggy_source"].strip():
        host_path = os.path.join(workspace, *meta["src_path_rel"].split("/"))
        try:
            Path(host_path).write_text(fixed_code, encoding="utf-8")
            trigger_pass, detail = run_tests(meta["workdir"], meta["trigger"], full=False)
            if trigger_pass:
                full_pass, full_detail = run_tests(meta["workdir"], full=True)
                detail = f"trigger pass; full suite: {full_detail}"
        except Exception as e:
            detail = f"apply_error: {e}"

    duration_ms = (datetime.utcnow() - started).total_seconds() * 1000
    rows.append({
        "bug_id": f"d4j-{args.project}-{bug}",
        "project": args.project,
        "status": "ok",
        "primary_class": meta["primary_class"],
        "src_chars": len(meta["buggy_source"]),
        "pre_repair_tests_pass": pre_pass,  # expected False (it's the buggy version)
        "consensus_approved": rep["consensus_approved"],
        "blocks": rep.get("reason", ""),
        "trigger_test_pass": trigger_pass,  # plausible
        "full_suite_pass": full_pass,  # correct (no regression)
        "safety_violation": bool(rep["consensus_approved"] and not full_pass),
        "fix_success": bool(full_pass),  # we count ONLY full-suite passes
        "detail": detail,
        "duration_ms": round(duration_ms, 2),
    })
    logger.info("bug done", bug=bug, approved=rep["consensus_approved"],
                trigger=trigger_pass, full=full_pass)
except Exception as e:
    logger.warning("bug crashed; recording and continuing",
                   bug=bug, error=str(e)[:200])
    rows.append({"bug_id": f"d4j-{args.project}-{bug}", "status": "error",
                 "detail": str(e)[:200]})
```

```
    # Summary
    ran = [r for r in rows if r.get("status") == "ok"]
    n = len(ran) or 1
    summary = {
        "experiment": "defects4j_cross_language",
        "project": args.project,
        "mode": args.mode,
        "n_bugs_requested": len(bug_ids),
        "n_bugs_evaluated": len(ran),
        "n_skipped_too_large": sum(1 for r in rows if r.get("status") == "skipped_too_large"),
        "n_checkout_failed": sum(1 for r in rows if r.get("status") == "checkout_failed"),
        "trigger_pass_rate": round(sum(1 for r in ran if r.get("trigger_test_pass")) / n, 4),
        "fix_success_rate": round(sum(1 for r in ran if r["fix_success"]) / n, 4),
        "consensus_approval_rate": round(sum(1 for r in ran if r["consensus_approved"]) / n, 4),
        "safety_violation_rate": round(sum(1 for r in ran if r["safety_violation"]) / n, 4),
        "consensus": am.consensus_config(),
    }

    os.makedirs(args.output_dir, exist_ok=True)
    prefix = args.prefix or f"d4j_{args.project}_{args.mode}"
    with open(os.path.join(args.output_dir, f"{prefix}__cases.csv"), "w", newline="", encoding="utf-8") as fh:
        # union of keys across rows
        fields = sorted({k for r in rows for k in r.keys()})
        w = csv.DictWriter(fh, fieldnames=fields)
        w.writeheader()
        w.writerows(rows)
    with open(os.path.join(args.output_dir, f"{prefix}__summary.json"), "w", encoding="utf-8") as fh:
        json.dump({"summary": summary, "cases": rows}, fh, indent=2)

    print("\n" + "=" * 60)
    print(f"DEFECTS4J CROSS-LANGUAGE RUN COMPLETE: {args.project} ({args.mode})")
    print("=" * 60)
    print(json.dumps(summary, indent=2))
    return 0


if __name__ == "__main__":
    import sys
    sys.exit(asyncio.run(main()))
```